

비트코인 자동투자봇 개발용

AI Agent 최적화 첨부본

Claude / Codex App이 바로 작업 지시로 이해하기 쉽도록 재구성한 버전

이 문서는 원본 선별 PDF를 AI 코드 에이전트에게 더 잘 전달하기 위한 앞부분 지침을 추가한 것입니다. 뒤쪽에는 기존 선별 PDF 전체가 부록으로 이어집니다.

구분	내용
주요 목적	BTC 현물 자동투자봇의 설계, 백테스트, 데이터 저장, 리스크 관리, 모의투자, 실행 엔진 구현에 필요한 작업 지침 제공
에이전트 사용법	이 앞부분을 먼저 읽고, 뒤쪽 원문 선별 페이지는 근거와 참고자료로 사용
중요 원칙	주식/ETF/Interactive Brokers 예제를 그대로 복사하지 말고 BTC 거래소 API와 24시간 365일 시장에 맞게 변환
주의	특정 매수/매도 추천이나 수익 보장이 아니라 기술 설계와 검증 참고용

1. 에이전트가 먼저 이해해야 할 핵심

목표는 가격을 맞추는 예측기가 아니라 검증 가능한 BTC 현물 자동투자 시스템입니다. 에이전트는 전략 수익률보다 데이터 정확도, 백테스트 현실성, 비용 모델, 리스크 관리, 모의투자 안전성을 우선해야 합니다.

- 데이터 수집 및 저장: OHLCV, 실시간 체결, 호가, 주문, 체결, 잔고, API 오류 로그를 분리 저장합니다.
- 백테스트: 과거 데이터를 한 번에 보지 않고 시간순으로 흘러보내 look-ahead bias를 막습니다.
- 비용 모델: 거래소 maker/taker 수수료, 스프레드, 슬리피지, 최소 주문금액, 부분체결을 반영합니다.
- 전략: DCA를 기준 전략으로 두고 이동평균 필터, 변동성 필터, 예측 모델은 단계적으로 붙입니다.
- 리스크: 최대 일일 손실, 최대 누적 낙폭, 과매매 제한, API 실패 횟수, 수동 킬스위치를 둡니다.
- 성과 측정: 총수익률보다 BTC 단순보유 대비 초과수익, MDD, Sharpe/Sortino, 수수료 총액을 함께 봅니다.
- 최적화: 가장 좋은 파라미터 하나가 아니라 여러 기간에서 안정적인 파라미터 영역을 찾습니다.

2. 기본 가정

항목	기본값
자산/방식	BTC 현물, 레버리지 없음, 숏 포지션 없음
거래 모드	dry-run 또는 paper trading 기본값. 실거래는 명시적 허용 전까지 비활성화
거래소	사용자가 지정하지 않았다면 ExchangeAdapter 인터페이스만 설계하고 특정 거래소 구현은 분리
벤치마크	현금 보유, BTC 단순보유, 단순 DCA
MVP 전략	DCA + 장기 이동평균 필터 + 변동성 기반 매수금액 조절
스택	기존 저장소 우선. 새 프로젝트라면 Python + SQLite/PostgreSQL + pytest 제안

3. 비목표와 금지사항

- 원문 PDF의 주식, ETF, Interactive Brokers 예제를 그대로 붙여 넣지 않습니다.
- Yahoo Finance, S&P500, SPY, AAPL 중심 코드는 BTC/KRW 또는 BTC/USDT 거래소 데이터 구조로 바꿉니다.
- 252거래일 기준 연환산을 BTC 24시간 365일 시장에 그대로 적용하지 않습니다.
- 수익률만 보고 전략을 선택하지 않습니다.
- 수수료, 스프레드, 슬리피지, 최소 주문금액, 부분체결 없는 백테스트 결과를 실거래 가능 결과로 취급하지 않습니다.
- API 키를 코드, 로그, 문서, 테스트 fixture에 하드코딩하지 않습니다.
- 실거래 주문 기능은 dry-run, idempotency, 주문 중복 방지, kill switch, 잔고 검증이 없으면 구현 완료로 보지 않습니다.

4. 권장 아키텍처

계층	역할
DataHandler	거래소 캔들, 체결, 호가, 잔고, 과거 데이터 iterator 제공
Strategy	DCA, 이동평균 필터, 변동성 필터, 예측 보조필터가 SignalEvent 생성
Portfolio	현금/BTC 잔고, 평가금액, 포지션 크기, 주문 가능 금액 계산
RiskManager	일일 손실, MDD, API 실패, 과매매, kill switch, 중복 주문 차단
ExecutionHandler	paper execution 또는 live exchange adapter를 통해 OrderEvent를 FillEvent로 변환
Backtest	Market -> Signal -> Order -> Fill 이벤트 루프 실행 및 성과 계산

핵심 흐름: MarketEvent -> Strategy -> SignalEvent -> Portfolio/Risk -> OrderEvent -> Execution -> FillEvent -> Portfolio Update

5. 추천 파일 구조

```

btc-bot/
  config/settings.example.yml
  src/data/exchange_data_handler.py
  src/data/storage.py
  src/data/validators.py
  src/events/event.py
  src/strategy/base.py
  src/strategy/dca.py
  src/strategy/ma_filter.py
  src/portfolio/portfolio.py
  src/risk/risk_manager.py
  src/execution/exchange_adapter.py
  src/execution/paper_execution.py
  src/execution/live_execution.py    # 기본 비활성화
  src/backtest/engine.py
  src/backtest/cost_model.py
  src/backtest/metrics.py
  tests/test_no_lookahead.py
  tests/test_cost_model.py
  tests/test_risk_manager.py
  tests/test_order_idempotency.py

```

백테스트와 실거래가 같은 전략 코드, 같은 포트폴리오 코드, 같은 리스크 코드를 사용하도록 구성합니다. 차이는 DataHandler와 ExecutionHandler만 교체하는 방식이 가장 안전합니다.

6. 개발 순서

Phase	목표
1	저장소 구조 분석, 거래소/기준통화/투자주기/주문방식 확인, 없으면 인터페이스 우선 설계
2	OHLCV, 체결, 호가, 주문, 잔고, API 오류 로그 저장 및 데이터 품질 검증
3	시간순/event-driven 백테스트, 비용 모델, 부분체결 시뮬레이션 구현
4	현금 100%, BTC 단순보유, 단순 DCA, DCA+이평 필터 벤치마크 구현
5	수익률, BTC 보유 대비 초과수익, MDD, Sharpe/Sortino, 수수료 총액 리포트
6	일일 손실 제한, MDD 정지, API 실패 정지, 중복 주문 방지, kill switch
7	Grid Search, walk-forward, 구간별 검증, 과최적화 방지
8	paper trading 검증 후 소액 실거래 준비. 실거래 기본 OFF

7. 테스트 요구사항

테스트	검증 내용
test_no_lookahead	전략이 미래 bar를 읽지 못하는지 검증
test_cost_model	maker/taker 수수료, 스프레드, 슬리피지가 반영되는지 검증
test_min_order_size	최소 주문금액 미만 주문 차단 검증
test_partial_fill	부분체결 후 잔고와 평균단가 검증
test_api_failure_halts_trading	API 오류 연속 발생 시 신규 주문 중단 검증
test_kill_switch	kill switch가 켜졌을 때 모든 신규 주문 차단 검증
test_order_idempotency	같은 신호 반복 처리 시 중복 주문 방지 검증
test_metrics	MDD, Sharpe, Sortino, 수수료 총액 계산 검증

8. 완료 기준

- 실행 가능한 테스트 명령이 제공됩니다.
- 백테스트 결과가 수수료/슬리피지 반영 전후로 분리되어 표시됩니다.
- BTC 단순보유, 단순 DCA와 비교됩니다.
- 리스크 중단 조건이 실제 주문 생성 전에 적용됩니다.
- 주문/체결/잔고 로그가 재현 가능하게 저장됩니다.
- 실거래 기능은 명시적 설정 없이는 켜지지 않습니다.
- API 키와 민감정보가 저장소에 포함되지 않습니다.
- 남은 가정과 위험을 숨기지 않고 보고합니다.

9. Claude 첨부용 프롬프트

아래 문장을 Claude에 PDF/MD와 함께 붙여 넣으면, 단순 요약이 아니라 프로젝트 설계/구현 지시로 받아들여 만들 수 있습니다.

첨부한 PDF/MD는 비트코인 자동투자봇 개발을 위한 AI Agent 최적화 참고문서다.

목표:

- 원문 주식/ETF 예제를 그대로 복사하지 말고, BTC 현물 자동투자봇에 맞게 변환한다.
- 우선순위는 전략 수익률보다 데이터 정확도, 이벤트 기반 백테스트, 비용 모델, 리스크 관리, 모의투자 안전성이다.

작업 방식:

1. 먼저 현재 저장소 구조를 분석한다.
2. 이미 있는 파일/설계를 존중해서 최소 변경으로 개선한다.
3. 거래소가 명시되어 있지 않으면 exchange adapter 인터페이스부터 만든다.
4. 실거래 주문은 기본 비활성화하고 dry-run/paper trading을 우선 구현한다.
5. 수수료, 스프레드, 슬리피지, 최소 주문금액, 부분체결 없는 백테스트는 완료로 보지 않는다.
6. 결과물은 설계 요약, 변경 파일 목록, 테스트 방법, 남은 리스크 순서로 보고한다.

먼저 구현하지 말고, 이 문서 기준으로 현재 프로젝트에 부족한 부분과 1차 구현 계획을 제시해라.

10. Codex App 첨부/저장소용 프롬프트

Codex App에는 PDF보다 Markdown을 저장소의 docs 또는 루트에 두는 방식이 더 유리합니다. 이 PDF와 함께 제공되는 BTC_BOT_AGENT_CONTEXT.md 파일을 우선 사용하세요.

Read `BTC\_BOT\_AGENT\_CONTEXT.md` first. Treat it as the project-level implementation guide.

Task:

Inspect the repository and propose a minimal, safe implementation plan for a BTC spot auto-investment bot.

Hard requirements:

- Do not enable live trading by default.
- Implement dry-run or paper trading first.
- Keep exchange-specific code behind an ExchangeAdapter interface.
- Backtesting must be event-driven or time-sequential to avoid look-ahead bias.
- Include fees, spread/slippage, minimum order size, and partial fill handling in the simulation model.
- Add risk guards: daily loss limit, max drawdown halt, API failure halt, duplicate order prevention, manual kill switch.
- Add tests for no-lookahead, cost model, risk guards, idempotency, and metrics.
- Do not hardcode API keys or secrets.

Output:

1. Brief architecture summary.
2. Files to create or modify.
3. Implementation steps in small commits.
4. Tests to add.
5. Any assumptions or missing information.

Start by reading the existing codebase. Do not make large rewrites unless necessary.

11. 원본 선별 PDF 사용 지도

파트	원본 PDF 페이지	에이전트 적용 포인트
백테스트 현실화	24-29	편향, 주문 타입, 수수료, 슬리피지, 시장충격 반영
자동 실행 구조	30-36	연구용 백테스터와 이벤트 기반 백테스터, VPS/서버 실행
전략 평가와 데이터 조건	43-46	수익률보다 Sharpe, MDD, 벤치마크 중심 평가
데이터 저장/정확도/자동화	56-60	DB 저장, 데이터 품질, 자동화 구조
데이터 빈도와 호가/체결	72-74, 83	1분봉, 틱, 오더북 저장 판단
시계열 검정	96-100	ADF, Hurst를 추세/횡보 판단 보조 지표로 사용
예측 모델	106-108, 112-115	직접 매매 신호보다 보조 필터로 사용
성과 측정	118-127	Equity curve, Sharpe, Sortino, CALMAR, MDD
리스크와 자금관리	128-135	전략 리스크, 운영 리스크, Kelly, VaR를 보수적으로 적용
이벤트 기반 엔진	138-164	Market/Signal/Order/Fill 및 핵심 컴포넌트 구조
전략 구현 예시	172-181, 188-189	이동평균 예시와 성과 그래프를 BTC 데이터로 변환
파라미터 최적화	190-207	Grid Search, cross validation, overfitting 방지

이 다음 페이지부터 기존 원본 선별 PDF 부록이 시작됩니다.

앞부분의 AI Agent 지침을 우선 적용하고, 뒤쪽 원문 선별 페이지는 세부 근거와 참고 자료로 사용하세요.

특히 원문 예제의 주식/ETF/Interactive Brokers 코드는 그대로 쓰지 말고, BTC/KRW 또는 BTC/USDT 거래소 API, 24시간 365일 거래, 소수점 수량, 거래소별 수수료 체계에 맞게 변환해야 합니다.

비트코인 자동투자봇 개발용 알고리즘 트레이딩 핵심 페이지 선별본

원본: Successful Algorithmic Trading - Michael L. Halls-Moore

이 PDF는 원본 전체를 그대로 복사한 것이 아니라, 비트코인 자동투자봇 개발에 직접 활용하기 좋은 파트만 선별해 묶은 문서입니다. 앞쪽에는 한국어 활용 가이드를 넣었고, 그 뒤에 원본 PDF의 선별 페이지가 이어집니다.

페이지 번호는 PDF 뷰어 기준의 원본 PDF 페이지 번호입니다. 책 내부 하단에 인쇄된 페이지 번호와 다를 수 있습니다.

주의: 이 자료는 자동투자봇 설계와 검증을 위한 기술 참고용입니다. 특정 매수/매도 추천이나 수익 보장을 의미하지 않습니다.

선별 기준	BTC 봇에서 직접 필요한 시스템 설계, 백테스트, 데이터, 성과측정, 리스크, 이벤트 기반 실행, 최적화 내용을 우선 포함했습니다.
제외 기준	주식 전용 브로커 설정, Interactive Brokers 세부 연결, S&P500 구성종목 스크래핑, 선물 연속계약처럼 BTC 단일 현물 자동투자에 직접성이 낮은 내용은 대부분 제외했습니다.
추천 활용법	먼저 10번 이벤트 기반 엔진을 봇 구조로 삼고, 1/8/9/12번 파트로 검증과 리스크 관리 규칙을 붙인 뒤, 11번의 이동평균 예시를 BTC 데이터로 바꿔 실험하세요.

선별 페이지 목차

구분	원본 PDF 페이지	우선순위	BTC 봇 적용 포인트
1. 백테스트 현실화	24-29	필수	BTC 봇에서는 백테스트를 수익률 확인용이 아니라 탈락 필터로 사용하고, 거래소 수수료/스프레드/슬리피지/부분체결을 반드시 반영.
2. 자동 실행 구조	30-36	필수	거래소 API 연결, 24시간 실행, VPS/서버 장애 대응, 신호-주문-체결 로그 구조 설계에 사용.
3. 전략 평가와 데이터 조건	43-46	필수	BTC 단순보유, 현금 보유, 경기매수(DCA)를 벤치마크로 두고 봇 전략의 초과 가치를 비교.
4. 데이터 저장/정확도/자동화	56-60	필수	거래소 캔들/체결/호가/주문/잔고/에러 로그를 DB에 저장하고, 누락/이상치/중복 체결을 검증.
5. 데이터 빈도와 호가/체결 데이터	72-74, 83	필수	1분봉/5분봉/1시간봉뿐 아니라 실시간 체결과 order book 데이터를 저장할 지 결정하는 기준으로 사용.
6. 시계열 검증: 평균회귀/추세 판단	96-100	선택	BTC 단일자산이 추세장인지 횡보장인지 판단하고, DCA 금액 조절 또는 추세 필터 조건에 활용.
7. 예측 모델과 분류기 성능평가	106-108, 112-115	선택	상승/하락 예측 모델을 직접 매매 신호로 쓰기보다 매수 금액 축소/확대 보조 필터로 사용.
8. 성과 측정	118-127	필수	봇 대시보드에 총수익률, BTC 보유 대비 초과수익, MDD, 수수료 총액, 거래 횟수, 월별 수익률을 포함.
9. 리스크와 자금관리	128-135	필수	일일 손실 제한, 최대 낙폭 제한, 거래소/API 장애, 키 유출, 중복 주문, 급락장 정지 조건 설계에 사용.
10. 이벤트 기반 트레이딩 엔진	138-164	최우선	백테스트와 실거래가 같은 코드 경로를 타게 만드는 핵심 구조. BTC 봇의 뼈대로 활용.
11. 전략 구현 예시와 성과 그래프	172-181, 188-189	선택	DCA + 이동평균 필터, 장기 추세 필터, 성과 리포트 생성 코드의 출발점으로 활용.
12. 파라미터 최적화와 과최적화 방지	190-207	필수	이동평균 기간, 매수 주기, 변동성 기준, 진입/청산 기준을 최적화하되 구간별 검증과 walk-forward 검증으로 과최적화 방지.

비트코인 봇에 적용할 핵심 설계 체크리스트

- 데이터 계층: OHLCV, 실시간 체결, 호가, 주문, 체결, 잔고, API 오류 로그를 분리 저장한다.
- 백테스트 계층: 과거 데이터를 한 번에 보지 않고 시간순으로 흘려보내 look-ahead bias를 막는다.
- 비용 모델: 거래소 maker/taker 수수료, 스프레드, 슬리피지, 최소 주문금액, 부분체결을 반영한다.
- 전략 계층: DCA를 기본 전략으로 두고 이동평균 필터, 변동성 필터, 예측 모델은 단계적으로 붙인다.
- 리스크 계층: 최대 일일 손실, 최대 누적 낙폭, 과매매 제한, API 실패 횟수, 수동 킬스위치를 둔다.
- 성과 계층: 총수익률보다 BTC 단순보유 대비 초과수익, MDD, Sharpe/Sortino, 수수료 총액을 함께 본다.
- 최적화 계층: 가장 좋은 파라미터 하나가 아니라 여러 기간에서 안정적인 파라미터 영역을 찾는다.

이후 원문 선별 페이지가 이어집니다. 원문 예제의 주식/ETF/Interactive Brokers 코드는 그대로 쓰지 말고, BTC/KRW 또는 BTC/USDT 거래소 API, 24시간 365일 거래, 소수점 수량, 거래소별 수수료 체계에 맞게 바꿔야 합니다.

원문 선별 범위 상세

1. 백테스트 현실화 | 원본 PDF 페이지 24-29

선별 이유: 전략 검증 목적, 백테스트 편향, 주문 타입, 수수료, 슬리피지, 시장충격, 실거래 괴리를 다룸.

BTC 적용: BTC 봇에서는 백테스트를 수익률 확인용이 아니라 탈락 필터로 사용하고, 거래소 수수료/스프레드/슬리피지/부분체결을 반드시 반영.

2. 자동 실행 구조 | 원본 PDF 페이지 30-36

선별 이유: 백테스팅 플랫폼, 연구용 백테스터와 이벤트 기반 백테스터, 지연시간, 실행 환경, VPS 배포를 다룸.

BTC 적용: 거래소 API 연결, 24시간 실행, VPS/서버 장애 대응, 신호-주문-체결 로그 구조 설계에 사용.

3. 전략 평가와 데이터 조건 | 원본 PDF 페이지 43-46

선별 이유: 전략을 수익률만이 아니라 Sharpe, 변동성, 승률/손익비, MDD, 유동성, 파라미터, 벤치마크로 평가하는 기준.

BTC 적용: BTC 단순보유, 현금 보유, 정기매수(DCA)를 벤치마크로 두고 봇 전략의 초과가치를 비교.

4. 데이터 저장/정확도/자동화 | 원본 PDF 페이지 56-60

선별 이유: 데이터 저장 형식, 관계형 DB, 과거 데이터 구조, 데이터 정확도 평가, 자동화 개념.

BTC 적용: 거래소 캔들/체결/호가/주문/잔고/에러 로그를 DB에 저장하고, 누락/이상치/중복 체결을 검증.

5. 데이터 빈도와 호가/체결 데이터 | 원본 PDF 페이지 72-74, 83

선별 이유: 일봉, 인트라데이 바, 틱, 오더북 데이터와 데이터 품질 문제를 설명.

BTC 적용: 1분봉/5분봉/1시간봉뿐 아니라 실시간 체결과 order book 데이터를 저장할지 결정하는 기준으로 사용.

6. 시계열 검정: 평균회귀/추세 판단 | 원본 PDF 페이지 96-100

선별 이유: ADF 테스트와 Hurst Exponent로 평균회귀/랜덤워크/추세 성향을 판별하는 방법.

BTC 적용: BTC 단일자산이 추세장인지 횡보장인지 판단하고, DCA 금액 조절 또는 추세 필터 조건에 활용.

7. 예측 모델과 분류기 성능평가 | 원본 PDF 페이지 106-108, 112-115

선별 이유: Hit rate, Confusion matrix, lagged return/volume feature, 예측 모델 구현과 결과 해석.

BTC 적용: 상승/하락 예측 모델을 직접 매매 신호로 쓰기보다 매수 금액 축소/확대 보조 필터로 사용.

8. 성과 측정 | 원본 PDF 페이지 118-127

선별 이유: 거래 분석, 수익률, Equity curve, Sharpe, Sortino, CALMAR, MDD와 드로우다운 기간.

BTC 적용: 봇 대시보드에 총수익률, BTC 보유 대비 초과수익, MDD, 수수료 총액, 거래 횟수, 월별 수익률을 포함.

9. 리스크와 자금관리 | 원본 PDF 페이지 128-135

선별 이유: 전략 리스크, 포트폴리오 리스크, 거래상대방 리스크, 운영 리스크, Kelly, VaR.

BTC 적용: 일일 손실 제한, 최대 낙폭 제한, 거래소/API 장애, 키 유출, 중복 주문, 급락장 정지 조건 설계에 사용.

10. 이벤트 기반 트레이딩 엔진 | 원본 PDF 페이지 138-164

선별 이유: Market/Signal/Order/Fill 이벤트, DataHandler, Strategy, Portfolio, ExecutionHandler, Backtest 구조.

BTC 적용: 백테스트와 실거래가 같은 코드 경로를 타게 만드는 핵심 구조. BTC 봇의 뼈대로 활용.

11. 전략 구현 예시와 성과 그래프 | 원본 PDF 페이지 172-181, 188-189

선별 이유: 이동평균 교차 전략, 예측 기반 전략, 성과 그래프 출력 예시.

BTC 적용: DCA + 이동평균 필터, 장기 추세 필터, 성과 리포트 생성 코드의 출발점으로 활용.

12. 파라미터 최적화와 과최적화 방지 | 원본 PDF 페이지 190-207

선별 이유: 파라미터 최적화, 과최적화, 교차검증, Grid Search, 성과 heatmap.

BTC 적용: 이동평균 기간, 매수 주기, 변동성 기준, 진입/청산 기준을 최적화하되 구간별 검증과 walk-forward 검증으로 과최적화 방지.

이 다음 페이지부터 원본 PDF 선별 페이지가 시작됩니다.

필요한 부분만 빠르게 찾아볼 수 있도록 PDF 북마크도 함께 구성했습니다.

Chapter 3

Successful Backtesting

Algorithmic backtesting requires knowledge of many areas, including psychology, mathematics, statistics, software development and market/exchange microstructure. I couldn't hope to cover all of those topics in one chapter, so I'm going to split them into two or three smaller pieces. What will we discuss in this section? I'll begin by defining backtesting and then I will describe the basics of how it is carried out. Then I will elucidate upon the biases we touched upon in previous chapters.

In subsequent chapters we will look at the details of strategy implementations that are often barely mentioned or ignored elsewhere. We will also consider how to make the backtesting process more realistic by including the idiosyncrasies of a *trading exchange*. Then we will discuss transaction costs and how to correctly model them in a backtest setting. We will end with a discussion on the *performance* of our backtests and finally provide detailed examples of common quant strategies.

Let's begin by discussing what backtesting is and why we should carry it out in our algorithmic trading.

3.1 Why Backtest Strategies?

Algorithmic trading stands apart from other types of investment classes because we can more reliably provide expectations about future performance from past performance, as a consequence of abundant data availability. The process by which this is carried out is known as **backtesting**.

In simple terms, backtesting is carried out by exposing your particular strategy algorithm to a stream of historical financial data, which leads to a set of **trading signals**. Each *trade* (which we will mean here to be a 'round-trip' of two signals) will have an associated profit or loss. The accumulation of this profit/loss over the duration of your strategy backtest will lead to the total profit and loss (also known as the 'P & L' or 'PnL'). That is the essence of the idea, although of course the "devil is always in the details"!

What are key reasons for backtesting an algorithmic strategy?

- **Filtration** - If you recall from the previous chapter on Strategy Identification, our goal at the initial research stage was to set up a strategy pipeline and then filter out any strategy that did not meet certain criteria. Backtesting provides us with another filtration mechanism, as we can eliminate strategies that do not meet our performance needs.
- **Modelling** - Backtesting allows us to (safely!) test new models of certain market phenomena, such as transaction costs, order routing, latency, liquidity or other *market microstructure* issues.
- **Optimisation** - Although *strategy optimisation* is fraught with biases, backtesting allows us to increase the performance of a strategy by modifying the quantity or values of the parameters associated with that strategy and recalculating its performance.

- **Verification** - Our strategies are often sourced externally, via our *strategy pipeline*. Backtesting a strategy ensures that it has not been incorrectly implemented. Although we will rarely have access to the signals generated by external strategies, we will often have access to the performance metrics such as the Sharpe Ratio and Drawdown characteristics. Thus we can compare them with our own implementation.

Backtesting provides a host of advantages for algorithmic trading. However, it is not always possible to straightforwardly backtest a strategy. In general, as the frequency of the strategy increases, it becomes harder to correctly model the microstructure effects of the market and exchanges. This leads to less reliable backtests and thus a trickier evaluation of a chosen strategy. This is a particular problem where the execution system is the key to the strategy performance, as with ultra-high frequency algorithms.

Unfortunately, backtesting is fraught with biases of all types and we will now discuss them in depth.

3.2 Backtesting Biases

There are many biases that can affect the performance of a backtested strategy. Unfortunately, these biases have a tendency to inflate the performance rather than detract from it. Thus you should always consider a backtest to be an idealised upper bound on the actual performance of the strategy. It is almost impossible to eliminate biases from algorithmic trading so it is our job to minimise them as best we can in order to make informed decisions about our algorithmic strategies.

There are four major biases that I wish to discuss: *Optimisation Bias*, *Look-Ahead Bias*, *Survivorship Bias* and *Cognitive Bias*.

3.2.1 Optimisation Bias

This is probably the most insidious of all backtest biases. It involves adjusting or introducing additional trading parameters until the strategy performance on the backtest data set is very attractive. However, once live the performance of the strategy can be markedly different. Another name for this bias is "curve fitting" or "data-snooping bias".

Optimisation bias is hard to eliminate as algorithmic strategies often involve many parameters. "Parameters" in this instance might be the entry/exit criteria, look-back periods, averaging periods (i.e the moving average smoothing parameter) or volatility measurement frequency. Optimisation bias can be minimised by keeping the number of parameters to a minimum and increasing the quantity of data points in the training set. In fact, one must also be careful of the latter as older training points can be subject to a prior *regime* (such as a regulatory environment) and thus may not be relevant to your current strategy.

One method to help mitigate this bias is to perform a *sensitivity analysis*. This means varying the parameters incrementally and plotting a "surface" of performance. Sound, fundamental reasoning for parameter choices should, with all other factors considered, lead to a smoother parameter surface. If you have a very jumpy performance surface, it often means that a parameter is not reflecting a phenomena and is an artefact of the test data. There is a vast literature on multi-dimensional optimisation algorithms and it is a highly active area of research. I won't dwell on it here, but keep it in the back of your mind when you find a strategy with a fantastic backtest!

3.2.2 Look-Ahead Bias

Look-ahead bias is introduced into a backtesting system when future data is accidentally included at a point in the simulation where that data would not have actually been available. If we are running the backtest chronologically and we reach time point N , then look-ahead bias occurs if data is included for any point $N + k$, where $k > 0$. Look-ahead bias errors can be incredibly subtle. Here are three examples of how look-ahead bias can be introduced:

- **Technical Bugs** - Arrays/vectors in code often have iterators or index variables. Incorrect *offsets* of these indices can lead to a look-ahead bias by incorporating data at $N + k$ for non-zero k .
- **Parameter Calculation** - Another common example of look-ahead bias occurs when calculating optimal strategy parameters, such as with linear regressions between two time series. If the whole data set (including future data) is used to calculate the regression coefficients, and thus retroactively applied to a trading strategy for optimisation purposes, then future data is being incorporated and a look-ahead bias exists.
- **Maxima/Minima** - Certain trading strategies make use of extreme values in any time period, such as incorporating the high or low prices in OHLC data. However, since these maximal/minimal values can only be calculated at the end of a time period, a look-ahead bias is introduced if these values are used -during- the current period. It is always necessary to lag high/low values by at least one period in any trading strategy making use of them.

As with optimisation bias, one must be extremely careful to avoid its introduction. It is often the main reason why trading strategies underperform their backtests significantly in "live trading".

3.2.3 Survivorship Bias

Survivorship bias is a particularly dangerous phenomenon and can lead to significantly inflated performance for certain strategy types. It occurs when strategies are tested on datasets that do not include the full universe of prior assets that may have been chosen at a particular point in time, but only consider those that have "survived" to the current time.

As an example, consider testing a strategy on a random selection of equities before and after the 2001 market crash. Some technology stocks went bankrupt, while others managed to stay afloat and even prospered. If we had restricted this strategy only to stocks which made it through the market drawdown period, we would be introducing a survivorship bias because they have already demonstrated their success to us. In fact, this is just another specific case of look-ahead bias, as future information is being incorporated into past analysis.

There are two main ways to mitigate survivorship bias in your strategy backtests:

- **Survivorship Bias Free Datasets** - In the case of equity data it is possible to purchase datasets that include delisted entities, although they are not cheap and only tend to be utilised by institutional firms. In particular, Yahoo Finance data is NOT survivorship bias free, and this is commonly used by many retail algo traders. One can also trade on asset classes that are not prone to survivorship bias, such as certain commodities (and their future derivatives).
- **Use More Recent Data** - In the case of equities, utilising a more recent data set mitigates the possibility that the stock selection chosen is weighted to "survivors", simply as there is less likelihood of overall stock delisting in shorter time periods. One can also start building a personal survivorship-bias free dataset by collecting data from current point onward. After 3-4 years, you will have a solid survivorship-bias free set of equities data with which to backtest further strategies.

We will now consider certain psychological phenomena that can influence your trading performance.

3.2.4 Cognitive Bias

This particular phenomena is not often discussed in the context of *quantitative* trading. However, it is discussed extensively in regard to more discretionary trading methods. When creating backtests over a period of 5 years or more, it is easy to look at an upwardly trending equity curve, calculate the compounded annual return, Sharpe ratio and even drawdown characteristics and be satisfied with the results. As an example, the strategy might possess a maximum relative

drawdown of 25% and a maximum drawdown duration of 4 months. This would not be atypical for a momentum strategy. It is straightforward to convince oneself that it is easy to tolerate such periods of losses because the overall picture is rosy. However, in practice, it is far harder!

If historical drawdowns of 25% or more occur in the backtests, then in all likelihood you will see periods of similar drawdown in live trading. These periods of drawdown are psychologically difficult to endure. I have observed first hand what an extended drawdown can be like, in an institutional setting, and it is not pleasant - even if the backtests suggest such periods will occur. The reason I have termed it a "bias" is that often a strategy which would otherwise be successful is stopped from trading during times of extended drawdown and thus will lead to significant underperformance compared to a backtest. Thus, even though the strategy is algorithmic in nature, psychological factors can still have a heavy influence on profitability. The takeaway is to ensure that if you see drawdowns of a certain percentage and duration in the backtests, then you should expect them to occur in live trading environments, and will need to persevere in order to reach profitability once more.

3.3 Exchange Issues

3.3.1 Order Types

One choice that an algorithmic trader must make is how and when to make use of the different *exchange orders* available. This choice usually falls into the realm of the *execution system*, but we will consider it here as it can greatly affect strategy backtest performance. There are two types of order that can be carried out: **market orders** and **limit orders**.

A market order executes a trade immediately, irrespective of available prices. Thus large trades executed as market orders will often get a mixture of prices as each subsequent limit order on the opposing side is filled. Market orders are considered *aggressive* orders since they will almost certainly be filled, albeit with a potentially unknown cost.

Limit orders provide a mechanism for the strategy to determine the worst price at which the trade will get executed, with the caveat that the trade may not get filled partially or fully. Limit orders are considered *passive* orders since they are often unfilled, but when they are a price is guaranteed. An individual exchange's collection of limit orders is known as the **limit order book**, which is essentially a queue of buy and sell orders at certain sizes and prices.

When backtesting, it is essential to model the effects of using market or limit orders correctly. For high-frequency strategies in particular, backtests can significantly outperform live trading if the effects of market impact and the limit order book are not modelled accurately.

3.3.2 Price Consolidation

There are particular issues related to backtesting strategies when making use of daily data in the form of Open-High-Low-Close (OHLC) figures, especially for equities. Note that this is precisely the form of data given out by Yahoo Finance, which is a very common source of data for retail algorithmic traders!

Cheap or free datasets, while suffering from survivorship bias (which we have already discussed above), are also often composite price feeds from multiple exchanges. This means that the extreme points (i.e. the open, close, high and low) of the data are very susceptible to "outlying" values due to small orders at regional exchanges. Further, these values are also sometimes more likely to be tick-errors that have yet to be removed from the dataset.

This means that if your trading strategy makes extensive use of any of the OHLC points specifically, backtest performance can differ from live performance as orders might be routed to different exchanges depending upon your broker and your available access to liquidity. The only way to resolve these problems is to make use of higher frequency data or obtain data directly from an individual exchange itself, rather than a cheaper composite feed.

3.3.3 Forex Trading and ECNs

The backtesting of foreign exchange strategies is somewhat trickier to implement than that of equity strategies. Forex trading occurs across multiple venues and Electronic Communication Networks (ECN). The bid/ask prices achieved on one venue can differ substantially from those on another venue. One must be extremely careful to make use of pricing information from the particular venue you will be trading on in the backtest, as opposed to a consolidated feed from multiple venues, as this will be significantly more indicative of the prices you are likely to achieve going forward.

Another idiosyncrasy of the foreign exchange markets is that brokers themselves are not obligated to share *trade* prices/sizes with every trading participant, since this is their proprietary information[6]. Thus it is more appropriate to use bid-ask quotes in your backtests and to be extremely careful of the variation of transaction costs between brokers/venues.

3.3.4 Shorting Constraints

When carrying out short trades in the backtest it is necessary to be aware that some equities may not have been available (due to the lack of availability in that stock to borrow) or due to a market constraint, such as the US SEC banning the shorting of financial stocks during the 2008 market crisis.

This can severely inflate backtesting returns so be careful to include such short sale constraints within your backtests, or avoid shorting at all if you believe there are likely to be liquidity constraints in the instruments you trade.

3.4 Transaction Costs

One of the most **prevalent beginner mistakes** when implementing trading models is to neglect (or grossly underestimate) the effects of *transaction costs* on a strategy. Though it is often assumed that transaction costs only reflect broker commissions, there are in fact many other ways that costs can be accrued on a trading model. The three main types of costs that must be considered include:

3.4.1 Commission

The most direct form of transaction costs incurred by an algorithmic trading strategy are commissions and fees. All strategies require some form of access to an *exchange*, either directly or through a brokerage intermediary ("the broker"). These services incur an incremental cost with each trade, known as *commission*.

Brokers generally provide many services, although quantitative algorithms only really make use of the exchange infrastructure. Hence brokerage commissions are often small on per trade basis. Brokers also charge *fees*, which are costs incurred to clear and settle trades. Further to this are *taxes* imposed by regional or national governments. For instance, in the UK there is a *stamp duty* to pay on equities transactions. Since commissions, fees and taxes are generally fixed, they are relatively straightforward to implement in a backtest engine (see below).

3.4.2 Slippage

Slippage is the difference in price achieved between the time when a trading system decides to transact and the time when a transaction is actually carried out at an exchange. Slippage is a considerable component of transaction costs and can make the difference between a very profitable strategy and one that performs poorly. Slippage is a function of the underlying asset volatility, the latency between the trading system and the exchange and the type of strategy being carried out.

An instrument with higher volatility is more likely to be moving and so prices between signal and execution can differ substantially. Latency is defined as the time difference between signal generation and point of execution. Higher frequency strategies are more sensitive to latency

issues and improvements of milliseconds on this latency can make all the difference towards profitability. The type of strategy is also important. Momentum systems suffer more from slippage on average because they are trying to purchase instruments that are already moving in the forecast direction. The opposite is true for mean-reverting strategies as these strategies are moving in a direction opposing the trade.

3.4.3 Market Impact

Market impact is the cost incurred to traders due to the supply/demand dynamics of the exchange (and asset) through which they are trying to trade. A large order on a relatively illiquid asset is likely to *move the market* substantially as the trade will need to access a large component of the current supply. To counter this, large block trades are broken down into smaller "chunks" which are transacted periodically, as and when new liquidity arrives at the exchange. On the opposite end, for highly liquid instruments such as the S&P500 E-Mini index futures contract, low volume trades are unlikely to adjust the "current price" in any great amount.

More illiquid assets are characterised by a larger **spread**, which is the difference between the current bid and ask prices on the **limit order book**. This spread is an additional transaction cost associated with any trade. Spread is a very important component of the total transaction cost - as evidenced by the myriad of UK spread-betting firms whose advertising campaigns express the "tightness" of their spreads for heavily traded instruments.

3.5 Backtesting vs Reality

In summary there are a staggering array of factors that can be simulated in order to generate a realistic backtest. The dangers of overfitting, poor data cleansing, incorrect handling of transaction costs, market regime change and trading constraints often lead to a backtest performance that differs substantially from a live strategy deployment.

Thus one must be very aware that future performance is very unlikely to match historical performance directly. We will discuss these issues in further detail when we come to implement an event-driven backtesting engine near the end of the book.

Chapter 4

Automated Execution

Automated execution is the process of letting the strategy automatically generate execution signals that are sent to the broker without any human intervention. This is the purest form of algorithmic trading strategy, as it minimises issues due to human intervention. It is the type of system that we will consider most often during this book.

4.1 Backtesting Platforms

The software landscape for strategy backtesting is vast. Solutions range from fully-integrated institutional grade sophisticated software through to programming languages such as C++, Python and R where nearly everything must be written from scratch (or suitable 'plugins' obtained). As quant traders we are interested in the balance of being able to "own" our trading technology stack versus the speed and reliability of our development methodology. Here are the key considerations for software choice:

- **Programming Skill** - The choice of environment will in a large part come down to your ability to program software. I would argue that being in control of the total stack will have a greater effect on your long term PnL than outsourcing as much as possible to vendor software. This is due to the downside risk of having external bugs or idiosyncrasies that you are unable to fix in vendor software, which would otherwise be easily remedied if you had more control over your "tech stack". You also want an environment that strikes the right balance between productivity, library availability and speed of execution. I make my own personal recommendation below.
- **Execution Capability/Broker Interaction** - Certain backtesting software, such as Tradestation, ties in directly with a brokerage. I am not a fan of this approach as reducing transaction costs are often a big component of getting a higher Sharpe ratio. If you're tied into a particular broker (and Tradestation "forces" you to do this), then you will have a harder time transitioning to new software (or a new broker) if the need arises. Interactive Brokers provide an API which is robust, albeit with a slightly obtuse interface.
- **Customisation** - An environment like MATLAB or Python gives you a great deal of flexibility when creating algo strategies as they provide fantastic libraries for nearly any mathematical operation imaginable, but also allow extensive customisation where necessary.
- **Strategy Complexity** - Certain software just isn't cut out for heavy number crunching or mathematical complexity. Excel is one such piece of software. While it is good for simpler strategies, it cannot really cope with numerous assets or more complicated algorithms, at speed.
- **Bias Minimisation** - Does a particular piece of software or data lend itself more to trading biases? You need to make sure that if you want to create all the functionality yourself,

that you don't introduce bugs which can lead to biases. An example here is look-ahead bias, which Excel minimises, while a vectorised research backtester might lend itself to accidentally.

- **Speed of Development** - One shouldn't have to spend months and months implementing a backtest engine. Prototyping should only take a few weeks. Make sure that your software is not hindering your progress to any great extent, just to grab a few extra percentage points of execution speed.
- **Speed of Execution** - If your strategy is completely dependent upon execution timeliness (as in HFT/UHFT) then a language such as C or C++ will be necessary. However, you will be verging on Linux kernel optimisation and FPGA usage for these domains, which is outside the scope of the book.
- **Cost** - Many of the software environments that you can program algorithmic trading strategies with are completely free and open source. In fact, many hedge funds make use of open source software for their entire algo trading stacks. In addition, Excel and MATLAB are both relatively cheap and there are even free alternatives to each.

Different strategies will require different software packages. HFT and UHFT strategies will be written in C/C++. These days such strategies are often carried out on GPUs and FPGAs. Conversely, low-frequency directional equity strategies are easy to implement in TradeStation, due to the "all in one" nature of the software/brokerage.

4.1.1 Programming

Custom development of a backtesting language within a first-class programming language provides the most flexibility when testing a strategy. Conversely, a vendor-developed integrated backtesting platform will always have to make assumptions about how backtests are carried out. The choice of available programming languages is large and diverse. It is not obvious before development which language would be suitable.

Once a strategy has been codified into systematic rules it is necessary to backtest it in such a manner that the quantitative trader is confident that its future performance will be reflective of its past performance. There are generally two forms of backtesting system that are utilised to test this hypothesis. Broadly, they are categorised as *research back testers* and *event-driven back testers*.

4.1.2 Research Tools

The simpler form of a backtesting tool, the research tool, is usually considered first. The research tool is used to quickly ascertain whether a strategy is likely to have any performance. Such tools often make unrealistic assumptions about transaction costs, likely fill prices, shorting constraints, venue dependence, risk management and a host of other issues that were outlined in the previous chapter. Common tools for research include MATLAB, R, Python and Excel.

The research stage is useful because the software packages provide significant *vectorised* capability, which leads to good execution speed and straightforward implementation (less lines of code). Thus it is possible to test multiple strategies, combinations and variants in a rapid, iterative manner.

While such tools are often used for both backtesting and execution, such research environments are generally not suitable for strategies that approach intraday trading at higher frequencies (sub-minute). This is because these environments do not often possess the necessary libraries to connect to real-time market data vendor servers or can interface with brokerage APIs in a clean fashion.

Despite these executional shortcomings, research environments are heavily used within the professional quantitative environment. They are the "firs test" for all strategy ideas before promoting them to a more rigorous check within a realistic backtesting environment.

4.1.3 Event-Driven Backtesting

Once a strategy has been deemed suitable on a research basis it must be tested in a more realistic fashion. Such realism attempts to account for the majority (if not all) of the issues described in the previous chapter. The ideal situation is to be able to use the same trade generation code for historical backtesting as well as live execution. This can be achieved using an *event-driven backtester*.

Event-driven systems are widely used in software engineering, commonly for handling graphical user interface (GUI) input within window-based operating systems. They are also ideal for algorithmic trading. Such systems are often written in high-performance languages such as C++, C# and Java.

Consider a situation where an automated trading strategy is connected to a real-time market feed and a broker (these two may be one and the same). New market information will be sent to the system, which triggers an *event* to generate a new trading signal and thus an execution *event*. Thus such a system is in a continuous loop waiting to receive events and handle them appropriately.

It is possible to generate sub-components such as a historic data handler and brokerage simulator, which can mimic their live counterparts. This allows backtesting strategies in a manner that is extremely similar to live execution.

The disadvantage of such systems is that they are far more complicated to design and implement than a simpler research tool. Thus the "time to market" is longer. They are more prone to bugs and require a reasonable knowledge of programming and, to a degree, software development methodology.

4.1.4 Latency

In engineering terms, *latency* is defined as the time interval between a simulation and a response. For our purposes it will generally refer to the round-trip time delay between the generation of an execution signal and the receipt of the fill information from a broker that is carrying out the execution.

Such latency is rarely an issue on low-frequency interday strategies since the likely price movement during the latency period will not affect the strategy to any great extent. Unfortunately, the same is not true of higher-frequency strategies. At these frequencies latency becomes important. The ultimate goal is to reduce latency as much as possible in order to minimise *slippage*, as discussed in the previous chapter.

Decreasing latency involves minimising the "distance" between the algorithmic trading system and the ultimate exchange on which an order is being executed. This can involve shortening the geographic distance between systems (and thus reducing travel down network cabling), reducing the processing carried out in networking hardware (important in HFT strategies) or choosing a brokerage with more sophisticated infrastructure.

Decreasing latency becomes exponentially more expensive as a function of "internet distance" (i.e. the network distance between two servers). Thus, for a high-frequency trader, a compromise must be reached between expenditure of latency-reduction vs the gain from minimising slippage. These issues will be discussed in the section on *Colocation* below.

4.1.5 Language Choices

Some issues that drive language choice have already been outlined. Now we will consider the benefits and drawbacks of individual programming languages. I have broadly categorised the languages into high-performance/harder development vs lower-performance/easier development. These are subjective terms and some will disagree depending upon their background.

One of the most important aspects of programming a custom backtesting environment is that the programmer is familiar with the tools being used. That is probably a more important criterion than speed of development. However, for those that are new to the programming language landscape, the following should clarify what tends to be utilised within algorithmic trading.

C++, C# and Java

C++, C# and Java are all examples of *general purpose* object-oriented programming languages. That is, they can be used without a corresponding IDE, are all cross-platform (can be run on Windows, Mac OSX or Linux), have a wide range of libraries for nearly any imaginable task and possess have rapid execution speed.

If ultimate execution speed is desired then C++ (or C) is likely to be the best choice. It offers the most flexibility for managing memory and optimising execution speed. This flexibility comes at a price. C++ is notoriously tricky to learn well and can often lead to subtle bugs. Development time can take much longer than in other languages.

C# and Java are similar in that they both require all components to be objects, with the exception of primitive data types such as floats and integers. They differ from C++ in that they both perform automatic *garbage collection*. This means that memory does not have to be manually de-allocated upon destruction of an object. Garbage collection adds a performance overhead but it makes development substantially more rapid. These languages are both good choices for developing a backtester as they have native GUI capabilities, numerical analysis libraries and fast execution speed.

Personally, I would make use of C++ for creating an event-driven backtester that needs extremely rapid execution speed, such as for HFT. This is only if I feel that a Python event-driven system was becoming a bottleneck, as the latter language would be my first choice for such a system.

MATLAB, R and Python

MATLAB is a commercial integrated development environment (IDE) for numerical computation. It has gained wide acceptance in the academic, engineering and financial sectors. It has a huge array of numerical libraries for many scientific computing tasks and possesses a rapid execution speed, assuming that any algorithm being developed is subject to *vectorisation* or *parallelisation*. It is expensive and this sometimes makes it less appealing to retail traders on a budget. MATLAB is sometimes used for direct execution through to a brokerage such as Interactive Brokers.

R is less of a general purpose programming language and more of a statistics scripting environment. It is free, open-source, cross-platform and contains a huge array of freely-available statistical packages for carrying out extremely advanced analysis. R is very widely used in the quantitative hedge fund industry for statistical/strategy research. While it is possible to connect R to a brokerage, is not well suited to the task and should be considered more of a research tool. In addition, it lacks execution speed unless operations are vectorised.

I've grouped Python under this heading, although it sits somewhere between MATLAB, R and the aforementioned general-purpose languages. It is free, open-source and cross-platform. It is *interpreted* as opposed to *compiled*, which makes it natively slower than C++. Despite this it contains a library for carrying out nearly any task imaginable, from scientific computing through to low-level web server design. In particular, it contains NumPy, SciPy, pandas, matplotlib and scikit-learn, which provide a robust numerical research environment that, when vectorised, is comparable to compiled language execution speed.

Further, it has mature libraries for connecting to brokerages. This makes it a "one-stop shop" for creating an event-driven backtesting and live execution environment without having to step into other languages. Execution speed is more than sufficient for intraday traders trading on the time scale of minutes. Finally, it is very straightforward to pick up and learn, when compared to lower-level languages like C++. For these reasons we make extensive use of Python within this book.

4.1.6 Integrated Development Environments

The term IDE has multiple meanings within algorithmic trading. Software developers use it to mean a GUI that allows programming with syntax highlighting, file browsing, debugging and code execution features. Algorithmic traders use it to mean a fully-integrated backtesting/trading environment, including historical or real-time data download, charting, statistical evaluation and

live execution. For our purposes, I use the term to mean any environment (often GUI-based) that is *not* a general purpose programming language, such as C++ or Python. MATLAB is considered an IDE, for instance.

Excel

While some purer quants may look down on Excel, I have found it to be extremely useful for "sanity checking" of results. The fact that all of the data is directly available, rather than hidden behind objects, makes it straightforward to implement very basic signal/filter strategies. Brokerages, such as Interactive Brokers, also allow DDE plugins that allow Excel to receive real-time market data and execute trading orders.

Despite the ease of use, Excel is extremely slow for any reasonable scale of data or level of numerical computation. I only use it to error-check when developing against other strategies and to make sure I've avoided look-ahead bias, which is easy to see in Excel due to the spreadsheet nature of the software.

If you are uncomfortable with programming languages and are carrying out an interday strategy, then Excel may be the perfect choice.

Commercial/Retail Backtesting Software

The market for retail charting, "technical analysis" and backtesting software is extremely competitive. Features offered by such software include real-time charting of prices, a wealth of technical indicators, customised backtesting languages and automated execution.

Some vendors provide an all-in-one solution, such as TradeStation. TradeStation are an online brokerage who produce trading software (also known as TradeStation) that provides electronic order execution across multiple asset classes. I don't currently believe that they offer a direct API for automated execution, rather orders must be placed through the software. This is in contrast to Interactive Brokers, who have a more stripped down charting/trading interface (Trader WorkStation), but offer both their proprietary real-time market/order execution APIs and a FIX interface.

Another extremely popular platform is MetaTrader, which is used in foreign exchange trading for creating 'Expert Advisors'. These are custom scripts written in a proprietary language that can be used for automated trading. I haven't had much experience with either TradeStation or MetaTrader so I won't spend too much time discussing their merits.

Such tools are useful if you are not comfortable with in depth software development and wish a lot of the details to be taken care of. However, with such systems a lot of flexibility is sacrificed and you are often tied to a single brokerage.

Web-Based Tools

The two current popular web-based backtesting systems are Quantopian (<https://www.quantopian.com/>) and QuantConnect (<https://www.quantconnect.com/>). The former makes use of Python (and ZipLine, see below) while the latter utilises C#. Both provide a wealth of historical data. Quantopian currently supports live trading with Interactive Brokers, while QuantConnect is working towards live trading.

Open Source Backtesting Software

In addition to the commercial offerings, there are open source alternatives for backtesting software.

Algo-Trader is a Swiss-based firm that offer both an open-source and a commercial license for their system. From what I can gather the offering seems quite mature and they have many institutional clients. The system allows full historical backtesting and *complex event processing* and they tie into Interactive Brokers. The Enterprise edition offers substantially more high performance features.

Marketcetera provide a backtesting system that can tie into many other languages, such as Python and R, in order to leverage code that you might have already written. The 'Strategy

Studio' provides the ability to write backtesting code as well as optimised execution algorithms and subsequently transition from a historical backtest to live paper trading.

ZipLine is the Python library that powers the Quantopian service mentioned above. It is a fully event-driven backtest environment and currently supports US equities on a minutely-bar basis. I haven't made extensive use of ZipLine, but I know others who feel it is a good tool. There are still many areas left to improve, but the team are constantly working on the project so it is very actively maintained.

There are also some Github/Google Code hosted projects that you may wish to look into. I have not spent any great deal of time investigating them. Such projects include OpenQuant (<http://code.google.com/p/openquant/>), TradeLink (<https://code.google.com/p/tradelink/>) and PyAlgoTrade (<http://gbeced.github.io/pyalgotrade/>).

Institutional Backtesting Software

Institutional-grade backtesting systems, such as Deltix and QuantHouse, are not often utilised by retail algorithmic traders. The software licenses are generally well outside the budget for infrastructure. That being said, such software is widely used by quant funds, proprietary trading houses, family offices and the like.

The benefits of such systems are clear. They provide an all-in-one solution for data collection, strategy development, historical backtesting and live execution across single instruments or portfolios, up to the ultra-high frequency level. Such platforms have had extensive testing and plenty of "in the field" usage and so are considered robust.

The systems are event-driven and as such the backtesting environment can often simulate the live environment well. The systems also support optimised execution algorithms, which attempt to minimise transaction costs.

I have to admit that I have not had much experience of Deltix or QuantHouse beyond some cursory overviews. That being said, the budget alone puts them out of reach of most retail traders, so I won't dwell on these systems.

4.2 Colocation

The software landscape for algorithmic trading has now been surveyed. It is now time to turn attention towards implementation of the hardware that will execute our strategies.

A retail trader will likely be executing their strategy from home during market hours, turning on their PC, connecting to the brokerage, updating their market software and then allowing the algorithm to execute automatically during the day. Conversely, a professional quant fund with significant *assets under management* (AUM) will have a dedicated exchange-colocated server infrastructure in order to reduce latency as far as possible to execute their high speed strategies.

4.2.1 Home Desktop

The simplest approach to hardware deployment is simply to carry out an algorithmic strategy with a home desktop computer connected to the brokerage via a broadband (or similar) connection.

While this approach is straightforward to get started it does suffer from many drawbacks. Primarily, the desktop machine is subject to power failure, unless backed up by a UPS. In addition, a home internet connection is also at the mercy of the ISP. Power loss or internet connectivity failure could occur at a crucial moment in trading, leaving the algorithmic trader with open positions that are unable to be closed.

Secondly, a desktop machine must occasionally be restarted, often due to the reliability of the operating system. This means that the strategy suffers from a degree of indirect manual intervention. If this occurs outside of trading hours the problem is mitigated. However, if a computer needs a restart during trading hours the problem is similar to a power loss. Unclosed positions may still be subject to risk.

Component failure also leads to the same set of "downtime" problems. A failure in the hard disk, monitor or motherboard often occurs at precisely the wrong time. For all of these reasons

I hesitate to recommend a home desktop approach to algorithmic trading. If you do decide to pursue this approach, make sure to have both a backup computer AND a backup internet connection (e.g. a 3G dongle) that you can use to close out positions under a downtime situation.

4.2.2 VPS

The next level up from a home desktop is to make use of a **virtual private server** (VPS). A VPS is a remote server system often marketed as a "cloud" service. They are far cheaper than a corresponding dedicated server, since a VPS is actually a partition of a much larger server, with a virtual isolated operating system environment solely available to you. CPU load is shared between multiple servers and a portion of the systems RAM is allocated to the VPS.

Common VPS providers include Amazon EC2 and Rackspace Cloud. They provide entry-level systems with low RAM and basic CPU usage, through to enterprise-ready high RAM, high CPU servers. For the majority of algorithmic retail traders, the entry level systems suffice for low-frequency intraday or interday strategies and smaller historical data databases.

The benefits of a VPS-based system include 24/7 availability (with a certain realistic downtime!), more robust monitoring capabilities, easy "plugins" for additional services, like file storage or managed databases and a flexible architecture. The drawbacks include expense as the system grows, since dedicated hardware becomes far cheaper per performance, assuming colocation away from an exchange, as well as handling failure scenarios (i.e. by creating a second identical VPS, for instance).

In addition, latency is not always improved by choosing a VPS/cloud provider. Your home location may be closer to a particular financial exchange than the data centres of your cloud provider. This is somewhat mitigated by choosing a firm that provide VPS geared specifically for algorithmic trading which are located at or near exchanges, however these will likely cost more than a "traditional" VPS provider such as Amazon or Rackspace.

4.2.3 Exchange

In order to get the best latency minimisation and fastest systems, it is necessary to colocate a dedicated server (or set of servers) directly at the exchange data centre. This is a prohibitively expensive option for nearly all retail algorithmic traders (unless they're very well capitalised). It is really the domain of the professional quantitative fund or brokerage.

As I mentioned above, a more realistic option is to purchase a VPS system from a provider that is located near an exchange. I won't dwell too heavily on exchange colocation, as the topic is somewhat outside the scope of the book.

By continuing to monitor the above sources on a weekly, or even daily, basis you are setting yourself up to receive a consistent list of strategies from a diverse range of sources. The next step is to determine how to reject a large subset of these strategies in order to minimise wasting your time and backtesting resources on strategies that are likely to be unprofitable.

5.3 Evaluating Trading Strategies

The first, and arguably most obvious consideration is whether you actually **understand the strategy**. Would you be able to explain the strategy concisely or does it require a string of caveats and endless parameter lists? In addition, does the strategy have a good, solid basis in reality? For instance, could you point to some behavioural rationale or fund structure constraint that might be causing the pattern(s) you are attempting to exploit? Would this constraint hold up to a regime change, such as a dramatic regulatory environment disruption? Does the strategy rely on complex statistical or mathematical rules? Does it apply to any financial time series or is it specific to the asset class that it is claimed to be profitable on? You should constantly be thinking about these factors when evaluating new trading methods, otherwise you may waste a significant amount of time attempting to backtest and optimise unprofitable strategies.

Once you have determined that you understand the basic principles of the strategy you need to decide whether it fits with your aforementioned personality profile. This is not as vague a consideration as it sounds! Strategies will differ substantially in their performance characteristics. There are certain personality types that can handle more significant periods of drawdown, or are willing to accept greater risk for larger return. Despite the fact that we, as quants, try and eliminate as much cognitive bias as possible and should be able to evaluate a strategy dispassionately, biases will always creep in. Thus we need a consistent, unemotional means through which to assess the performance of strategies. Here is the list of criteria that I judge a potential new strategy by:

- **Methodology** - Is the strategy momentum based, mean-reverting, market-neutral, directional? Does the strategy rely on sophisticated (or complex!) statistical or machine learning techniques that are hard to understand and require a PhD in statistics to grasp? Do these techniques introduce a significant quantity of parameters, which might lead to optimisation bias? Is the strategy likely to withstand a *regime change* (i.e. potential new regulation of financial markets)?
- **Sharpe Ratio** - The Sharpe ratio heuristically characterises the reward/risk ratio of the strategy. It quantifies how much return you can achieve for the level of volatility endured by the equity curve. Naturally, we need to determine the period and frequency that these returns and volatility (i.e. standard deviation) are measured over. A higher frequency strategy will require greater sampling rate of standard deviation, but a shorter overall time period of measurement, for instance.
- **Leverage** - Does the strategy require significant leverage in order to be profitable? Does the strategy necessitate the use of leveraged derivatives contracts (futures, options, swaps) in order to make a return? These leveraged contracts can have heavy volatility characteristics and thus can easily lead to *margin calls*. Do you have the trading capital and the temperament for such volatility?
- **Frequency** - The frequency of the strategy is intimately linked to your technology stack (and thus technological expertise), the Sharpe ratio and overall level of transaction costs. All other issues considered, higher frequency strategies require more capital, are more sophisticated and harder to implement. However, assuming your backtesting engine is sophisticated and bug-free, they will often have far higher Sharpe ratios.
- **Volatility** - Volatility is related strongly to the "risk" of the strategy. The Sharpe ratio characterises this. Higher volatility of the underlying asset classes, if unhedged, often leads to higher volatility in the equity curve and thus smaller Sharpe ratios. I am of course assuming that the positive volatility is approximately equal to the negative volatility. Some strategies may have greater downside volatility. You need to be aware of these attributes.

- **Win/Loss, Average Profit/Loss** - Strategies will differ in their win/loss and average profit/loss characteristics. One can have a very profitable strategy, even if the number of losing trades exceed the number of winning trades. Momentum strategies tend to have this pattern as they rely on a small number of "big hits" in order to be profitable. Mean-reversion strategies tend to have opposing profiles where more of the trades are "winners", but the losing trades can be quite severe.
- **Maximum Drawdown** - The maximum drawdown is the largest overall peak-to-trough percentage drop on the equity curve of the strategy. Momentum strategies are well known to suffer from periods of extended drawdowns (due to a string of many incremental losing trades). Many traders will give up in periods of extended drawdown, even if historical testing has suggested this is "business as usual" for the strategy. You will need to determine what percentage of drawdown (and over what time period) you can accept before you cease trading your strategy. This is a highly personal decision and thus must be considered carefully.
- **Capacity/Liquidity** - At the retail level, unless you are trading in a highly illiquid instrument (like a small-cap stock), you will not have to concern yourself greatly with strategy *capacity*. Capacity determines the scalability of the strategy to further capital. Many of the larger hedge funds suffer from significant capacity problems as their strategies increase in capital allocation.
- **Parameters** - Certain strategies (especially those found in the machine learning community) require a large quantity of parameters. Every extra parameter that a strategy requires leaves it more vulnerable to optimisation bias (also known as "curve-fitting"). You should try and target strategies with as few parameters as possible or make sure you have sufficient quantities of data with which to test your strategies on.
- **Benchmark** - Nearly all strategies (unless characterised as "absolute return") are measured against some performance benchmark. The benchmark is usually an *index* that characterises a large sample of the underlying asset class that the strategy trades in. If the strategy trades large-cap US equities, then the S&P500 would be a natural benchmark to measure your strategy against. You will hear the terms "alpha" and "beta", applied to strategies of this type.

Notice that we have not discussed the actual *returns* of the strategy. Why is this? In isolation, the returns actually provide us with limited information as to the effectiveness of the strategy. They don't give you an insight into leverage, volatility, benchmarks or capital requirements. Thus strategies are rarely judged on their returns alone. Always consider the risk attributes of a strategy before looking at the returns.

At this stage many of the strategies found from your pipeline will be rejected out of hand, since they won't meet your capital requirements, leverage constraints, maximum drawdown tolerance or volatility preferences. The strategies that do remain can now be considered for *backtesting*. However, before this is possible, it is necessary to consider one final rejection criteria - that of available historical data on which to test these strategies.

5.4 Obtaining Historical Data

Nowadays, the breadth of the technical requirements across asset classes for historical data storage is substantial. In order to remain competitive, both the buy-side (funds, prop-desks) and sell-side (broker/dealers) invest heavily in their technical infrastructure. It is imperative to consider its importance. In particular, we are interested in timeliness, accuracy and storage requirements. We will be discussing data storage in later chapters of the book.

In the previous section we had set up a strategy pipeline that allowed us to reject certain strategies based on our own personal rejection criteria. In this section we will filter more strategies based on our own preferences for obtaining historical data. The chief considerations (especially at retail practitioner level) are the costs of the data, the storage requirements and your level of

technical expertise. We also need to discuss the different types of available data and the different considerations that each type of data will impose on us.

Let's begin by discussing the types of data available and the key issues we will need to think about, with the understanding that we will explore these issues in significant depth in the remainder of the book:

- **Fundamental Data** - This includes data about macroeconomic trends, such as interest rates, inflation figures, corporate actions (dividends, stock-splits), SEC filings, corporate accounts, earnings figures, crop reports, meteorological data etc. This data is often used to value companies or other assets on a *fundamental* basis, i.e. via some means of expected future cash flows. It does not include stock price series. Some fundamental data is freely available from government websites. Other long-term historical fundamental data can be extremely expensive. Storage requirements are often not particularly large, unless thousands of companies are being studied at once.
- **News Data** - News data is often qualitative in nature. It consists of articles, blog posts, microblog posts ("tweets") and editorial. Machine learning techniques such as *classifiers* are often used to interpret *sentiment*. This data is also often freely available or cheap, via subscription to media outlets. The newer "NoSQL" document storage databases are designed to store this type of unstructured, qualitative data.
- **Asset Price Data** - This is the traditional data domain of the quant. It consists of time series of asset prices. Equities (stocks), fixed income products (bonds), commodities and foreign exchange prices all sit within this class. Daily historical data is often straightforward to obtain for the simpler asset classes, such as equities. However, once accuracy and cleanliness are included and statistical biases removed, the data can become expensive. In addition, time series data often possesses significant storage requirements especially when intraday data is considered.
- **Financial Instruments** - Equities, bonds, futures and the more exotic derivative options have very different characteristics and parameters. Thus there is no "one size fits all" database structure that can accommodate them. Significant care must be given to the design and implementation of database structures for various financial instruments.
- **Frequency** - The higher the frequency of the data, the greater the costs and storage requirements. For low-frequency strategies, daily data is often sufficient. For high frequency strategies, it might be necessary to obtain tick-level data and even historical copies of particular trading exchange *order book* data. Implementing a storage engine for this type of data is very technologically intensive and only suitable for those with a strong programming/technical background.
- **Benchmarks** - The strategies described above will often be compared to a *benchmark*. This usually manifests itself as an additional financial time series. For equities, this is often a national stock benchmark, such as the S&P500 index (US) or FTSE100 (UK). For a fixed income fund, it is useful to compare against a basket of bonds or fixed income products. The "risk-free rate" (i.e. appropriate interest rate) is also another widely accepted benchmark. All asset class categories possess a favoured benchmark, so it will be necessary to research this based on your particular strategy, if you wish to gain interest in your strategy externally.
- **Technology** - The technology stacks behind a financial data storage centre are complex. However, it does generally centre around a database cluster engine, such as a Relational Database Management System (RDBMS), such as MySQL, SQL Server, Oracle or a Document Storage Engine (i.e. "NoSQL"). This is accessed via "business logic" application code that queries the database and provides access to external tools, such as MATLAB, R or Excel. Often this business logic is written in C++, Java or Python. You will also need to host this data somewhere, either on your own personal computer, or remotely via internet servers. Products such as Amazon Web Services have made this simpler and cheaper in

recent years, but it will still require significant technical expertise to achieve in a robust manner.

As can be seen, once a strategy has been identified via the pipeline it will be necessary to evaluate the availability, costs, complexity and implementation details of a particular set of historical data. You may find it is necessary to reject a strategy based solely on historical data considerations. This is a big area and teams of PhDs work at large funds making sure pricing is accurate and timely. Do not underestimate the difficulties of creating a robust data centre for your backtesting purposes!

I do want to say, however, that many backtesting platforms can provide this data for you automatically - at a cost. Thus it will take much of the implementation pain away from you, and you can concentrate purely on strategy implementation and optimisation. Tools like TradeStation possess this capability. However, my personal view is to implement as much as possible internally and avoid outsourcing parts of the stack to software vendors. I prefer higher frequency strategies due to their more attractive Sharpe ratios, but they are often tightly coupled to the technology stack, where advanced optimisation is critical.

Chapter 7

Financial Data Storage

In algorithmic trading the spotlight usually shines on the *alpha model* component of the full trading system. This component generates the trading signals, prior to filtration by a risk management and portfolio construction system. As such, algo traders often spend a significant portion of their research time refining the alpha model in order to optimise one or more metrics prior to deployment of the strategy into production.

However, an alpha model is only as good as the data being fed into it. This concept is nicely characterised by the old computer science adage of "*garbage in, garbage out.*" It is absolutely crucial that accurate, timely data is used to feed the alpha model. Otherwise results will be at best poor or at worst completely incorrect. This will lead to significant underperformance when system is deployed live.

In this chapter we will discuss issues surrounding the acquisition and provision of timely accurate data for an algorithmic strategy backtesting system and ultimately a trading execution engine. In particular we will study how to obtain financial data and how to store it. Subsequent chapters will discuss how to clean it and how to export it. In the financial industry this type of data service is known as a **securities master database**.

7.1 Securities Master Databases

A securities master is an organisation-wide database that stores **fundamental, pricing** and **transactional** data for a variety of financial instruments across asset classes. It provides access to this information in a consistent manner to be used by other departments such as risk management, clearing/settlement and proprietary trading.

In large organisations a range of instruments and data will be stored. Here are some of the instruments that might be of interest to a firm:

- Equities
- Equity Options
- Indices
- Foreign Exchange
- Interest Rates
- Futures
- Commodities
- Bonds - Government and Corporate
- Derivatives - Caps, Floors, Swaps

Securities master databases often have teams of developers and data specialists ensuring *high availability* within a financial institution. While this is necessary in large companies, at the retail level or in a small fund a securities master can be far simpler. In fact, while large securities masters make use of expensive enterprise database and analysis systems, it is possibly to use commodity open-source software to provide the same level of functionality, assuming a well-optimised system.

7.2 Financial Datasets

For the retail algorithmic trader or small quantitative fund the most common data sets are end-of-day and intraday historical pricing for equities, indices, futures (mainly commodities or fixed income) and foreign exchange (forex). In order to simplify this discussion we will concentrate solely on end-of-day (EOD) data for equities, ETFs and equity indices. Later sections will discuss adding higher frequency data, additional asset classes and derivatives data, which have more advanced requirements.

EOD data for equities is easy to obtain. There are a number of services providing access for free via web-available APIs:

- **Yahoo Finance** - <http://finance.yahoo.com>
- **Google Finance** - <https://www.google.com/finance>
- **QuantQuote** - <https://www.quantquote.com> (S&P500 EOD data only)
- **EODData** - <http://eoddata.com> (requires registration)

It is straightforward to manually download historical data for individual securities but it becomes time-consuming if many stocks need to be downloaded daily. Thus an important component of our securities master will be automatically updating the data set.

Another issue is *look-back period*. How far in the past do we need to go with our data? This will be specific to the requirements of your trading strategy, but there are certain problems that span all strategies. The most common is **regime change**, which is often characterised by a new regulatory environment, periods of higher/lower volatility or longer-term trending markets. For instance a long-term short-directional trend-following/momentum strategy would likely perform very well from 2000-2003 or 2007-2009. However it would have had a tough time from 2003-2007 or 2009 to the present.

My rule of thumb is to obtain as much data as possible, especially for EOD data where storage is cheap. Just because the data exists in your security master, does not mean it must be utilised. There are caveats around performance, as larger database tables mean longer query times (see below), but the benefits of having more sample points generally outweighs any performance issues.

As with all financial data it is imperative to be aware of errors, such as incorrect high/low prices or **survivorship bias**, which I have discussed at length in previous chapters.

7.3 Storage Formats

There are three main ways to store financial data. They all possess varying degrees of access, performance and structural capabilities. We will consider each in turn.

7.3.1 Flat-File Storage

The simplest data store for financial data, and the way in which you are likely to receive the data from any data vendors, is the flat-file format. Flat-files often make use of the Comma-Separated Variable (CSV) format, which store a two-dimensional matrix of data as a series of rows, with column data separated via a delimiter (often a comma, but can be whitespace, such as a space or tab). For EOD pricing data, each row represents a trading day via the OHLC paradigm (i.e. the prices at the open, high, low and close of the trading period).

The advantage of flat-files are their simplicity and ability to be heavily compressed for archiving or download. The main disadvantages lie in their lack of query capability and poor performance for iteration across large datasets. **SQLite** and **Excel** mitigate some of these problems by providing certain querying capabilities.

7.3.2 Document Stores/NoSQL

Document stores/NoSQL databases, while certainly not a new concept, have gained significant prominence in recent years due to their use at "web-scale" firms such as Google, Facebook and Twitter. They differ substantially from RDBMS systems in that there is no concept of table schemas. Instead, there are *collections* and *documents*, which are the closest analogies to tables and records, respectively. A wide taxonomy of document stores exist, the discussion of which is well outside this chapter! However, some of the more popular stores include **MongoDB**, **Cassandra** and **CouchDB**.

Document stores, in financial applications, are mostly suited to fundamental or meta data. Fundamental data for financial assets comes in many forms, such as corporate actions, earnings statements, SEC filings etc. Thus the schema-less nature of NoSQL DBs is well-suited. However, NoSQL DBs are not well designed for time-series such as high-resolution pricing data and so we won't be considering them further in this chapter.

7.3.3 Relational Database Management Systems

A *relational database management system* (RDBMS) makes use of the *relational model* to store data. These databases are particular well-suited to financial data because different "objects" (such as exchanges, data sources, prices) can be separated into tables with relationships defined between them.

RDBMS make use of **Structured Query Language** (SQL) in order to perform complex data queries on financial data. Examples of RDBMS include **Oracle**, **MySQL**, **SQLServer** and **PostgreSQL**.

The main advantages of RDBMS are their simplicity of installation, platform-independence, ease of querying, ease of integration with major backtest software and high-performance capabilities at large scale (although some would argue the latter is not the case!). Their disadvantages are often due to the complexity of customisation and difficulties of achieving said performance without underlying knowledge of how RDBMS data is stored. In addition, they possess semi-rigid schemas and so data often has to be modified to fit into such designs. This is unlike NoSQL data stores, where there is no schema.

For all of the future historical pricing implementation code in the book we will make use of the MySQL RDBMS. It is free, open-source, cross-platform, highly robust and its behaviour at scale is well-documented, which makes it a sensible choice for quant work.

7.4 Historical Data Structure

There is a significant body of theory and academic research carried out in the realm of computer science for the optimal design for data stores. However, we won't be going into too much detail as it is easy to get lost in minutiae! Instead I will present a common pattern for the construction of an equities security master, which you can modify as you see fit for your own applications.

The first task is to define our *entities*, which are elements of the financial data that will eventually map to tables in the database. For an equities master database I foresee the following entities:

- **Exchanges** - What is the ultimate original source of the data?
- **Vendor** - Where is a particular data point obtained from?
- **Instrument/Ticker** - The ticker/symbol for the equity or index, along with corporate information of the underlying firm or fund.

- **Price** - The actual price for a particular security on a particular day.
- **Corporate Actions** - The list of all stock splits or dividend adjustments (this may lead to one or more tables), necessary for adjusting the pricing data.
- **National Holidays** - To avoid mis-classifying trading holidays as missing data errors, it can be useful to store national holidays and cross-reference.

There are significant issues with regards to storing canonical tickers. I can attest to this from first hand experience at a hedge fund dealing with this exact problem! Different vendors use different methods for resolving tickers and thus combining multiple sources for accuracy. Further, companies become bankrupt, are exposed to M&A activity (i.e. become acquired and change names/symbols) and can have multiple publicly traded share classes. Many of you will not have to worry about this because your universe of tickers will be limited to the larger index constituents (such as the S&P500 or FTSE350).

7.5 Data Accuracy Evaluation

Historical pricing data from vendors is prone to many forms of error:

- **Corporate Actions** - Incorrect handling of stock splits and dividend adjustments. One must be absolutely sure that the formulae have been implemented correctly.
- **Spikes** - Pricing points that greatly exceed certain historical volatility levels. One must be careful here as these spikes do occur - see the May Flash Crash for a scary example. Spikes can also be caused by not taking into account stock splits when they do occur. *Spike filter* scripts are used to notify traders of such situations.
- **OHLC Aggregation** - Free OHLC data, such as from Yahoo/Google is particular prone to 'bad tick aggregation' situations where smaller exchanges process small trades well above the 'main' exchange prices for the day, thus leading to over-inflated maxima/minima once aggregated. This is less an 'error' as such, but more of an issue to be wary of.
- **Missing Data** - Missing data can be caused by lack of trades in a particular time period (common in second/minute resolution data of illiquid small-caps), by trading holidays or simply an error in the exchange system. Missing data can be *padded* (i.e. filled with the previous value), *interpolated* (linearly or otherwise) or ignored, depending upon the trading system.

Many of these errors rely on manual judgement in order to decide how to proceed. It is possible to automate the notification of such errors, but it is much harder to automate their solution. For instance, one must choose the threshold for being told about spikes - how many standard deviations to use and over what look-back period? Too high a stdev will miss some spikes, but too low and many unusual news announcements will lead to false positives. All of these issues require advanced judgement from the quant trader.

It is also necessary to decide how to fix errors. Should errors be corrected as soon as they're known, and if so, should an audit trail be carried out? This will require an extra table in the DB. This brings us to the topic of back-filling, which is a particularly insidious issue for backtesting. It concerns automatic correction of bad data upstream. If your data vendor corrects a historical error, but a backtested trading strategy is in production based on research from their previous bad data then decisions need to be made regarding the strategy effectiveness. This can be somewhat mitigated by being fully aware of your strategy performance metrics (in particular the variance in your win/loss characteristics for each trade). Strategies should be chosen or designed such that a single data point cannot skew the performance of the strategy to any great extent.

7.6 Automation

The benefit of writing software scripts to carry out the download, storage and cleaning of the data is that scripts can be automated via tools provided by the operating system. In UNIX-based systems (such as Mac OSX or Linux), one can make use of the **crontab**, which is a continually running process that allows specific scripts to be executed at custom-defined times or regular periods. There is an equivalent process on MS Windows known as the Task Scheduler.

A production process, for instance, might automate the download all of the S&P500 end-of-day prices as soon as they're published via a data vendor. It will then automatically run the aforementioned missing data and spike filtration scripts, alerting the trader via email, SMS or some other form of notification. At this point any backtesting tools will automatically have access to recent data, without the trader having to lift a finger! Depending upon whether your trading system is located on a desktop or on a remote server you may choose however to have a semi-automated or fully-automated process for these tasks.

7.7 Data Availability

Once the data is automatically updated and residing in the RDBMS it is necessary to get it into the backtesting software. This process will be highly dependent upon how your database is installed and whether your trading system is local (i.e. on a desktop computer) or remote (such as with a co-located exchange server).

One of the most important considerations is to minimise excessive Input/Output (I/O) as this can be extremely expensive both in terms of time and money, assuming remote connections where bandwidth is costly. The best way to approach this problem is to only move data across a network connection that you need (via selective querying) or exporting and compressing the data.

Many RDBMS support *replication* technology, which allows a database to be cloned onto another remote system, usually with a degree of latency. Depending upon your setup and data quantity this may only be on the order of minutes or seconds. A simple approach is to replicate a remote database onto a local desktop. However, be warned that synchronisation issues are common and time consuming to fix!

7.8 MySQL for Securities Masters

Now that we have discussed the idea behind a security master database it's time to actually build one. For this we will make use of two open source technologies: the **MySQL** database and the **Python** programming language. At the end of this chapter you will have a fully fledged equities security master with which to conduct further data analysis for your quantitative trading research.

7.8.1 Installing MySQL

Installation of MySQL within Ubuntu is straightforward. Simply open up a terminal and type the following:

```
sudo apt-get install mysql-server
```

Eventually, you will be prompted for a root password. This is your primary administration password so do not forget it! Enter the password and the installation will proceed and finish.

7.8.2 Configuring MySQL

Now that MySQL is installed on your system we can create a new *database* and a *user* to interact with it. You will have been prompted for a root password on installation. To log on to MySQL from the command line use the following line and then enter your password:

```
mysql -u root -p
```

8.2 Frequency of Data

Frequency of data is one of the most important considerations when designing an algorithmic trading system. It will impact every design decision regarding the storage of data, backtesting a strategy and executing an algorithm.

Higher frequency strategies are likely to lead to more statistically robust analysis, simply due to the greater number of data points (and thus trades) that will be used. HFT strategies often require a significant investment of time and capital for development of the necessary software to carry them out.

Lower frequency strategies are easier to develop and deploy, since they require less automation. However, they will often generate far less trades than a higher-frequency strategy leading to a less statistically robust analysis.

8.2.1 Weekly and Monthly Data

Fundamental data is often reported on a weekly, monthly, quarterly or even yearly basis. Such data include payroll data, hedge fund performance reports, SEC filings, inflation-based indices (such as the Consumer Price Index, CPI), economic growth and corporate accounts.

Storage of such data is often suited to unstructured databases, such as MongoDB, which can handle hierarchically-nested data and thus allowing a reasonable degree of querying capability. The alternative is to store flat-file text in a RDBMS, which is less appropriate, since full-text querying is trickier.

8.2.2 Daily Data

The majority of retail algorithmic traders make use of daily ("end of day"/EOD) financial time series data, particularly in equities and foreign exchange. Such data is freely available (see below), but often of questionable quality and subject to certain biases. End-of-day data is often stored in RDBMS, since the nature of ticker/symbol mapping naturally applies to the relational model.

EOD data does not entail particularly large storage requirements. There are 252 trading days in a year for US exchanges and thus for a decade there will be 2,520 bars per security. Even with a universe of 10,000 symbols this is 25,200,000 bars, which can easily be handled within a relational database environment.

8.2.3 Intraday Bars

Intraday strategies often make use of hourly, fifteen-, five-, one-minutely or secondly OHLCV bars. Intraday feed providers such as QuantQuote and DTN IQFeed will often provide minutely or secondly bars based on their tick data.

Data at such frequencies will possess many "missing" bars simply because no trades were carried out in that time period. Pandas can be used to pad these values forward, albeit with a decrease in data accuracy. In addition pandas can also be used to create data on less granular timescales if necessary.

For a ten year period, minutely data will generate almost one million bars per security. Similarly for secondly data the number of data points over the same period will total almost sixty million per security. Thus to store one thousand of such securities will lead to sixty billion bars of data. This is a large amount of data to be kept in an RDBMS and consequently more sophisticated approaches are required.

Storage and retrieval of secondly data on this magnitude is somewhat outside the scope of this book so I won't discuss it further.

8.2.4 Tick and Order Book Data

When a trade is filled at an exchange, or other venue, a *tick* is generated. Tick feeds consist of all such transactions *per exchange*. Retail tick feeds are stored with each datum having a timestamp accurate to the millisecond level. Tick data often also includes the updated best bid/ask price. The storage of tick data is well beyond the scope of this book but needless to say

the volumes of such data are substantial. Common storage mechanisms include HDF5, kdb and simply flat-file/CSV.

Multiple *limit orders* at an exchange lead to the concept of an *order book*. This is essentially the list of all bid and ask limit orders at certain volumes for each market participant. It leads to the definition of the *bid-ask spread* (or simply the “spread”), which is the smallest difference in the bid and ask prices for the “top of book” orders. Creating a historical representation, or a market simulator, of a limit order book is usually necessary for carrying out ultra high frequency trading (UHFT) strategies. The storage of such data is complex and as such will be outside the scope of this book.

8.3 Sources of Data

There are numerous sources and vendors of financial data. They vary substantially in breadth, timeliness, quality and price.

Broadly speaking, financial market data provided on a delayed daily frequency or longer is available freely, albeit with dubious overall quality and the potential for survivorship bias. To obtain intraday data it is usually necessary to purchase a commercial data feed. The vendors of such feeds vary tremendously in their customer service capability, overall feed quality and breadth of instruments.

8.3.1 Free Sources

Free end-of-day bar data, which consists of Open-High-Low-Close-Volume (OHLCV) prices for instruments, is available for a wide range of US and international equities and futures from Yahoo Finance, Google Finance and Quandl.

Yahoo Finance

Yahoo Finance is the “go to” resource when forming an end-of-day US equities database. The breadth of data is extremely comprehensive, listing thousands of traded equities. In addition stock-splits and dividends are handled using a back-adjustment method, arising as the “Adj Close” column in the CSV output from the API (which we discuss below). Thus the data allows algorithmic traders to get started rapidly and for zero cost.

I have personally had a lot of experience in cleaning Yahoo data. I have to remark that the data can be quite erroneous. Firstly, it is subject to a problem known as *backfilling*. This problem occurs when past historical data is corrected at a future date, leading to poor quality backtests that change as your own database is re-updated. To handle this problem, a logging record is usually added to the securities master (in an appropriate logging table) whenever a historical data point is modified.

Secondly, the Yahoo feed only aggregates prices from a few sources to form the OHLCV points. This means that values around the open, high, low and close can be deceptive, as other exchanges/liquidity sources may have executed differing prices in excess of the values.

Thirdly, I have noticed that when obtaining financial data *en-masse* from Yahoo, that errors do creep into the API. For instance, multiple calls to the API with identical date/ticker parameters occasionally lead to differing result sets. This is clearly a substantial problem and must be carefully checked for.

In summary be prepared to carry out some extensive data cleansing on Yahoo Finance data, if you choose to use it to populate a large securities master, and need highly accurate data.

Quandl

Quandl is a relatively new service which purports to be “*The easiest way to find and use numerical data on the web*”. I believe they are well on the way to achieving that goal! The service provides a substantial daily data set of US and international equities, interest rates, commodities/futures, foreign exchange and other economic data. In addition, the database is continually expanded and the project is very actively maintained.

All of the data can be accessed by a very modern HTTP API (CSV, JSON, XML or HTML), with plugins for a wide variety of programming languages including R, Python, Matlab, Excel, Stata, Maple, C#, EViews, Java, C/C++, .NET, Clojure and Julia. Without an account 50 calls to the API are allowed per day, but this can be increased to 500 if registering an account. In fact, calls can be updated to 5,000 per hour if so desired by contacting the team.

I have not had a great deal of experience with Quandl "at scale" and so I can't comment on the level of errors within the dataset, but my feeling is that any errors are likely to be constantly reported and corrected. Thus they are worth considering as a primary data source for an end-of-day securities master.

Later in the chapter we will discuss how to obtain US Commodities Futures data from Quandl with Python and pandas.

8.3.2 Commercial Sources

In order to carry out intraday algorithmic trading it is usually necessary to purchase a commercial feed. Pricing can range anywhere from \$30 per month to around \$500 per month for "retail level" feeds. Institutional quality feeds will often be in the low-to-mid four figure range per month and as such I won't discuss them here.

EODData

I have utilised EODData in a fund context, albeit only with daily data and predominantly for foreign exchange. Despite their name they do provide a degree of intraday sources. The cost is \$25 per month for their "platinum" package.

The resource is very useful for finding a full list of traded symbols on global exchanges, but remember that this will be subject to survivorship bias as I believe the list represents current listed entities.

Unfortunately (at least back in 2010) I found that the stock split feed was somewhat inaccurate (at least when compared to Morningstar information). This led to some substantial spike issues (see below) in the data, which increased friction in the data cleansing process.

DTN IQFeed

DTN IQFeed are one of the most popular data feeds for the high-end retail algorithmic trader. They claim to have over 80,000 customers. They provide real-time tick-by-tick data unfiltered from the exchange as well as a large quantity of historic data.

The pricing starts at \$50 per month, but in reality will be in the \$150-\$200 per month range once particular services are selected and exchange fees are factored in. I utilise DTN IQFeed for all of my intraday equities and futures strategies. In terms of historical data, IQFeed provide for equities, futures and options:

- 180 calendar days of tick (every trade)
- 7+ years of 1 minute historical bars
- 15+ years of daily historical bars

The major disadvantage is that the DTN IQFeed software (the mini-server, not the charting tools) will only run on Windows. This may not be a problem if all of your algorithmic trading is carried out in this operating system, but I personally develop all my strategies in Ubuntu Linux. However, although I have not actively tested it, I have heard it is possible to run DTN IQFeed under the WINE emulator.

Below we will discuss how to obtain data from IQFeed using Python in Windows.


```

data = read_historical_data_socket(sock)
sock.close

# Remove all the endlines and line-ending
# comma delimiter from each record
data = "".join(data.split("\r"))
data = data.replace(",\n","\n")[:-1]

# Write the data stream to disk
f = open("%s.csv" % sym, "w")
f.write(data)
f.close()

```

With additional subscription options in the DTN IQFeed account, it is possible to download individual futures contracts (and back-adjusted continuous contracts), options and indices. DTN IQFeed also provides real-time tick streaming, but this form of data falls outside the scope of the book.

8.5 Cleaning Financial Data

Subsequent to the delivery of financial data from vendors it is necessary to perform *data cleansing*. Unfortunately this can be a painstaking process, but a hugely necessary one. There are multiple issues that require resolution: Incorrect data, consideration of data aggregation and backfilling. Equities and futures contracts possess their own unique challenges that must be dealt with prior to strategy research, including back/forward adjustment, continuous contract stitching and corporate action handling.

8.5.1 Data Quality

The reputation of a data vendor will often rest on its (perceived) data quality. In simple terms, bad or missing data leads to erroneous trading signals and thus potential loss. Despite this fact, many vendors are still plagued with poor or inconsistent data quality. Thus there is always a cleansing process necessary to be carried.

The main culprits in poor data quality are conflicting/incorrect data, opaque aggregation of multiple data sources and error correction ("backfilling").

Conflicting and Incorrect Data

Bad data can happen anywhere in the stream. Bugs in the software at an exchange can lead to erroneous prices when matching trades. This filters through to the vendor and subsequently the trader. Reputable vendors will attempt to flag upstream "bad ticks" and will often leave the "correction" of these points to the trader.

8.5.2 Continuous Futures Contracts

In this section we are going to discuss the characteristics of futures contracts that present a data challenge from a backtesting point of view. In particular, the notion of the "continuous contract". We will outline the main difficulties of futures and provide an implementation in Python with pandas that can partially alleviate the problems.

Brief Overview of Futures Contracts

Futures are a form of *contract* drawn up between two parties for the purchase or sale of a quantity of an underlying asset at a specified date in the future. This date is known as the *delivery* or *expiration*. When this date is reached the buyer must deliver the physical underlying (or cash equivalent) to the seller for the price agreed at the contract formation date.

Chapter 10

Time Series Analysis

In this chapter we are going to consider statistical tests that will help us identify price series that possess trending or mean-reverting behaviour. If we can identify such series statistically then we can capitalise on this behaviour by forming momentum or mean-reverting trading strategies.

In later chapters we will use these statistical tests to help us identify candidate time series and then create algorithmic strategies around them.

10.1 Testing for Mean Reversion

One of the key quantitative trading concepts is **mean reversion**. This process refers to a time series that displays a tendency to revert to a historical mean value. Such a time series can be exploited to generate trading strategies as we enter the market when a price series is far from the mean under the expectation that the series will return to a mean value, whereby we exit the market for a profit. Mean-reverting strategies form a large component of the *statistical arbitrage* quant hedge funds. In later chapters we will create both intraday and interday strategies that exploit mean-reverting behaviour.

The basic idea when trying to ascertain if a time series is mean-reverting is to use a statistical test to see if it differs from the behaviour of a *random walk*. A random walk is a time series where the next directional movement is completely independent of any past movements - in essence the time series has no "memory" of where it has been. A mean-reverting time series, however, is different. The change in the value of the time series in the next time period is proportional to the current value. Specifically, it is proportional to the difference between the mean historical price and the current price.

Mathematically, such a (continuous) time series is referred to as an **Ornstein-Uhlenbeck** process. If we can show, statistically, that a price series behaves like an Ornstein-Uhlenbeck series then we can begin the process of forming a trading strategy around it. Thus the goal of this chapter is to outline the statistical tests necessary to identify mean reversion and then use Python libraries (in particular *statsmodels*) in order to implement these tests. In particular, we will study the concept of **stationarity** and how to test for it.

As stated above, a *continuous* mean-reverting time series can be represented by an Ornstein-Uhlenbeck stochastic differential equation:

$$dx_t = \theta(\mu - x_t)dt + \sigma dW_t \quad (10.1)$$

Where θ is the rate of reversion to the mean, μ is the mean value of the process, σ is the variance of the process and W_t is a Wiener Process or Brownian Motion.

This equation essentially states that the change of the price series in the next continuous time period is proportional to the difference between the mean price and the current price, with the addition of Gaussian noise.

We can use this equation to motivate the definition of the Augmented Dickey-Fuller Test, which we will now describe.

10.1.1 Augmented Dickey-Fuller (ADF) Test

The ADF test makes use of the fact that if a price series possesses mean reversion, then the next price level will be proportional to the current price level. Mathematically, the ADF is based on the idea of testing for the presence of a **unit root** in an **autoregressive** time series sample.

We can consider a model for a time series, known as a *linear lag model of order p* . This model says that the change in the value of the time series is proportional to a constant, the time itself and the previous p values of the time series, along with an error term:

$$\Delta y_t = \alpha + \beta t + \gamma y_{t-1} + \delta_1 \Delta y_{t-1} + \cdots + \delta_{p-1} \Delta y_{t-p+1} + \epsilon_t \quad (10.2)$$

Where α is a constant, β represents the coefficient of a temporal trend and $\Delta y_t = y(t) - y(t-1)$. The role of the ADF hypothesis test is to ascertain, statistically, whether $\gamma = 0$, which would indicate (with $\alpha = \beta = 0$) that the process is a random walk and thus non mean reverting. Hence we are testing for the *null hypothesis* that $\gamma = 0$.

If the hypothesis that $\gamma = 0$ can be rejected then the following movement of the price series is proportional to the current price and thus it is unlikely to be a random walk. This is what we mean by a "statistical test".

So how is the ADF test carried out?

- Calculate the *test statistic*, DF_τ , which is used in the decision to reject the null hypothesis
- Use the *distribution* of the test statistic (calculated by Dickey and Fuller), along with the critical values, in order to decide whether to reject the null hypothesis

Let's begin by calculating the test statistic (DF_τ). This is given by the *sample* proportionality constant $\hat{\gamma}$ divided by the *standard error* of the sample proportionality constant:

$$DF_\tau = \frac{\hat{\gamma}}{SE(\hat{\gamma})} \quad (10.3)$$

Now that we have the test statistic, we can use the distribution of the test statistic calculated by Dickey and Fuller to determine the rejection of the null hypothesis for any chosen percentage critical value. The test statistic is a negative number and thus in order to be significant beyond the critical values, the number must be smaller (i.e. more negative) than these values.

A key practical issue for traders is that any constant long-term drift in a price is of a much smaller magnitude than any short-term fluctuations and so the drift is often assumed to be zero ($\beta = 0$) for the linear lag model described above.

Since we are considering a lag model of order p , we need to actually set p to a particular value. It is usually sufficient, for trading research, to set $p = 1$ to allow us to reject the null hypothesis. However, note that this technically introduces a parameter into a trading model based on the ADF.

To calculate the Augmented Dickey-Fuller test we can make use of the pandas and statsmodels libraries. The former provides us with a straightforward method of obtaining Open-High-Low-Close-Volume (OHLCV) data from Yahoo Finance, while the latter wraps the ADF test in a easy to call function. This prevents us from having to calculate the test statistic manually, which saves us time.

We will carry out the ADF test on a sample price series of Amazon stock, from 1st January 2000 to 1st January 2015.

Here is the Python code to carry out the test:

```
from __future__ import print_function

# Import the Time Series library
import statsmodels.tsa.stattools as ts

# Import Datetime and the Pandas DataReader
```

```

from datetime import datetime
import pandas.io.data as web

# Download the Amazon OHLCV data from 1/1/2000 to 1/1/2015
amzn = web.DataReader("AMZN", "yahoo", datetime(2000,1,1), datetime(2015,1,1))

# Output the results of the Augmented Dickey-Fuller test for Amazon
# with a lag order value of 1
ts.adfuller(amzn['Adj Close'], 1)

```

Here is the output of the Augmented Dickey-Fuller test for Amazon over the period. The first value is the calculated test-statistic, while the second value is the *p-value*. The fourth is the number of data points in the sample. The fifth value, the dictionary, contains the critical values of the test-statistic at the 1, 5 and 10 percent values respectively.

```

(0.049177575166452235,
 0.96241494632563063,
 1,
 3771,
 {'1%': -3.4320852842548395,
  '10%': -2.5671781529820348,
  '5%': -2.8623067530084247},
 19576.116041473877)

```

Since the calculated value of the test statistic is larger than any of the critical values at the 1, 5 or 10 percent levels, we cannot reject the null hypothesis of $\gamma = 0$ and thus we are unlikely to have found a mean reverting time series. This is in line with our tuition as most equities behave akin to Geometric Brownian Motion (GBM), i.e. a random walk.

This concludes how we utilise the ADF test. However, there are alternative methods for detecting mean-reversion, particularly via the concept of **stationarity**, which we will now discuss.

10.2 Testing for Stationarity

A time series (or stochastic process) is defined to be **strongly stationary** if its *joint probability distribution* is invariant under translations in time or space. In particular, and of key importance for traders, the mean and variance of the process do not change over time or space and they each do not follow a trend.

A critical feature of stationary price series is that the prices within the series diffuse from their initial value at a rate slower than that of a GBM. By measuring the rate of this diffusive behaviour we can identify the nature of the time series and thus detect whether it is mean-reverting.

We will now outline a calculation, namely the Hurst Exponent, which helps us to characterise the stationarity of a time series.

10.2.1 Hurst Exponent

The goal of the Hurst Exponent is to provide us with a scalar value that will help us to identify (within the limits of statistical estimation) whether a series is mean reverting, random walking or trending.

The idea behind the Hurst Exponent calculation is that we can use the variance of a log price series to assess the rate of diffusive behaviour. For an arbitrary time lag τ , the variance of τ is given by:

$$\text{Var}(\tau) = \langle |\log(t + \tau) - \log(t)|^2 \rangle \quad (10.4)$$

Where the brackets $\langle$ and $\rangle$ refer to the average over all values of t .

The idea is to compare the rate of diffusion to that of a GBM. In the case of a GBM, at large times (i.e. when τ is large) the variance of τ is proportional to τ :

$$\langle |\log(t + \tau) - \log(t)|^2 \rangle \sim \tau \quad (10.5)$$

If we find behaviour that differs from this relation, then we have identified either a trending or a mean-reverting series. The key insight is that if any sequential price movements possess non-zero correlation (known as autocorrelation) then the above relationship is not valid. Instead it can be modified to include an exponent value " $2H$ ", which gives us the Hurst Exponent value H :

$$\langle |\log(t + \tau) - \log(t)|^2 \rangle \sim \tau^{2H} \quad (10.6)$$

Thus it can be seen that if $H = 0.5$ we have a GBM, since it simply becomes the previous relation. However if $H \neq 0.5$ then we have trending or mean-reverting behaviour. In particular:

- $H < 0.5$ - The time series is mean reverting
- $H = 0.5$ - The time series is a Geometric Brownian Motion
- $H > 0.5$ - The time series is trending

In addition to characterisation of the time series the Hurst Exponent also describes the extent to which a series behaves in the manner categorised. For instance, a value of H near 0 is a highly mean reverting series, while for H near 1 the series is strongly trending.

To calculate the Hurst Exponent for the Amazon price series, as utilised above in the explanation of the ADF, we can use the following Python code:

```
from __future__ import print_function

from numpy import cumsum, log, polyfit, sqrt, std, subtract
from numpy.random import randn

def hurst(ts):
    """Returns the Hurst Exponent of the time series vector ts"""
    # Create the range of lag values
    lags = range(2, 100)

    # Calculate the array of the variances of the lagged differences
    tau = [sqrt(std(subtract(ts[lag:], ts[:-lag]))) for lag in lags]

    # Use a linear fit to estimate the Hurst Exponent
    poly = polyfit(log(lags), log(tau), 1)

    # Return the Hurst exponent from the polyfit output
    return poly[0]*2.0

# Create a Gometric Brownian Motion, Mean-Reverting and Trending Series
gbm = log(cumsum(randn(100000))+1000)
mr = log(randn(100000)+1000)
tr = log(cumsum(randn(100000)+1)+1000)

# Output the Hurst Exponent for each of the above series
# and the price of Amazon (the Adjusted Close price) for
# the ADF test given above in the article
print("Hurst(GBM):  %s" % hurst(gbm))
print("Hurst(MR):   %s" % hurst(mr))
print("Hurst(TR):   %s" % hurst(tr))
```

```
# Assuming you have run the above code to obtain 'amzn'!
print("Hurst(AMZN): %s" % hurst(amzn['Adj Close']))
```

The output from the Hurst Exponent Python code is given below:

```
Hurst(GBM):    0.502051910931
Hurst(MR):     0.000166110248967
Hurst(TR):     0.957701001252
Hurst(AMZN):   0.454337476553
```

From this output we can see that the GBM possesses a Hurst Exponent, H , that is almost exactly 0.5. The mean reverting series has H almost equal to zero, while the trending series has H close to 1.

Interestingly, Amazon has H also close to 0.5 indicating that it is similar to a GBM, at least for the sample period we're making use of!

10.3 Cointegration

It is actually very difficult to find a tradable asset that possesses mean-reverting behaviour. Equities broadly behave like GBMs and hence render the mean-reverting trade strategies relatively useless. However, there is nothing stopping us from creating a *portfolio* of price series that is stationary. Hence we can apply mean-reverting trading strategies to the portfolio.

The simplest form of mean-reverting trade strategies is the classic "pairs trade", which usually involves a dollar-neutral long-short pair of equities. The theory goes that two companies in the same sector are likely to be exposed to similar market factors, which affect their businesses. Occasionally their relative stock prices will diverge due to certain events, but will revert to the long-running mean.

Let's consider two energy sector equities Approach Resources Inc given by the ticker AREX and Whiting Petroleum Corp given by the ticker WLL. Both are exposed to similar market conditions and thus will likely have a stationary pairs relationship. We are now going to create some plots, using pandas and the Matplotlib libraries to demonstrate the cointegrating nature of AREX and WLL. The first plot (Figure 10.1) displays their respective price histories for the period Jan 1st 2012 to Jan 1st 2013.

If we create a scatter plot of their prices, we see that the relationship is broadly linear (see Figure 10.2) for this period.

The pairs trade essentially works by using a linear model for a relationship between the two stock prices:

$$y(t) = \beta x(t) + \epsilon(t) \quad (10.7)$$

Where $y(t)$ is the price of AREX stock and $x(t)$ is the price of WLL stock, both on day t .

If we plot the residuals $\epsilon(t) = y(t) - \beta x(t)$ (for a particular value of β that we will determine below) we create a new time series that, at first glance, looks relatively stationary. This is given in Figure 10.3.

We will describe the code for each of these plots below.

10.3.1 Cointegrated Augmented Dickey-Fuller Test

In order to statistically confirm whether this series is mean-reverting we could use one of the tests we described above, namely the Augmented Dickey-Fuller Test or the Hurst Exponent. However, neither of these tests will actually help us determine β , the hedging ratio needed to form the linear combination, they will only tell us whether, for a particular β , the linear combination is stationary.

This is where the Cointegrated Augmented Dickey-Fuller (CADF) test comes in. It determines the optimal hedge ratio by performing a linear regression against the two time series and then tests for stationarity under the linear combination.

Chapter 11

Forecasting

In this chapter we will create a statistically robust process for forecasting financial time series. These forecasts will form the basis for further automated trading strategies. We will expand on the topic of Statistical Learning discussed in the previous chapters and use a group of *classification algorithms* to help us predict market direction of financial time series.

Within this chapter we will be making use of **Scikit-Learn**, a statistical machine learning library for Python. Scikit-learn contains "ready-made" implementations of many machine learning techniques. Not only does this save us a great deal of time in implementing our trading algorithms, but it minimises the risk of bugs introduced by our own code. It also allows additional verification against machine learning libraries written in other packages such as R or C++. This gives us a great deal of confidence if we need to create our own custom implementation, for reasons of execution speed, say.

We will begin by discussing ways of measuring forecaster performance for the particular case of machine learning techniques used. Then we will consider the predictive factors that can be used in forecasting techniques and how to choose good factors. Then we will consider various supervised classifier algorithms. Finally, we will attempt to forecast the daily direction of the S&P500, which will later form the basis of an algorithmic trading strategy.

11.1 Measuring Forecasting Accuracy

Before we discuss choices of predictor and specific classification algorithms we must discuss their performance characteristics and how to evaluate them. The particular class of methods that we are interested in involves *binary supervised classification*. That is, we will attempt to predict whether the percentage return for a particular future day is positive or negative (i.e. whether our financial asset has risen or dropped in price).

In a production forecaster, using a regression-type technique, we would be very concerned with the *magnitude* of this prediction and the deviations of the prediction from the actual value.

To assess the performance of these classifiers we can make use of the following two measures, namely the **Hit-Rate** and **Confusion Matrix**.

11.1.1 Hit Rate

The simplest question that we could ask of our supervised classifier is "*How many times did we predict the correct direction, as a percentage of all predictions?*". This motivates the definition of the *training hit rate* is given by the following formula[9]:

$$\frac{1}{n} \sum_{j=1}^n I(y_j = \hat{y}_j) \quad (11.1)$$

Where $\hat{y}_j$ is the prediction (up or down) for the j th time period (e.g. a day) using a particular classifier. $I(y_j = \hat{y}_j)$ is the *indicator function* and is equal to 1 if $y_j = \hat{y}_j$ and 0 if $y_j \neq \hat{y}_j$.

Hence the hit rate provides a percentage value as to the number of times a classifier correctly predicted the up or down direction.

Scikit-Learn provides a method to calculate the hit rate for us as part of the classification/-training process.

11.1.2 Confusion Matrix

The *confusion matrix* (or *contingency table*) is the next logical step after calculating the hit rate. It is motivated by asking "How many times did we predict up correctly and how many times did we predict down correctly? Did they differ substantially?".

For instance, it might turn out that a particular algorithm is consistently more accurate at predicting "down days". This motivates a strategy that emphasises shorting of a financial asset to increase profitability.

A confusion matrix characterises this idea by determining the *false positive rate* (known statistically as a Type I error) and *false negative rate* (known statistically as a Type II error) for a supervised classifier. For the case of binary classification (up or down) we will have a 2x2 matrix:

$$\begin{pmatrix} U_T & U_F \\ D_F & D_T \end{pmatrix}$$

Where U_T represents correctly classified up periods, U_F represents incorrectly classified up periods (i.e. classified as down), D_F represents incorrectly classified down periods (i.e. classified as up) and D_T represents correctly classified down periods.

In addition to the hit rate, Scikit-Learn provides a method to calculate the confusion matrix for us as part of the classification/training process.

11.2 Factor Choice

One of the most crucial aspects of asset price forecasting is choosing the factors used as predictors. There are a staggering number of potential factors to choose and this can seem overwhelming to an individual unfamiliar with financial forecasting. However, even simple machine learning techniques will produce relatively good results when used with well-chosen factors. Note that the converse is not often the case. "Throwing an algorithm at a problem" will usually lead to poor forecasting accuracy.

Factor choice is carried out by trying to determine the fundamental drivers of asset movement. In the case of the S&P500 it is clear that the 500 constituents, in a weighted manner, will be fundamental drivers of the price, by definition! Clearly we would know the exact price of the S&P500 series if we knew the instantaneous value of its constituents, but is there any predictive power in using the prior history of returns for each constituent in predicting the series itself?

Alternatively, could we consider exchange rates with countries that carry out a lot of trade with the US as drivers of the price? We could even consider more fundamental economic and corporate factors such as interest rates, inflation, quarterly earnings.

The accuracy of the forecaster will in large part be due to the skill of the modeller in determining the right factors prior to carrying out model fitting.

11.2.1 Lagged Price Factors and Volume

The first type of factor that is often considered in forecasting a time series are prior historical values of the time series itself. Thus a set of p factors could be easily obtained by creating p lags of the time series close price. Consider a daily time series. For each particular current day k , the factors would be the historical daily values at time periods $k-1, k-2, \dots, k-p$.

In addition to the price series itself we can also incorporate traded volume as an indicator, since it is provided when using OHLCV data (as is obtained from Yahoo Finance, Google Finance or Quandl for instance). Thus we can create a $p+1$ -dimensional feature vector for each day of the time series, which incorporates the p time lags and the volume series. This naturally leads

to a set of pairs (X_k, y_k) representing the $p + 1$ -dimensional feature vector X_k at day k and the actual current closing price on day k , y_k . This is all we need to begin a supervised classification exercise.

Below we will consider such a lagged time series for the S&P500 and apply multiple machine learning techniques to see if we can forecast its direction.

11.2.2 External Factors

While lagged time series and volume information are a good starting point for time series analysis, we are far from restricted to such data. There are a vast amount of macroeconomic time series and asset prices series on which to consider forecasts. For instance we may wish to provide a long-term forecast of commodities prices based on weather patterns, or ascertain foreign exchange price direction movements via international interest rate movements.

If such a relationship between series can be ascertained and shown to be statistically significant, then we are at the point of being able to consider a robust trading model. We won't dwell on such relationships too much here, as our goal is to introduce the idea of modelling and machine learning techniques. It is easy enough to form hypotheses about economic relationships and obtain the time series data either from a repository such as Quandl, or directly from government statistics websites.

11.3 Classification Models

The field of machine learning is vast and there are many models to choose from, particularly in the realm of supervised classification. New models are being introduced on a monthly basis through the academic literature. It would be impractical to provide an exhaustive list of supervised classifiers in this chapter, rather we will consider some of the more popular techniques from the field.

11.3.1 Logistic Regression

The first technique we will consider is **logistic regression** (LR). In our case we are going to use logistic regression to measures the relationship between a *binary categorical dependent variable* (i.e. "up" or "down" periods) and multiple independent *continuous variables*, such as the lagged percentage returns of a financial asset.

The logistic regression model provides the *probability* that a particular subsequent time period will be categorised as "up" or "down". Thus the model introduces a *parameter*, namely the probability threshold for classifying whether a subsequent time period is "up" or "down". Below, we will take this threshold to be 50% (i.e. 0.5), but it can certainly be modified to produce alternative predictions.

Logistic regression is based on the logistic formula to model the probability of obtaining an "up" day ($Y = U$) based on the continuous factors.

In this case, consider the situation where we are interested in predicting the subsequent time period from the previous two lagged returns, which we will denote by (L_1, L_2) . The formula below gives the probability for having an up day, given that we have observed the returns on the previous time periods, L_1 and L_2 :

$$p(Y = U|L_1, L_2) = \frac{e^{\beta_0 + \beta_1 L_1 + \beta_2 L_2}}{1 + e^{\beta_0 + \beta_1 L_1 + \beta_2 L_2}} \quad (11.2)$$

The logistic function is used instead of a linear function (i.e. in linear regression) because it provides a probability between $[0, 1]$ for all values of L_1 and L_2 . In a linear regression setting it is possible to obtain negative probabilities for these continuous variables so we need another function.

To fit the model (i.e. estimate the β_i coefficients) the **maximum likelihood method** is used. Fortunately for us the implementation of the fitting and prediction of the logistic regression

The solution is to modify the kernel used by the SVM, which has the effect of allowing non-linear decision boundaries. Thus they are quite flexible models. However, the right SVM boundary needs to be chosen for the best results. SVM are especially good in text classification problems with high dimensionality. They are disadvantaged by their computational complexity, difficulty of tuning and the fact the the fitted model is difficult to interpret.

11.4 Forecasting Stock Index Movement

The S&P500 is a weighted index of the 500 largest publicly traded companies by market capitalisation in the US stock market. It is often utilised as an equities benchmark. Many derivative products exist in order to allow speculation or hedging on the index. In particular, the S&P500 E-Mini Index Futures Contract is an extremely liquid means of trading the index.

In this section we are going to use a set of classifiers to predict the direction of the closing price at day k based solely on price information known at day $k - 1$. An upward directional move means that the closing price at k is higher than the price at $k - 1$, while a downward move implies a closing price at k lower than at $k - 1$.

If we can determine the direction of movement in a manner that significantly exceeds a 50% hit rate, with low error and a good statistical significance, then we are on the road to forming a basic systematic trading strategy based on our forecasts.

11.4.1 Python Implementations

For the implementation of these forecasters we will make use of NumPy, Pandas and Scikit-Learn, which were installed in the previous chapters.

The first step is to import the relevant modules and libraries. We're going to import the LogisticRegression, LDA, QDA, LinearSVC (a linear Support Vector Machine), SVC (a non-linear Support Vector Machine) and RandomForest classifiers for this forecast:

```
#!/usr/bin/python
# -*- coding: utf-8 -*-

# forecast.py

from __future__ import print_function

import datetime
import numpy as np
import pandas as pd
import sklearn

from pandas.io.data import DataReader
from sklearn.ensemble import RandomForestClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.lda import LDA
from sklearn.metrics import confusion_matrix
from sklearn.qda import QDA
from sklearn.svm import LinearSVC, SVC
```

Now that the libraries are imported, we need to create a pandas DataFrame that contains the lagged percentage returns for a prior number of days (defaulting to five). `create_lagged_series` will take a stock symbol (as recognised by Yahoo Finance) and create a lagged DataFrame across the period specified. The code is well commented so it should be straightforward to see what is going on:

```
def create_lagged_series(symbol, start_date, end_date, lags=5):
    """
    This creates a Pandas DataFrame that stores the
```

```

percentage returns of the adjusted closing value of
a stock obtained from Yahoo Finance, along with a
number of lagged returns from the prior trading days
(lags defaults to 5 days). Trading volume, as well as
the Direction from the previous day, are also included.
"""

# Obtain stock information from Yahoo Finance
ts = DataReader(
    symbol, "yahoo",
    start_date=datetime.timedelta(days=365),
    end_date
)

# Create the new lagged DataFrame
tslag = pd.DataFrame(index=ts.index)
tslag["Today"] = ts["Adj Close"]
tslag["Volume"] = ts["Volume"]

# Create the shifted lag series of prior trading period close values
for i in range(0, lags):
    tslag["Lag%s" % str(i+1)] = ts["Adj Close"].shift(i+1)

# Create the returns DataFrame
tsret = pd.DataFrame(index=tslag.index)
tsret["Volume"] = tslag["Volume"]
tsret["Today"] = tslag["Today"].pct_change()*100.0

# If any of the values of percentage returns equal zero, set them to
# a small number (stops issues with QDA model in Scikit-Learn)
for i,x in enumerate(tsret["Today"]):
    if (abs(x) < 0.0001):
        tsret["Today"][i] = 0.0001

# Create the lagged percentage returns columns
for i in range(0, lags):
    tsret["Lag%s" % str(i+1)] = \
        tslag["Lag%s" % str(i+1)].pct_change()*100.0

# Create the "Direction" column (+1 or -1) indicating an up/down day
tsret["Direction"] = np.sign(tsret["Today"])
tsret = tsret[tsret.index >= start_date]

return tsret

```

We tie the classification procedure together with a `__main__` function. In this instance we're going to attempt to forecast the US stock market direction in 2005, using returns data from 2001 to 2004.

Firstly we create a lagged series of the S&P500 using five lags. The series also includes trading volume. However, we are going to restrict the predictor set to use only the first two lags. Thus we are implicitly stating to the classifier that the further lags are of less predictive value. *As an aside, this effect is more concretely studied under the statistical concept of autocorrelation, although this is beyond the scope of the book.*

After creating the predictor array X and the response vector y , we can partition the arrays into a *training* and a *test* set. The former subset is used to actually train the classifier, while the latter is used to actually test the performance. We are going to split the training and testing set on the 1st January 2005, leaving a full trading years worth of data (approximately 250 days) for

the testing set.

Once we create the training/testing split we need to create an array of classification models, each of which is in a tuple with an abbreviated name attached. While we have not set any parameters for the Logistic Regression, Linear/Quadratic Discriminant Analysers or Linear Support Vector Classifier models, we have used a set of default parameters for the Radial Support Vector Machine (RSVM) and the Random Forest (RF).

Finally we iterate over the models. We train (fit) each model on the training data and then make predictions on the testing set. Finally we output the hit rate and the confusion matrix for each model:

```
if __name__ == "__main__":
    # Create a lagged series of the S&P500 US stock market index
    snpret = create_lagged_series(
        "^GSPC", datetime.datetime(2001,1,10),
        datetime.datetime(2005,12,31), lags=5
    )

    # Use the prior two days of returns as predictor
    # values, with direction as the response
    X = snpret[["Lag1","Lag2"]]
    y = snpret["Direction"]

    # The test data is split into two parts: Before and after 1st Jan 2005.
    start_test = datetime.datetime(2005,1,1)

    # Create training and test sets
    X_train = X[X.index < start_test]
    X_test = X[X.index >= start_test]
    y_train = y[y.index < start_test]
    y_test = y[y.index >= start_test]

    # Create the (parametrised) models
    print("Hit Rates/Confusion Matrices:\n")
    models = [("LR", LogisticRegression()),
              ("LDA", LDA()),
              ("QDA", QDA()),
              ("LSVC", LinearSVC()),
              ("RSVM", SVC(
                  C=1000000.0, cache_size=200, class_weight=None,
                  coef0=0.0, degree=3, gamma=0.0001, kernel='rbf',
                  max_iter=-1, probability=False, random_state=None,
                  shrinking=True, tol=0.001, verbose=False
              )),
              ("RF", RandomForestClassifier(
                  n_estimators=1000, criterion='gini',
                  max_depth=None, min_samples_split=2,
                  min_samples_leaf=1, max_features='auto',
                  bootstrap=True, oob_score=False, n_jobs=1,
                  random_state=None, verbose=0
              ))]

    # Iterate through the models
    for m in models:

        # Train each of the models on the training set
        m[1].fit(X_train, y_train)
```

```
# Make an array of predictions on the test set
pred = m[1].predict(X_test)

# Output the hit-rate and the confusion matrix for each model
print("%s:\n%0.3f" % (m[0], m[1].score(X_test, y_test)))
print("%s\n" % confusion_matrix(pred, y_test))
```

11.4.2 Results

The output from all of the classification models is as follows. You will likely see different values on the RF (Random Forest) output as it is inherently stochastic in its construction:

Hit Rates/Confusion Matrices:

```
LR:
0.560
[[ 35  35]
 [ 76 106]]
```

```
LDA:
0.560
[[ 35  35]
 [ 76 106]]
```

```
QDA:
0.599
[[ 30  20]
 [ 81 121]]
```

```
LSVC:
0.560
[[ 35  35]
 [ 76 106]]
```

```
RSVM:
0.563
[[  9   8]
 [102 133]]
```

```
RF:
0.504
[[48 62]
 [63 79]]
```

Note that all of the hit rates lie between 50% and 60%. Thus we can see that the lagged variables are not hugely indicative of future direction. However, if we look at the quadratic discriminant analyser we can see that its overall predictive performance on the test set is just under 60%.

The confusion matrix for this model (and the others in general) also states that the true positive rate for the "down" days is much higher than the "up" days. Thus if we are to create a trading strategy based off this information we could consider restricting trades to short positions of the S&P500 as a potential means of increasing profitability.

In later chapters we will use these models as a basis of a trading strategy by incorporating them directly into the event-driven backtesting framework and using a direct instrument, such as an exchange traded fund (ETF), in order to give us access to trading the S&P500.

Chapter 12

Performance Measurement

Performance measurement is an absolutely crucial component of algorithmic trading. Without assessment of performance, along with solid record keeping, it is difficult, if not impossible, to determine if our strategy returns have been due to luck or due to some actual edge over the market.

In order to be successful in algorithmic trading it is necessary to be aware of all of the factors that can affect the profitability of trades, and ultimately strategies. We should be constantly trying to find improvements in all aspects of the algorithmic trading stack. In particular we should always be trying to minimise our transaction costs (fees, commission and slippage), improve our software and hardware, improve the cleanliness of our data feeds and continually seek out new strategies to add to a portfolio. Performance measurement in all these areas provides a yardstick upon which to measure alternatives.

Ultimately, algorithmic trading is about generating profits. Hence it is imperative that we measure the performance, at multiple levels of granularity, of how and why our system is producing these profits. This motivates performance assessment at the level of trades, strategies and portfolios. In particular we are looking out for:

- Whether the systematic rules codified by the strategy actually produce a consistent return and whether the strategy possesses positive performance in the backtests.
- Whether a strategy maintains this positive performance in a live implementation or whether it needs to be retired.
- The ability to compare multiple strategies/portfolios such that we can reduce the *opportunity cost* associated with allocating a limited amount of trading capital.

The particular items of quantitative analysis of performance that we will be interested in are as follows:

- **Returns** - The most visible aspect of a trading strategy concerns the percentage gain since inception, either in a backtest or a live trading environment. The two major performance measures here are Total Return and Compound Annual Growth Rate (CAGR).
- **Drawdowns** - A *drawdown* is a period of negative performance, as defined from a prior *high-water mark*, itself defined as the previous highest peak on a strategy or portfolio *equity curve*. We will define this more concretely below, but you can think of it for now as a (somewhat painful!) downward slope on your performance chart.
- **Risk** - Risk comprises many areas, and we'll spend significant time going over them in the following chapter, but generally it refers to both risk of capital loss, such as with drawdowns, and volatility of returns. The latter usually being calculated as an annualised standard deviation of returns.
- **Risk/Reward Ratio** - Institutional investors are mainly interested with *risk-adjusted returns*. Since higher volatility can often lead to higher returns at the expense of greater

drawdowns, they are always concerned with how much risk is being taken on per unit of return. Consequently a range of performance measures have been invented to quantify this aspect of strategy performance, namely the Sharpe Ratio, Sortino Ratio and CALMAR Ratio, among others. The *out of sample Sharpe* is often the first metric to be discussed by institutional investors when discussing strategy performance.

- **Trade Analysis** - The previous measures of performance are all applicable to *strategies* and *portfolios*. It is also instructive to look at the performance of individual trades and many measures exist to characterise their performance. In particular, we will quantify the number of winning/losing trades, mean profit per trade and win/loss ratio among others.

Trades are the most granular aspect of an algorithmic strategy and hence we will begin by discussing trade analysis.

12.1 Trade Analysis

The first step in analysing any strategy is to consider the performance of the actual trades. Such metrics can vary dramatically between strategies. A classic example would be the difference in performance metrics of a trend-following strategy when compared to a mean-reverting strategy.

Trend-following strategies usually consist of many losing trades, each with a likely small loss. The lesser quantity of profitable trades occur when a trend has been established and the performance from these positive trades can significantly exceed the losses of the larger quantity of losing trades. Pair-trading mean-reverting strategies display the opposing character. They generally consist of many small profitable trades. However, if a series does not mean revert in the manner expected then the long/short nature of the strategy can lead to substantial losses. This could potentially wipe out the large quantity of small gains.

It is essential to be aware of the nature of the trade profile of the strategy and your own psychological profile, as the two will need to be in alignment. Otherwise you will find that you may not be able to persevere through a period of tough drawdown.

We now review the statistics that are of interest to us as the trade level.

12.1.1 Summary Statistics

When considering our trades, we are interested in the following set of statistics. Here "period" refers to the time period covered by the trading bar containing OHLCV data. For long-term strategies it is often the case that daily bars are used. For higher frequency strategies we may be interested in hourly or minutely bars.

- **Total Profit/Loss (PnL)** - The total PnL straightforwardly states whether a particular trade has been profitable or not.
- **Average Period PnL** - The avg. period PnL states whether a bar, on average, generates a profit or loss.
- **Maximum Period Profit** - The largest bar-period profit made by this trade so far.
- **Maximum Period Loss** - The largest bar-period loss made by this trade so far. Note that this says nothing about future maximum period loss! A future loss could be much larger than this.
- **Average Period Profit** - The average over the trade lifetime of all profitable periods.
- **Average Period Loss** - The average over the trade lifetime of all unprofitable periods.
- **Winning Periods** - The count of all winning periods.
- **Losing Periods** - The count of all losing periods.
- **Percentage Win/Loss Periods** - The percentage of all winning periods to losing periods. Will differ markedly for trend-following and mean-reverting type strategies.

Thankfully, it is straightforward to generate this information from our portfolio output and so the need for manual record keeping is completely eliminated. However, this leads to the danger that we never actually stop to analyse the data!

It is imperative that trades are evaluated at least once or twice a month. Doing so is a useful early warning detection system that can help identify when strategy performance begins to degrade. It is often much better than simply considering the cumulative PnL alone.

12.2 Strategy and Portfolio Analysis

Trade-level analysis is extremely useful in longer-term strategies, particularly with strategies that employ complex trades, such as those that involve derivatives. For higher-frequency strategies, we will be less interested in any individual trade and instead will want to consider the performance measures of the strategy instead. Obviously for longer-term strategies, we are equally as interested in the overall strategy performance. We are primarily interested in the following three key areas:

- **Returns Analysis** - The returns of a strategy encapsulate the concept of profitability. In institutional settings they are generally quoted net of fees and so provide a true picture of how much money was made on money invested. Returns can be tricky to calculate, especially with cash inflows/outflows.
- **Risk/Reward Analysis** - Generally the first consideration that external investors will have in a strategy is its out of sample *Sharpe Ratio* (which we describe below). This is an industry standard metric which attempts to characterise how much return was achieved per unit of risk.
- **Drawdown Analysis** - In an institutional setting, this is probably the most important of the three aspects. The profile and extent of the drawdowns of a strategy, portfolio or fund form a key component in risk management. We'll define drawdowns below.

Despite the fact that I have emphasised their institutional performance, as a retail trader these are still highly important metrics and with suitable risk management (see next chapter) will form the basis of a continual strategy evaluation procedure.

12.2.1 Returns Analysis

The most widely quoted figures when discussing strategy performance, in both institutional and retail settings, are often *total return*, *annual returns* and *monthly returns*. It is extremely common to see a hedge fund performance newsletter with a monthly return "grid". In addition, everybody will want to know what the "return" of the strategy is.

Total return is relatively straightforward to calculate, at least in a retail setting with no external investors or cash inflows/outflows. In percentage terms it is simply calculated as:

$$r_t = (P_f - P_i)/P_i \times 100 \quad (12.1)$$

Where r_t is the total return, P_f is the final portfolio dollar value and P_i is the initial portfolio value. We are mostly interested in *net* total return, that is the value of the portfolio/fund after all trading/business costs have been deducted.

Note that this formula is only applicable to long-only un-leveraged portfolios. If we wish to add in short selling or leverage we need to modify how we calculate returns because we are technically trading on a larger borrowed portfolio than that used here. This is known as a *margin portfolio*.

For instance, consider the case where a trading strategy has gone long 1,000 USD of one asset and then shorted 1,000 USD of another asset. This is a *dollar-neutral* portfolio and the total *notional traded* is 2,000 USD. If 200 USD was generated from this strategy then gross return on this notional is 10%. It becomes more complex when you factor in borrowing costs and interest rates to fund the margin. Factoring in these costs leads to the net total return, which is the value that is often quoted as "total return".

Equity Curve

The equity curve is often one of the most emphasised visualisations on a hedge fund performance report - assuming the fund is doing well! It is a plot of the portfolio value of the fund over time. In essence it is used to show how the account has grown since fund inception. Equally, in a retail setting it is used to show growth of account equity through time. See Fig 12.2.1 for a typical equity curve plot:

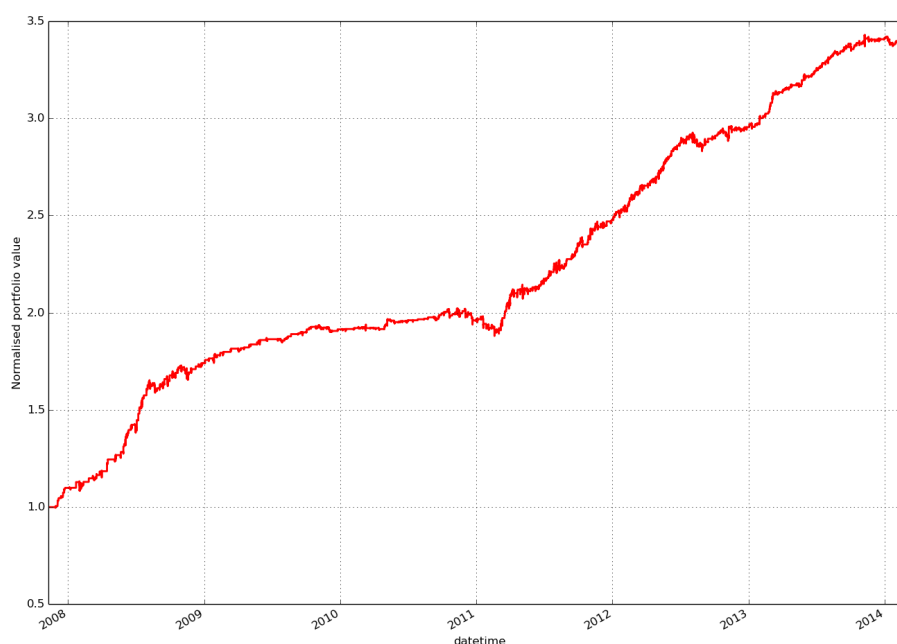


Figure 12.1: Typical intraday strategy equity curve

What is the benefit of such a plot? In essence it gives a "flavour" as to the past volatility of the strategy, as well as a visual indication of whether the strategy has suffered from prolonged periods of plateau or even drawdown. It essentially provides answers as to how the total return figure calculated at the end of the strategy trading period was arrived at.

In an equity curve we are seeking to determine how unusual historical events have shaped the strategy. For instance, a common question asks if there was excess volatility in the strategy around 2008. Another question might concern its consistency of returns.

One must be extremely careful with interpretation of equity curves as when marketed they are generally shown as "upward sloping lines". Interesting insight can be gained via truncation of such curves, which can emphasise periods of intense volatility or prolonged drawdown that may otherwise not seem as severe when considering the whole time period. Thus an equity curve needs to be considered in context with other analysis, in particular risk/reward analysis and drawdown analysis.

12.2.2 Risk/Reward Analysis

As we alluded to above the concept of risk-to-reward analysis is extremely important in an institutional setting. This **does not** mean that as a retail investor we can ignore the concept. You should pay significant attention to risk/reward metrics for your strategy as they will have a significant impact on your drawdowns, leverage and overall compound growth rate.

These concepts will be expanded on in the next chapter on Risk and Money Management. For now we will discuss the common ratios, and in particular the Sharpe Ratio, which is ubiquitous as a comparative measure in quantitative finance. Since it is held in such high regard across institutionalised quantitative trading, we will go into a reasonable amount of detail.

Sharpe Ratio

Consider the situation where we are presented with two strategies possessing identical returns. How do we know which one contains more risk? Further, what do we even mean by "more risk"? In finance, we are often concerned with volatility of returns and periods of drawdown. Thus if one of these strategies has a significantly higher volatility of returns we would likely find it less attractive, despite the fact that its historical returns might be similar if not identical. These problems of strategy comparison and risk assessment motivate the use of the **Sharpe Ratio**.

William Forsyth Sharpe is a Nobel-prize winning economist, who helped create the Capital Asset Pricing Model (CAPM) and developed the Sharpe Ratio in 1966 (later updated in 1994). The Sharpe Ratio S is defined by the following relation:

$$S = \frac{\mathbb{E}(R_a - R_b)}{\sqrt{\text{Var}(R_a - R_b)}} \quad (12.2)$$

Where R_a is the period return of the asset or strategy and R_b is the period return of a suitable *benchmark*, such as a risk-free interest rate.

The ratio compares the mean average of the *excess returns* of the asset or strategy with the standard deviation of those excess returns. Thus a lower volatility of returns will lead to a greater Sharpe ratio, assuming identical mean returns.

The "Sharpe Ratio" often quoted by those carrying out trading strategies is the **annualised Sharpe**, the calculation of which depends upon the trading period of which the returns are measured. Assuming there are N trading periods in a year, the annualised Sharpe is calculated as follows:

$$S_A = \sqrt{N} \frac{\mathbb{E}(R_a - R_b)}{\sqrt{\text{Var}(R_a - R_b)}}$$

Note that the Sharpe ratio itself **MUST** be calculated based on the Sharpe of that particular time period type. For a strategy based on trading period of days, $N = 252$ (as there are 252 trading days in a year, not 365), and R_a, R_b must be the daily returns. Similarly for hours $N = 252 \times 6.5 = 1638$, not $N = 252 \times 24 = 6048$, since there are only 6.5 hours in a trading day (at least for most US equities markets!).

The formula for the Sharpe ratio above alludes to the use of a *benchmark*. A benchmark is used as a "yardstick" or a "hurdle" that a particular strategy must overcome for it to worth consideration. For instance, a simple long-only strategy using US large-cap equities should hope to beat the S&P500 index on average, or match it for less volatility, otherwise what is to be gained by not simply investing in the index at far lower management/performance fees?

The choice of benchmark can sometimes be unclear. For instance, should a sector Exchange Traded Fund (ETF) be utilised as a performance benchmark for individual equities, or the S&P500 itself? Why not the Russell 3000? Equally should a hedge fund strategy be benchmarking itself against a market index or an index of other hedge funds?

There is also the complication of the "risk free rate". Should domestic government bonds be used? A basket of international bonds? Short-term or long-term bills? A mixture? Clearly there are plenty of ways to choose a benchmark. The Sharpe ratio generally utilises the risk-free rate and often, for US equities strategies, this is based on 10-year government Treasury bills.

In one particular instance, for market-neutral strategies, there is a particular complication regarding whether to make use of the risk-free rate or zero as the benchmark. The market index itself should not be utilised as the strategy is, by design, market-neutral. The correct choice for a market-neutral portfolio is not to subtract the risk-free rate because it is *self-financing*. Since you gain a credit interest, R_f , from holding a margin, the actual calculation for returns

is: $(R_a + R_f) - R_f = R_a$. Hence there is no actual subtraction of the risk-free rate for dollar neutral strategies.

Despite the prevalence of the Sharpe ratio within quantitative finance, it does suffer from some limitations. The Sharpe ratio is *backward looking*. It only accounts for *historical* returns distribution and volatility, not those occurring in the *future*. When making judgements based on the Sharpe ratio there is an implicit assumption that the past will be similar to the future. This is evidently not always the case, particular under *market regime changes*.

The Sharpe ratio calculation assumes that the returns being used are normally distributed (i.e. *Gaussian*). Unfortunately, markets often suffer from kurtosis above that of a normal distribution. Essentially the distribution of returns has "fatter tails" and thus extreme events are more likely to occur than a Gaussian distribution would lead us to believe. Hence, the Sharpe ratio is poor at characterising *tail risk*.

This can be clearly seen in strategies which are highly prone to such risks. For instance, the sale of call options aka "pennies under a steam roller". A steady stream of option premia are generated by the sale of call options over time, leading to a low volatility of returns, with a strong excess returns above a benchmark. In this instance the strategy would possess a high Sharpe ratio based on historical data. However, it does not take into account that such options may be *called*, leading to significant drawdowns or even wipeout in the equity curve. Hence, as with any measure of algorithmic trading strategy performance the Sharpe ratio cannot be used in isolation.

Although this point might seem obvious to some, transaction costs **MUST** be included in the calculation of Sharpe ratio in order for it to be realistic. There are countless examples of trading strategies that have high Sharpes, and thus a likelihood of great profitability, only to be reduced to low Sharpe, low profitability strategies once realistic costs have been factored in. This means making use of the *net returns* when calculating in excess of the benchmark. Hence transaction costs must be factored in *upstream* of the Sharpe ratio calculation.

One obvious question that has remained unanswered thus far is "What is a good Sharpe Ratio for a strategy?". This is actually quite a difficult question to answer because each investor has a differing risk profile. The general rule of thumb is that quantitative strategies with annualised Sharpe Ratio $S < 1$ should not often be considered. However, there are exceptions to this, particularly in the trend-following futures space.

Quantitative funds tend to ignore any strategies that possess a Sharpe ratios $S < 2$. One prominent quantitative hedge fund that I am familiar with wouldn't even consider strategies that had Sharpe ratios $S < 3$ while in research. As a retail algorithmic trader, if you can achieve an out of sample (i.e. live trading!) Sharpe ratio $S > 2$ then you are doing very well.

The Sharpe ratio will often increase with trading frequency. Some high frequency strategies will have high single (and sometimes low double) digit Sharpe ratios, as they can be profitable almost every day and certainly every month. These strategies rarely suffer from catastrophic risk (in the sense of great loss) and thus minimise their volatility of returns, which leads to such high Sharpe ratios. Be aware though that high-frequency strategies such as these can simply cease to function very suddenly, which is another aspect of risk not fully reflected in the Sharpe ratio.

Let's now consider some actual Sharpe examples. We will start simply, by considering a long-only buy-and-hold of an individual equity then consider a market-neutral strategy. Both of these examples have been carried out with Pandas.

The first task is to actually obtain the data and put it into a Pandas DataFrame object. In the prior chapter on securities master implementation with Python and MySQL we created a system for achieving this. Alternatively, we can make use of this simpler code to grab Google Finance data directly and put it straight into a DataFrame. At the bottom of this script I have created a function to calculate the annualised Sharpe ratio based on a time-period returns stream:

```
#!/usr/bin/python
# -*- coding: utf-8 -*-

# sharpe.py

from __future__ import print_function
```

```

import datetime
import numpy as np
import pandas as pd
import pandas.io.data as web

def annualised_sharpe(returns, N=252):
    """
    Calculate the annualised Sharpe ratio of a returns stream
    based on a number of trading periods, N. N defaults to 252,
    which then assumes a stream of daily returns.

    The function assumes that the returns are the excess of
    those compared to a benchmark.
    """
    return np.sqrt(N) * returns.mean() / returns.std()

```

Now that we have the ability to obtain data from Google Finance and straightforwardly calculate the annualised Sharpe ratio, we can test out a buy and hold strategy for two equities. We will use Google (GOOG) from Jan 1st 2000 to Jan 1st 2013.

We can create an additional helper function that allows us to quickly see buy-and-hold Sharpe across multiple equities for the same (hardcoded) period:

```

def equity_sharpe(ticker):
    """
    Calculates the annualised Sharpe ratio based on the daily
    returns of an equity ticker symbol listed in Google Finance.

    The dates have been hardcoded here for brevity.
    """
    start = datetime.datetime(2000,1,1)
    end = datetime.datetime(2013,1,1)

    # Obtain the equities daily historic data for the desired time period
    # and add to a pandas DataFrame
    pdf = web.DataReader(ticker, 'google', start, end)

    # Use the percentage change method to easily calculate daily returns
    pdf['daily_ret'] = pdf['Close'].pct_change()

    # Assume an average annual risk-free rate over the period of 5%
    pdf['excess_daily_ret'] = pdf['daily_ret'] - 0.05/252

    # Return the annualised Sharpe ratio based on the excess daily returns
    return annualised_sharpe(pdf['excess_daily_ret'])

```

For Google, the Sharpe ratio for buying and holding is 0.703:

```

>>> equity_sharpe('GOOG')
0.70265563285799615

```

Now we can try the same calculation for a market-neutral strategy. The goal of this strategy is to fully isolate a particular equity's performance from the market in general. The simplest way to achieve this is to go short an equal amount (in dollars) of an Exchange Traded Fund (ETF) that is designed to track such a market. The most obvious choice for the US large-cap equities market is the S&P500 index, which is tracked by the SPDR ETF, with the ticker of SPY.

To calculate the annualised Sharpe ratio of such a strategy we will obtain the historical prices for SPY and calculate the percentage returns in a similar manner to the previous stocks, with

the exception that we will not use the risk-free benchmark. We will calculate the *net daily returns* which requires subtracting the difference between the long and the short returns and then dividing by 2, as we now have twice as much trading capital. Here is the Python/pandas code to carry this out:

```
def market_neutral_sharpe(ticker, benchmark):
    """
    Calculates the annualised Sharpe ratio of a market
    neutral long/short strategy involving the long of 'ticker'
    with a corresponding short of the 'benchmark'.
    """
    start = datetime.datetime(2000, 1, 1)
    end = datetime.datetime(2013, 1, 1)

    # Get historic data for both a symbol/ticker and a benchmark ticker
    # The dates have been hardcoded, but you can modify them as you see fit!
    tick = web.DataReader(ticker, 'google', start, end)
    bench = web.DataReader(benchmark, 'google', start, end)

    # Calculate the percentage returns on each of the time series
    tick['daily_ret'] = tick['Close'].pct_change()
    bench['daily_ret'] = bench['Close'].pct_change()

    # Create a new DataFrame to store the strategy information
    # The net returns are (long - short)/2, since there is twice
    # the trading capital for this strategy
    strat = pd.DataFrame(index=tick.index)
    strat['net_ret'] = (tick['daily_ret'] - bench['daily_ret'])/2.0

    # Return the annualised Sharpe ratio for this strategy
    return annualised_sharpe(strat['net_ret'])
```

For Google, the Sharpe ratio for the long/short market-neutral strategy is **0.832**:

```
>>> market_neutral_sharpe('GOOG', 'SPY')
0.83197496084314604
```

We will now briefly consider other risk/reward ratios.

Sortino Ratio

The Sortino ratio is motivated by the fact that the Sharpe ratio captures both upward and downward volatility in its denominator. However, investors (and hedge fund managers) are generally not too bothered when we have significant upward volatility! What is actually of interest from a risk management perspective is downward volatility and periods of drawdown.

Thus the Sortino ratio is defined as the mean excess return divided by the mean downside deviation:

$$\text{Sortino} = \frac{\mathbb{E}(R_a - R_b)}{\sqrt{\text{Var}(R_a - R_b)_d}} \quad (12.3)$$

The Sortino is sometimes quoted in an institutional setting, but is certainly not as prevalent as the Sharpe ratio.

CALMAR Ratio

One could also argue that investors/traders are concerned solely with the maximum extent of the drawdown, rather than the average drawdown. This motivates the CALMAR (CALifornia

Managed Accounts Reports) ratio, also known as the Drawdown ratio, which provides a ratio of mean excess return to the maximum drawdown:

$$\text{CALMAR} = \frac{\mathbb{E}(R_a - R_b)}{\text{max. drawdown}} \quad (12.4)$$

Once again, the CALMAR is not as widely used as the Sharpe ratio.

12.2.3 Drawdown Analysis

In my opinion the concept of drawdown is the most important aspect of performance measurement for an algorithmic trading system. Simply put, if your account equity is wiped out then none of the other performance metrics matter! Drawdown analysis concerns the measurement of drops in account equity from previous *high water marks*. A high water mark is defined as the last account equity peak reached on the equity curve.

In an institutional setting the concept of drawdown is especially important as most hedge funds are remunerated only when the account equity is continually creating new high water marks. That is, a fund manager is not paid a performance fee while the fund remains "under water", i.e. the account equity is in a period of *drawdown*.

Most investors would be concerned at a drawdown of 10% in a fund, and would likely redeem their investment once a drawdown exceeds 30%. In a retail setting the situation is very different. Individuals are likely to be able to suffer deeper drawdowns in the hope of gaining higher returns.

Maximum Drawdown and Duration

The two key drawdown metrics are the *maximum drawdown* and the *drawdown duration*. The first describes the largest percentage drop from a previous peak to the current or previous trough in account equity. It is often quoted in an institutional setting when trying to market a fund. Retail traders should also pay significant attention to this figure. The second describes the actual duration of the drawdown. This figure is usually quoted in days, but higher frequency strategies might use a more granular time period.

In backtests these measures provide *some* idea of how a strategy *might* perform in the future. The overall account equity curve might look quite appealing after a calculated backtest. However, an upward equity curve can easily mask how difficult previous periods of drawdown might actually have been to experience.

When a strategy begins dropping below 10% of account equity, or even below 20%, it requires significant willpower to continue with the strategy, despite the fact that the strategy may have historically, at least in the backtests, been through similar periods. This is a consistent issue with algorithmic trading and systematic trading in general. It naturally motivates the need to set prior drawdown boundaries and specific rules, such as an account-wide "stop loss" that will be carried out in the event of a drawdown breaching these levels.

Drawdown Curve

While it is important to be aware of the maximum drawdown and drawdown duration, it is significantly more instructive to see a time series plot of the strategy drawdown over the trading duration.

Fig 12.2.3 quite clearly shows that this particular strategy suffered from a relatively sustained period of drawdown beginning in Q3 of 2010 and finishing in Q2 of 2011, reaching a maximum drawdown of 14.8%. While the strategy itself continued to be significantly profitable over the long term, this particular period would have been very difficult to endure. In addition, this is the maximum *historical* drawdown that has occurred *to date*. The strategy may be subject to an even greater drawdown in the future. Thus it is necessary to consider drawdown curves, as with other historical looking performance measures, in the context with which they have been generated, namely via historical, and not future, data.

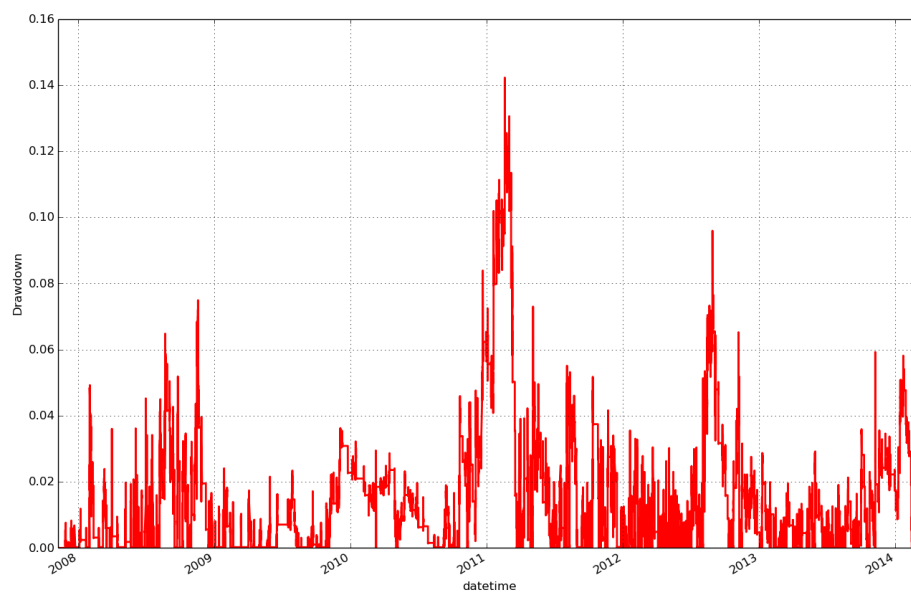


Figure 12.2: Typical intraday strategy drawdown curve

In the following chapter we will consider the concept of quantitative risk management and describe techniques that can help us to minimise drawdowns and maximise returns, all the while keeping to a reasonable degree of risk.

Chapter 13

Risk and Money Management

This chapter is concerned with managing risk as applied to quantitative trading strategies. This usually comes in two flavours, firstly identifying and mitigating internal and external factors that can affect the performance or operation of an algorithmic trading strategy and secondly, how to optimally manage the strategy portfolio in order to maximise growth rate and minimise account drawdowns.

In the first section we will consider different sources of risk (both intrinsic and extrinsic) that might affect the long-term performance of an algorithmic trading business - either retail or institutional.

In the second section we will look at money management techniques that can simultaneously protect our portfolio from ruin and also attempt to maximise the long-term growth rate of equity.

In the final section we consider institutional-level risk management techniques that can easily be applied in a retail setting to help protect trading capital.

13.1 Sources of Risk

There are numerous sources of risk that can have an impact on the correct functioning of an algorithmic trading strategy. "Risk" is usually defined in this context to mean chance of account losses. However, I am going to define it in a much broader context to mean any factor that provides a degree of uncertainty and could affect the performance of our strategies or portfolio.

The broad areas of risk that we will consider include **Strategy Risk**, **Portfolio Risk**, **Market Risk**, **Counterparty Risk** and **Operational Risk**.

13.1.1 Strategy Risk

Strategy risk, or *model risk*, encompasses the class of risks that arise from the design and implementation of a trading strategy based on a statistical model. It includes all of the previous issues we have discussed in the Successful Backtesting chapter, such as curve-fitting, survivorship bias and look-ahead bias. It also includes other topics related directly to the statistical analysis of the strategy model.

Any statistical model is based on assumptions. These assumptions are sometimes not considered in proper depth or ignored entirely. This means that the statistical model based upon these assumptions may be inappropriate and hence lead to poor predictive or inferential capability. A general example occurs in the setting of linear regression. Linear regression makes the assumption that the response data are *homoscedastic* (i.e. the responses have a constant variance in their errors). If this is not the case then linear regression provides less precision in the estimates of parameters.

Many quantitative strategies make use of *descriptive statistics* of historical price data. In particular, they will often use *moments* of the data such as the mean, variance, skew and kurtosis of strategy returns. Such models (including the Kelly Criterion outlined below) generally rely on these moments being constant in time. Under a *market regime change* these moments can be

drastically altered and hence lead to degradation of the model. Models with "rolling parameters" are usually utilised in order to mitigate this issue.

13.1.2 Portfolio Risk

A Portfolio contains one or more strategies. Thus it is indirectly subject to Strategy Risk as outlined above. In addition there are specific risks that occur at the portfolio level. These are usually only considered in an institutional setting or in a high-end retail setting where portfolio tracking is being carried out on a stable of trading strategies.

When regressing portfolio returns to a set of *factors*, such as industry sectors, asset classes or groups of financial entities it is possible to ascertain if the portfolio is heavily "loaded" into a particular factor. For instance, an equities portfolio may be extremely heavy on technology stocks and thus is extremely exposed to any issues that affect the tech sector as a whole. Hence it is often necessary - at the portfolio level - to override particular strategies in order to account for overloaded factor risk. This is often a more significant concern in an institutional setting where there is more capital to be allocated and the preservation of capital takes precedence to the long-term growth rate of the capital. However, it should certainly be considered even as a retail algorithmic trading investor.

Another issue that is largely an institutional issue (unless trading more illiquid assets) are limits on daily trading volume. For retail traders, executing strategies in the large-cap or commodities futures markets, there is no real concern regarding market impact. However, in less liquid instruments one has to be careful to not be trading a significant percentage of the daily traded volume, due to potential market impact and thus invalidation of a previously backtested trading model (which often do not take into account market impact). To avoid this it is necessary to calculate the *average daily volume* (using a mean over a loopback period, for instance) and stay within small percentage limits of this figure.

Running a portfolio of strategies brings up the issue of *strategy correlation*. Correlations can be estimated via statistical techniques such as the Pearson Product Moment Correlation Coefficient. However, correlation itself is not a static entity and is also subject to swift change, especially under market-wide liquidity constraints, often known as *financial contagion*. In general, strategies should be designed to avoid correlation with each other by virtue of differing asset classes or time horizons. Rolling correlations can be estimated over a large time-period and should be a standard part of your backtest, if considering a portfolio approach.

13.1.3 Counterparty Risk

Counterparty risk is generally considered a form of *credit risk*. It is the risk that a counterparty will not pay an obligation on a financial asset under which they are liable. There is an entire subset of quantitative finance related to the pricing of counterparty risk hedging instruments, but this is not of primary interest to us as retail algorithmic traders. We are more concerned with the risk of default from *suppliers* such as an exchange or brokerage.

While this may seem academic, I can personally assure you that these issues are quite real! In an institutional setting I have experienced first hand a brokerage bankruptcy under conditions that meant not all of the trading capital was returned. Thus I now factor such risks into a portfolio. The suggested means of mitigating this issue is to utilise multiple brokerages, although when trading under margin this can make the trading logistics somewhat tricky.

Counterparty risk is generally more of a concern in an institutional setting so I won't dwell on it too much here!

13.1.4 Operational Risk

Operation risk encompasses sources of risk from within a fund or trading operational infrastructure, including business/entrepreneurial risk, IT risk and external regulatory or legal changes. These topics aren't often discussed in any great depth, which I believe is somewhat short-sighted since they have the potential to completely halt a trading operation permanently.

Infrastructure risk is often associated with information technology systems and other related trading infrastructure. This also includes employee risk (such as fraud, sudden departure). As the scale of an infrastructure grows so does the likelihood of the "single point of failure" (SPOF). This is a critical component in the trading infrastructure that, under malfunction, can lead to a catastrophe halting of the entire operation. In the IT sense, this is usually the consequence of a badly thought out architecture. In a non-IT sense this can be the consequence of a badly designed organisation chart.

These issues are still entirely relevant for the retail trader. Often, an IT/trading infrastructure can end up being "patchy" and "hacked together". In addition, poor record keeping and other administrative failures can lead to huge potential tax burdens. Thankfully, "cloud" architecture provides the ability for redundancy in systems and automation of processes can lead to solid administrative habits. This type of behaviour, that is consideration of risks from sources other than the market and the strategy, can often make the difference between a successful long-term algorithmic trader and the individual who gives up due to catastrophic operation breakdown.

An issue that affects the hedge fund world is that of reporting and compliance. Post-2008 legislation has put a heavy burden on asset management firms, which can have a large impact on their cash-flow and operating expenditure. For an individual thinking of incorporating such a firm, in order to expand a strategy or run with external funds, it is prudent to keep on top of the legislation and regulatory environment, since it is somewhat of a "moving target".

13.2 Money Management

This section deals with one of the most fundamental concepts in trading - both discretionary and algorithmic - namely, money management. A naive investor/trader might believe that the only important investment objective is to simply make as much money as possible. However the reality of long-term trading is more complex. Since market participants have differing risk preferences and constraints there are many objectives that investors may possess.

Many retail traders consider the *only* goal to be a continual increase of account equity, with little or no consideration given to the "risk" of a strategy that achieves this growth. More sophisticated retail investors measure account drawdowns, and depending upon their risk preferences, may be able to cope with a substantial drop in account equity (say 50%). The reason they can deal with a drawdown of this magnitude is that they realise, quantitatively, that this behaviour may be optimal for the long-term growth rate of the portfolio, via the use of *leverage*.

An institutional investor is likely to consider risk in a different light. Often institutional investors have mandated maximum drawdowns (say 20%), with significant consideration given to sector allocation and average daily volume limits. They would be additional constraints on the "optimisation problem" of capital allocation to strategies. These factors might even be more important than maximising the long-term growth rate of the portfolio.

Thus we are in a situation where we can strike a balance between maximising long-term growth rate via leverage and minimising our "risk" by trying to limit the duration and extent of the drawdown. The major tool that will help us achieve this is called the Kelly Criterion.

13.2.1 Kelly Criterion

Within this section the Kelly Criterion is going to be our tool to control leverage of, and allocation towards, a set of algorithmic trading strategies that make up a multi-strategy portfolio.

We will define **leverage** as the ratio of the size of a portfolio to the actual account equity within that portfolio. To make this clear we can use the analogy of purchasing a house with a mortgage. Your down payment (or "deposit" for those of us in the UK!) constitutes your account equity, while the down payment plus the mortgage value constitutes the equivalent of the size of a portfolio. Thus a down payment of 50,000 USD on a 200,000 USD house (with a mortgage of 150,000 USD) constitutes a leverage of $(150000 + 50000)/50000 = 4$. Thus in this instance you would be 4x leveraged on the house. A margin account portfolio behaves similarly. There is a "cash" component and then more stock can be borrowed on margin, to provide the leverage.

Before we state the Kelly Criterion specifically I want to outline the assumptions that go into its derivation, which have varying degrees of accuracy:

- Each algorithmic trading strategy will be assumed to possess a returns stream that is *normally distributed* (i.e. *Gaussian*). Further, each strategy has its own *fixed* mean and standard deviation of returns. The formula assumes that these mean and std values *do not change*, i.e. that they are same in the past as in the future. This is clearly not the case with most strategies, so be aware of this assumption.
- The returns being considered here are *excess returns*, which means they are net of all financing costs such as interest paid on margin and transaction costs. If the strategy is being carried out in an institutional setting, this also means that the returns are net of management and performance fees.
- All of the trading profits are reinvested and no withdrawals of equity are carried out. This is clearly not as applicable in an institutional setting where the above mentioned management fees are taken out and investors often make withdrawals.
- All of the strategies are statistically independent (there is no correlation between strategies) and thus the covariance matrix between strategy returns is diagonal.

Now we come to the actual Kelly Criterion! Let's imagine that we have a set of N algorithmic trading strategies and we wish to determine both how to apply optimal leverage per strategy in order to maximise growth rate (but minimise drawdowns) and how to allocate capital between each strategy. If we denote the allocation between each strategy i as a vector f of length N , s.t. $f = (f_1, \dots, f_N)$, then the Kelly Criterion for optimal allocation to each strategy f_i is given by:

$$f_i = \mu_i / \sigma_i^2 \quad (13.1)$$

Where μ_i are the mean excess returns and σ_i are the standard deviation of excess returns for a strategy i . This formula essentially describes the optimal leverage that should be applied to each strategy.

While the Kelly Criterion f_i gives us the optimal leverage and strategy allocation, we still need to actually calculate our expected long-term compounded growth rate of the portfolio, which we denote by g . The formula for this is given by:

$$g = r + S^2/2 \quad (13.2)$$

Where r is the risk-free interest rate, which is the rate at which you can borrow from the broker, and S is the annualised Sharpe Ratio of the strategy. The latter is calculated via the annualised mean excess returns divided by the annualised standard deviations of excess returns. See the previous chapter on Performance Measurement for details on the Sharpe Ratio.

A Realistic Example

Let's consider an example in the single strategy case ($i = 1$). Suppose we go long a mythical stock XYZ that has a mean annual return of $m = 10.7\%$ and an annual standard deviation of $\sigma = 12.4\%$. In addition suppose we are able to borrow at a risk-free interest rate of $r = 3.0\%$. This implies that the mean excess returns are $\mu = m - r = 10.7 - 3.0 = 7.7\%$. This gives us a Sharpe Ratio of $S = 0.077/0.124 = 0.62$.

With this we can calculate the optimal Kelly leverage via $f = \mu/\sigma^2 = 0.077/0.124^2 = 5.01$. Thus the Kelly leverage says that for a 100,000 USD portfolio we should borrow an additional 401,000 USD to have a total portfolio value of 501,000 USD. *In practice it is unlikely that our brokerage would let us trade with such substantial margin and so the Kelly Criterion would need to be adjusted.*

We can then use the Sharpe ratio S and the interest rate r to calculate g , the expected long-term compounded growth rate. $g = r + S^2/2 = 0.03 + 0.62^2/2 = 0.22$, i.e. 22%. Thus we should *expect* a return of 22% a year from this strategy.

Kelly Criterion in Practice

It is important to be aware that the Kelly Criterion requires a continuous rebalancing of capital allocation in order to remain valid. Clearly this is not possible in the discrete setting of actual trading and so an approximation must be made. The standard "rule of thumb" here is to update the Kelly allocation once a day. Further, the Kelly Criterion itself should be recalculated periodically, using a trailing mean and standard deviation with a lookback window. Again, for a strategy that trades roughly once a day, this lookback should be set to be on the order of 3-6 months of daily returns.

Here is an example of rebalancing a portfolio under the Kelly Criterion, which can lead to some counter-intuitive behaviour. Let's suppose we have the strategy described above. We have used the Kelly Criterion to borrow cash to size our portfolio to 501,000 USD. Let's assume we make a healthy 5% return on the following day, which boosts our account size to 526,050 USD. The Kelly Criterion tells us that we should borrow *more* to keep the same leverage factor of 5.01. In particular our account equity is 126,050 USD on a portfolio of 526,050, which means that the current leverage factor is 4.17. To increase it to 5.01, we need to borrow an additional 105,460 USD in order to increase our account size to 631,510.5 USD (this is 5.01×126050).

Now consider that the following day we lose 10% on our portfolio (ouch!). This means that the total portfolio size is now 568,359.45 USD (631510.5×0.9). Our total account *equity* is now 62,898.95 USD ($126050 - 631510.45 \times 0.1$). This means our current leverage factor is $568359.45/62898.95 = 9.03$. Hence we need to reduce our account by *selling* 253,235.71 USD of stock in order to reduce our total portfolio value to 315,123.73 USD, such that we have a leverage of 5.01 again ($315123.73/62898.95 = 5.01$).

Hence we have *bought* into a profit and *sold* into a loss. This process of selling into a loss may be extremely emotionally difficult, but it is mathematically the "correct" thing to do, assuming that the assumptions of Kelly have been met! It is the approach to follow in order to maximise long-term compounded growth rate.

You may have noticed that the absolute values of money being re-allocated between days were rather severe. This is a consequence of both the artificial nature of the example and the extensive leverage employed. 10% loss in a day is not particularly common in higher-frequency algorithmic trading, but it does serve to show how extensive leverage can be on absolute terms.

Since the estimation of means and standard deviations are always subject to uncertainty, in practice many traders tend to use a more conservative leverage regime such as the Kelly Criterion divided by two, affectionately known as "half-Kelly". The Kelly Criterion should really be considered as an upper bound of leverage to use, rather than a direct specification. If this advice is not heeded then using the direct Kelly value can lead to ruin (i.e. account equity disappearing to zero) due to the non-Gaussian nature of the strategy returns.

Should You Use The Kelly Criterion?

Every algorithmic trader is different and the same is true of risk preferences. When choosing to employ a leverage strategy (of which the Kelly Criterion is one example) you should consider the risk mandates that you need to work under. In a retail environment you are able to set your own maximum drawdown limits and thus your leverage can be increased. In an institutional setting you will need to consider risk from a very different perspective and the leverage factor will be one component of a much larger framework, usually under many other constraints.

13.3 Risk Management

13.3.1 Value-at-Risk

Estimating the risk of loss to an algorithmic trading strategy, or portfolio of strategies, is of extreme importance for long-term capital growth. Many techniques for risk management have been developed for use in institutional settings. One technique in particular, known as **Value at Risk** or **VaR**, will be the topic of this section.

We will be applying the concept of VaR to a single strategy or a set of strategies in order to help us quantify risk in our trading portfolio. The definition of VaR is as follows:

VaR provides an *estimate*, under a given degree of confidence, of the size of a loss from a portfolio over a given time period.

In this instance "portfolio" can refer to a single strategy, a group of strategies, a trader's book, a prop desk, a hedge fund or an entire investment bank. The "given degree of confidence" will be a value of, say, 95% or 99%. The "given time period" will be chosen to reflect one that would lead to a minimal *market impact* if a portfolio were to be liquidated.

For example, a VaR equal to 500,000 USD at 95% confidence level for a time period of a day would simply state that there is a 95% probability of losing no more than 500,000 USD in the following day. Mathematically this is stated as:

$$P(L \leq -5.0 \times 10^5) = 0.05 \quad (13.3)$$

Or, more generally, for loss L exceeding a value VaR with a confidence level c we have:

$$P(L \leq -VaR) = 1 - c \quad (13.4)$$

The "standard" calculation of VaR makes the following assumptions:

- **Standard Market Conditions** - VaR is not supposed to consider extreme events or "tail risk", rather it is supposed to provide the expectation of a loss under normal "day-to-day" operation.
- **Volatilities and Correlations** - VaR requires the volatilities of the assets under consideration, as well as their respective correlations. These two quantities are tricky to estimate and are subject to continual change.
- **Normality of Returns** - VaR, in its standard form, assumes the returns of the asset or portfolio are *normally distributed*. This leads to more straightforward analytical calculation, but it is quite unrealistic for most assets.

13.4 Advantages and Disadvantages

VaR is pervasive in the financial industry, hence you should be familiar with the benefits and drawbacks of the technique. Some of the advantages of VaR are as follows:

- VaR is very straightforward to calculate for individual assets, algo strategies, quant portfolios, hedge funds or even bank prop desks.
- The time period associated with the VaR can be modified for multiple trading strategies that have different time horizons.
- Different values of VaR can be associated with different forms of risk, say broken down by asset class or instrument type. This makes it easy to interpret where the majority of portfolio risk may be clustered, for instance.
- Individual strategies can be constrained as can entire portfolios based on their individual VaR.
- VaR is straightforward to interpret by (potentially) non-technical external investors and fund managers.

However, VaR is not without its disadvantages:

- VaR does not discuss the magnitude of the expected loss beyond the value of VaR, i.e. it will tell us that we are likely to see a loss *exceeding* a value, but not how much it exceeds it.

- It does not take into account extreme events, but only typical market conditions.
- Since it uses historical data (it is rearward-looking) it will not take into account future market regime shifts that can change volatilities and correlations of assets.

VaR should not be used in isolation. It should always be used with a suite of risk management techniques, such as diversification, optimal portfolio allocation and prudent use of leverage.

Methods of Calculation

As of yet we have not discussed the actual calculation of VaR, either in the general case or a concrete trading example. There are three techniques that will be of interest to us. The first is the variance-covariance method (using normality assumptions), the second is a Monte Carlo method (based on an underlying, potentially non-normal, distribution) and the third is known as historical bootstrapping, which makes use of historical returns information for assets under consideration.

In this section we will concentrate on the Variance-Covariance Method.

Variance-Covariance Method

Consider a portfolio of P dollars, with a confidence level c . We are considering daily returns, with asset (or strategy) historical standard deviation σ and mean μ . Then the *daily* VaR, under the variance-covariance method for a single asset (or strategy) is calculated as:

$$P - (P(\alpha(1 - c) + 1)) \quad (13.5)$$

Where α is the inverse of the cumulative distribution function of a normal distribution with mean μ and standard deviation σ .

We can use the SciPy and pandas libraries in order to calculate these values. If we set $P = 10^6$ and $c = 0.99$, we can use the SciPy ppf method to generate the values for the inverse cumulative distribution function to a normal distribution with μ and σ obtained from some real financial data, in this case the historical daily returns of CitiGroup (we could easily substitute the returns of an algorithmic strategy in here):

```
#!/usr/bin/python
# -*- coding: utf-8 -*-

# var.py

from __future__ import print_function

import datetime

import numpy as np
import pandas.io.data as web
from scipy.stats import norm

def var_cov_var(P, c, mu, sigma):
    """
    Variance-Covariance calculation of daily Value-at-Risk
    using confidence level c, with mean of returns mu
    and standard deviation of returns sigma, on a portfolio
    of value P.
    """
    alpha = norm.ppf(1-c, mu, sigma)
    return P - P*(alpha + 1)
```

```

if __name__ == "__main__":
    start = datetime.datetime(2010, 1, 1)
    end = datetime.datetime(2014, 1, 1)

    citi = web.DataReader("C", 'yahoo', start, end)
    citi["rets"] = citi["Adj Close"].pct_change()

    P = 1e6    # 1,000,000 USD
    c = 0.99   # 99% confidence interval
    mu = np.mean(citi["rets"])
    sigma = np.std(citi["rets"])

    var = var_cov_var(P, c, mu, sigma)
    print("Value-at-Risk: $%0.2f" % var)

```

The calculated value of VaR is given by:

Value-at-Risk: \$56503.12

VaR is an extremely useful and pervasive technique in all areas of financial management, but it is not without its flaws. David Einhorn, the renowned hedge fund manager, has famously described VaR as "an airbag that works all the time, except when you have a car accident." Indeed, you should always use VaR as an augmentation to your risk management overlay, not as a single indicator!

Chapter 14

Event-Driven Trading Engine Implementation

This chapter provides an implementation for a fully self-contained event-driven backtest system written in Python. In particular this chapter has been written to expand on the details that are usually omitted from other algorithmic trading texts and papers. The following code will allow you to simulate high-frequency (minute to second) strategies across the forecasting, momentum and mean reversion domains in the equities, foreign exchange and futures markets.

With extensive detail comes complexity, however. The backtesting system provided here requires many components, each of which are comprehensive entities in themselves. The first step is thus to outline what event-driven software is and then describe the components of the backtester and how the entire system fits together.

14.1 Event-Driven Software

Before we delve into development of such a backtester we need to understand the concept of event-driven systems. Video games provide a natural use case for event-driven software and provide a straightforward example to explore. A video game has multiple components that interact with each other in a real-time setting at high framerates. This is handled by running the entire set of calculations within an "infinite" loop known as the event-loop or game-loop.

At each tick of the game-loop a function is called to receive the latest event, which will have been generated by some corresponding prior action within the game. Depending upon the nature of the event, which could include a key-press or a mouse click, some subsequent action is taken, which will either terminate the loop or generate some additional events. The process will then continue.

Here is some example pseudo-code:

```
while True: # Run the loop forever
    new_event = get_new_event() # Get the latest event

    # Based on the event type, perform an action
    if new_event.type == "LEFT_MOUSE_CLICK":
        open_menu()
    elif new_event.type == "ESCAPE_KEY_PRESS":
        quit_game()
    elif new_event.type == "UP_KEY_PRESS":
        move_player_north()
    # ... and many more events

    redraw_screen() # Update the screen to provide animation
    tick(50) # Wait 50 milliseconds
```


The code is continually checking for new events and then performing actions based on these events. In particular it allows the illusion of real-time response handling because the code is continually being looped and events checked for. As will become clear this is precisely what we need in order to carry out high frequency trading simulation.

14.1.1 Why An Event-Driven Backtester?

Event-driven systems provide many advantages over a vectorised approach:

- **Code Reuse** - An event-driven backtester, by design, can be used for both historical backtesting and live trading with minimal switch-out of components. This is not true of vectorised backtesters where all data must be available at once to carry out statistical analysis.
- **Lookahead Bias** - With an event-driven backtester there is no lookahead bias as market data receipt is treated as an "event" that must be acted upon. Thus it is possible to "drip feed" an event-driven backtester with market data, replicating how an order management and portfolio system would behave.
- **Realism** - Event-driven backtesters allow significant customisation over how orders are executed and transaction costs are incurred. It is straightforward to handle basic market and limit orders, as well as market-on-open (MOO) and market-on-close (MOC), since a custom exchange handler can be constructed.

Although event-driven systems come with many benefits they suffer from two major disadvantages over simpler vectorised systems. Firstly they are significantly more complex to implement and test. There are more "moving parts" leading to a greater chance of introducing bugs. To mitigate this proper software testing methodology such as test-driven development can be employed.

Secondly they are slower to execute compared to a vectorised system. Optimal vectorised operations are unable to be utilised when carrying out mathematical calculations.

14.2 Component Objects

To apply an event-driven approach to a backtesting system it is necessary to define our components (or objects) that will handle specific tasks:

- **Event** - The Event is the fundamental class unit of the event-driven system. It contains a type (such as "MARKET", "SIGNAL", "ORDER" or "FILL") that determines how it will be handled within the event-loop.
- **Event Queue** - The Event Queue is an in-memory Python Queue object that stores all of the Event sub-class objects that are generated by the rest of the software.
- **DataHandler** - The DataHandler is an abstract base class (ABC) that presents an interface for handling both historical or live market data. This provides significant flexibility as the Strategy and Portfolio modules can thus be reused between both approaches. The DataHandler generates a new MarketEvent upon every heartbeat of the system (see below).
- **Strategy** - The Strategy is also an ABC that presents an interface for taking market data and generating corresponding SignalEvents, which are ultimately utilised by the Portfolio object. A SignalEvent contains a ticker symbol, a direction (LONG or SHORT) and a timestamp.
- **Portfolio** - This is a class hierarchy which handles the order management associated with current and subsequent positions for a strategy. It also carries out risk management across the portfolio, including sector exposure and position sizing. In a more sophisticated implementation this could be delegated to a RiskManagement class. The Portfolio takes SignalEvents from the Queue and generates OrderEvents that get added to the Queue.

- **ExecutionHandler** - The ExecutionHandler simulates a connection to a brokerage. The job of the handler is to take OrderEvents from the Queue and execute them, either via a simulated approach or an actual connection to a live brokerage. Once orders are executed the handler creates FillEvents, which describe what was actually transacted, including fees, commission and slippage (if modelled).
- **Backtest** - All of these components are wrapped in an event-loop that correctly handles all Event types, routing them to the appropriate component.

Despite the quantity of components, this is quite a basic model of a trading engine. There is significant scope for expansion, particularly in regard to how the Portfolio is used. In addition differing transaction cost models might also be abstracted into their own class hierarchy.

14.2.1 Events

The first component to be discussed is the Event class hierarchy. In this infrastructure there are four types of events which allow communication between the above components via an event queue. They are a MarketEvent, SignalEvent, OrderEvent and FillEvent.

Event

The parent class in the hierarchy is called Event. It is a base class and does not provide any functionality or specific interface. Since in many implementations the Event objects will likely develop greater complexity it is thus being "future-proofed" by creating a class hierarchy.

```
#!/usr/bin/python
# -*- coding: utf-8 -*-

# event.py

from __future__ import print_function

class Event(object):
    """
    Event is base class providing an interface for all subsequent
    (inherited) events, that will trigger further events in the
    trading infrastructure.
    """
    pass
```

MarketEvent

MarketEvents are triggered when the outer while loop of the backtesting system begins a new "heartbeat". It occurs when the DataHandler object receives a new update of market data for any symbols which are currently being tracked. It is used to trigger the Strategy object generating new trading signals. The event object simply contains an identification that it is a market event, with no other structure.

```
# event.py

class MarketEvent(Event):
    """
    Handles the event of receiving a new market update with
    corresponding bars.
    """

    def __init__(self):
```

```

"""
    Initialises the MarketEvent.
"""
self.type = 'MARKET'

```

SignalEvent

The Strategy object utilises market data to create new SignalEvents. The SignalEvent contains a strategy ID, a ticker symbol, a timestamp for when it was generated, a direction (long or short) and a "strength" indicator (this is useful for mean reversion strategies). The SignalEvents are utilised by the Portfolio object as advice for how to trade.

```

# event.py

class SignalEvent(Event):
    """
        Handles the event of sending a Signal from a Strategy object.
        This is received by a Portfolio object and acted upon.
    """

    def __init__(self, strategy_id, symbol, datetime, signal_type, strength):
        """
            Initialises the SignalEvent.

            Parameters:
            strategy_id - The unique identifier for the strategy that
                          generated the signal.
            symbol - The ticker symbol, e.g. 'GOOG'.
            datetime - The timestamp at which the signal was generated.
            signal_type - 'LONG' or 'SHORT'.
            strength - An adjustment factor "suggestion" used to scale
                      quantity at the portfolio level. Useful for pairs strategies.
        """

        self.type = 'SIGNAL'
        self.strategy_id = strategy_id
        self.symbol = symbol
        self.datetime = datetime
        self.signal_type = signal_type
        self.strength = strength

```

OrderEvent

When a Portfolio object receives SignalEvents it assesses them in the wider context of the portfolio, in terms of risk and position sizing. This ultimately leads to OrderEvents that will be sent to an ExecutionHandler.

The OrderEvent is slightly more complex than a SignalEvent since it contains a quantity field in addition to the aforementioned properties of SignalEvent. The quantity is determined by the Portfolio constraints. In addition the OrderEvent has a `print_order()` method, used to output the information to the console if necessary.

```

# event.py

class OrderEvent(Event):
    """
        Handles the event of sending an Order to an execution system.
        The order contains a symbol (e.g. GOOG), a type (market or limit),

```

```

quantity and a direction.
"""

def __init__(self, symbol, order_type, quantity, direction):
    """
    Initialises the order type, setting whether it is
    a Market order ('MKT') or Limit order ('LMT'), has
    a quantity (integral) and its direction ('BUY' or
    'SELL').

    Parameters:
    symbol - The instrument to trade.
    order_type - 'MKT' or 'LMT' for Market or Limit.
    quantity - Non-negative integer for quantity.
    direction - 'BUY' or 'SELL' for long or short.
    """

    self.type = 'ORDER'
    self.symbol = symbol
    self.order_type = order_type
    self.quantity = quantity
    self.direction = direction

def print_order(self):
    """
    Outputs the values within the Order.
    """
    print(
        "Order: Symbol=%s, Type=%s, Quantity=%s, Direction=%s" %
        (self.symbol, self.order_type, self.quantity, self.direction)
    )

```

FillEvent

When an ExecutionHandler receives an OrderEvent it must transact the order. Once an order has been transacted it generates a FillEvent, which describes the cost of purchase or sale as well as the transaction costs, such as fees or slippage.

The FillEvent is the Event with the greatest complexity. It contains a timestamp for when an order was filled, the symbol of the order and the exchange it was executed on, the quantity of shares transacted, the actual price of the purchase and the commission incurred.

The commission is calculated using the Interactive Brokers commissions. For US API orders this commission is 1.30 USD minimum per order, with a flat rate of either 0.013 USD or 0.08 USD per share depending upon whether the trade size is below or above 500 units of stock.

```

# event.py

class FillEvent(Event):
    """
    Encapsulates the notion of a Filled Order, as returned
    from a brokerage. Stores the quantity of an instrument
    actually filled and at what price. In addition, stores
    the commission of the trade from the brokerage.
    """

    def __init__(self, timeindex, symbol, exchange, quantity,
                 direction, fill_cost, commission=None):

```

```

"""
Initialises the FillEvent object. Sets the symbol, exchange,
quantity, direction, cost of fill and an optional
commission.

If commission is not provided, the Fill object will
calculate it based on the trade size and Interactive
Brokers fees.

Parameters:
timeindex - The bar-resolution when the order was filled.
symbol - The instrument which was filled.
exchange - The exchange where the order was filled.
quantity - The filled quantity.
direction - The direction of fill ('BUY' or 'SELL')
fill_cost - The holdings value in dollars.
commission - An optional commission sent from IB.
"""

self.type = 'FILL'
self.timeindex = timeindex
self.symbol = symbol
self.exchange = exchange
self.quantity = quantity
self.direction = direction
self.fill_cost = fill_cost

# Calculate commission
if commission is None:
    self.commission = self.calculate_ib_commission()
else:
    self.commission = commission

def calculate_ib_commission(self):
    """
    Calculates the fees of trading based on an Interactive
    Brokers fee structure for API, in USD.

    This does not include exchange or ECN fees.

    Based on "US API Directed Orders":
    https://www.interactivebrokers.com/en/index.php?
    f=commission&p=stocks2
    """
    full_cost = 1.3
    if self.quantity <= 500:
        full_cost = max(1.3, 0.013 * self.quantity)
    else: # Greater than 500
        full_cost = max(1.3, 0.008 * self.quantity)
    return full_cost

```

14.2.2 Data Handler

One of the goals of an event-driven trading system is to minimise duplication of code between the backtesting element and the live execution element. Ideally it would be optimal to utilise the same signal generation methodology and portfolio management components for both historical

testing and live trading. In order for this to work the Strategy object which generates the Signals, and the Portfolio object which provides Orders based on them, must utilise an identical interface to a market feed for both historic and live running.

This motivates the concept of a class hierarchy based on a DataHandler object, which gives all subclasses an interface for providing market data to the remaining components within the system. In this way any subclass data handler can be "swapped out", without affecting strategy or portfolio calculation.

Specific example subclasses could include HistoricCSVDataHandler, QuandlDataHandler, SecuritiesMasterDataHandler, InteractiveBrokersMarketFeedDataHandler etc. In this chapter we are only going to consider the creation of a historic CSV data handler, which will load intraday CSV data for equities in an Open-Low-High-Close-Volume-OpenInterest set of bars. This can then be used to "drip feed" on a bar-by-bar basis the data into the Strategy and Portfolio classes on every heartbeat of the system, thus avoiding lookahead bias.

The first task is to import the necessary libraries. Specifically it will be necessary to import pandas and the abstract base class tools. Since the DataHandler generates MarketEvents, event.py is also needed as described above.

```
#!/usr/bin/python
# -*- coding: utf-8 -*-

# data.py

from __future__ import print_function

from abc import ABCMeta, abstractmethod
import datetime
import os, os.path

import numpy as np
import pandas as pd

from event import MarketEvent
```

The DataHandler is an abstract base class (ABC), which means that it is impossible to instantiate an instance directly. Only subclasses may be instantiated. The rationale for this is that the ABC provides an interface that all subsequent DataHandler subclasses must adhere to thereby ensuring compatibility with other classes that communicate with them.

We make use of the `__metaclass__` property to let Python know that this is an ABC. In addition we use the `@abstractmethod` decorator to let Python know that the method will be overridden in subclasses (this is identical to a *pure virtual method* in C++).

There are six methods listed for the class. The first two methods, `get_latest_bar` and `get_latest_bars`, are used to retrieve a recent subset of the historical trading bars from a stored list of such bars. These methods come in handy within the Strategy and Portfolio classes, due to the need to constantly be aware of current market prices and volumes.

The following method, `get_latest_bar_datetime`, simply returns a Python datetime object that represents the timestamp of the bar (e.g. a date for daily bars or a minute-resolution object for minutely bars).

The following two methods, `get_latest_bar_value` and `get_latest_bar_values`, are convenience methods used to retrieve individual values from a particular bar, or list of bars. For instance it is often the case that a strategy is only interested in closing prices. In this instance we can use these methods to return a list of floating point values representing the closing prices of previous bars, rather than having to obtain it from the list of bar objects. This generally increases efficiency of strategies that utilise a "lookback window", such as those involving regressions.

The final method, `update_bars`, provides a "drip feed" mechanism for placing bar information on a new data structure that strictly prohibits lookahead bias. This is one of the key differences between an event-driven backtesting system and one based on vectorisation. Notice that exceptions will be raised if an attempted instantiation of the class occurs:

```
# data.py

class DataHandler(object):
    """
    DataHandler is an abstract base class providing an interface for
    all subsequent (inherited) data handlers (both live and historic).

    The goal of a (derived) DataHandler object is to output a generated
    set of bars (OHLCVI) for each symbol requested.

    This will replicate how a live strategy would function as current
    market data would be sent "down the pipe". Thus a historic and live
    system will be treated identically by the rest of the backtesting suite.
    """

    __metaclass__ = ABCMeta

    @abstractmethod
    def get_latest_bar(self, symbol):
        """
        Returns the last bar updated.
        """
        raise NotImplementedError("Should implement get_latest_bar()")

    @abstractmethod
    def get_latest_bars(self, symbol, N=1):
        """
        Returns the last N bars updated.
        """
        raise NotImplementedError("Should implement get_latest_bars()")

    @abstractmethod
    def get_latest_bar_datetime(self, symbol):
        """
        Returns a Python datetime object for the last bar.
        """
        raise NotImplementedError("Should implement
        get_latest_bar_datetime()")

    @abstractmethod
    def get_latest_bar_value(self, symbol, val_type):
        """
        Returns one of the Open, High, Low, Close, Volume or OI
        from the last bar.
        """
        raise NotImplementedError("Should implement
        get_latest_bar_value()")

    @abstractmethod
    def get_latest_bars_values(self, symbol, val_type, N=1):
        """
        Returns the last N bar values from the
        latest_symbol list, or N-k if less available.
        """
        raise NotImplementedError("Should implement
        get_latest_bars_values()")
```

```

@abstractmethod
def update_bars(self):
    """
    Pushes the latest bars to the bars_queue for each symbol
    in a tuple OHLCVI format: (datetime, open, high, low,
    close, volume, open interest).
    """
    raise NotImplementedError("Should implement update_bars()")

```

In order to create a backtesting system based on historical data we need to consider a mechanism for importing data via common sources. We've discussed the benefits of a Securities Master Database in previous chapters. Thus a good candidate for making a DataHandler class would be to couple it with such a database.

However, for clarity in this chapter, I want to discuss a simpler mechanism, that of importing (potentially large) comma-separated variable (CSV) files. This will allow us to focus on the mechanics of creating the DataHandler, rather than be concerned with the "boilerplate" code of connecting to a database and using SQL queries to grab data.

Thus we are going to define the HistoricCSVDDataHandler subclass, which is designed to process multiple CSV files, one for each traded symbol, and convert these into a dictionary of pandas DataFrames that can be accessed by the previously mentioned bar methods.

The data handler requires a few parameters, namely an Event Queue on which to push MarketEvent information to, the absolute path of the CSV files and a list of symbols. Here is the initialisation of the class:

```

# data.py

class HistoricCSVDDataHandler(DataHandler):
    """
    HistoricCSVDDataHandler is designed to read CSV files for
    each requested symbol from disk and provide an interface
    to obtain the "latest" bar in a manner identical to a live
    trading interface.
    """

    def __init__(self, events, csv_dir, symbol_list):
        """
        Initialises the historic data handler by requesting
        the location of the CSV files and a list of symbols.

        It will be assumed that all files are of the form
        'symbol.csv', where symbol is a string in the list.

        Parameters:
        events - The Event Queue.
        csv_dir - Absolute directory path to the CSV files.
        symbol_list - A list of symbol strings.
        """
        self.events = events
        self.csv_dir = csv_dir
        self.symbol_list = symbol_list

        self.symbol_data = {}
        self.latest_symbol_data = {}
        self.continue_backtest = True

        self._open_convert_csv_files()

```


The handler will look for files in the absolute directory `csv_dir` and try to open them with the format of "SYMBOL.csv", where SYMBOL is the ticker symbol (such as GOOG or AAPL). The format of the files matches that provided by Yahoo Finance, but is easily modified to handle additional data formats, such as those provided by Quandl or DTN IQFeed. The opening of the files is handled by the `_open_convert_csv_files` method below.

One of the benefits of using pandas as a datastore internally within the `HistoricCSVDataHandler` is that the indexes of all symbols being tracked can be merged together. This allows missing data points to be padded forward, backward or interpolated within these gaps such that tickers can be compared on a bar-to-bar basis. This is necessary for mean-reverting strategies, for instance. Notice the use of the union and reindex methods when combining the indexes for all symbols:

```
# data.py

def _open_convert_csv_files(self):
    """
    Opens the CSV files from the data directory, converting
    them into pandas DataFrames within a symbol dictionary.

    For this handler it will be assumed that the data is
    taken from Yahoo. Thus its format will be respected.
    """
    comb_index = None
    for s in self.symbol_list:
        # Load the CSV file with no header information, indexed on date
        self.symbol_data[s] = pd.io.parsers.read_csv(
            os.path.join(self.csv_dir, '%s.csv' % s),
            header=0, index_col=0, parse_dates=True,
            names=[
                'datetime', 'open', 'high',
                'low', 'close', 'volume', 'adj_close'
            ]
        ).sort()

        # Combine the index to pad forward values
        if comb_index is None:
            comb_index = self.symbol_data[s].index
        else:
            comb_index.union(self.symbol_data[s].index)

        # Set the latest symbol_data to None
        self.latest_symbol_data[s] = []

    # Reindex the dataframes
    for s in self.symbol_list:
        self.symbol_data[s] = self.symbol_data[s].\
            reindex(index=comb_index, method='pad').iterrows()
```

The `_get_new_bar` method creates a generator to provide a new bar. This means that subsequent calls to the method will *yield* a new bar until the end of the symbol data is reached:

```
# data.py

def _get_new_bar(self, symbol):
    """
    Returns the latest bar from the data feed.
    """
    for b in self.symbol_data[symbol]:
```

```
yield b
```

The first abstract methods from `DataHandler` to be implemented are `get_latest_bar` and `get_latest_bars`. These methods simply provide either a bar or list of the last N bars from the `latest_symbol_data` structure:

```
# data.py
```

```
def get_latest_bar(self, symbol):
    """
    Returns the last bar from the latest_symbol list.
    """
    try:
        bars_list = self.latest_symbol_data[symbol]
    except KeyError:
        print("That symbol is not available in the historical data set.")
        raise
    else:
        return bars_list[-1]

def get_latest_bars(self, symbol, N=1):
    """
    Returns the last N bars from the latest_symbol list,
    or N-k if less available.
    """
    try:
        bars_list = self.latest_symbol_data[symbol]
    except KeyError:
        print("That symbol is not available in the historical data set.")
        raise
    else:
        return bars_list[-N:]
```

The next method, `get_latest_bar_datetime`, queries the latest bar for a datetime object representing the "last market price":

```
def get_latest_bar_datetime(self, symbol):
    """
    Returns a Python datetime object for the last bar.
    """
    try:
        bars_list = self.latest_symbol_data[symbol]
    except KeyError:
        print("That symbol is not available in the historical data set.")
        raise
    else:
        return bars_list[-1][0]
```

The next two methods being implemented are `get_latest_bar_value` and `get_latest_bar_values`. Both methods make use of the Python `getattr` function, which queries an object to see if a particular attribute exists on an object. Thus we can pass a string such as "open" or "close" to `getattr` and obtain the value direct from the bar, thus making the method more flexible. This stops us having to write methods of the type `get_latest_bar_close`, for instance:

```
def get_latest_bar_value(self, symbol, val_type):
    """
    Returns one of the Open, High, Low, Close, Volume or OI
    values from the pandas Bar series object.
    """
```

```

    try:
        bars_list = self.latest_symbol_data[symbol]
    except KeyError:
        print("That symbol is not available in the historical data set.")
        raise
    else:
        return getattr(bars_list[-1][1], val_type)

    def get_latest_bars_values(self, symbol, val_type, N=1):
        """
        Returns the last N bar values from the
        latest_symbol list, or N-k if less available.
        """
        try:
            bars_list = self.get_latest_bars(symbol, N)
        except KeyError:
            print("That symbol is not available in the historical data set.")
            raise
        else:
            return np.array([getattr(b[1], val_type) for b in bars_list])

```

The final method, `update_bars`, is the second abstract method from `DataHandler`. It simply generates a `MarketEvent` that gets added to the queue as it appends the latest bars to the `latest_symbol_data` dictionary:

data.py

```

def update_bars(self):
    """
    Pushes the latest bar to the latest_symbol_data structure
    for all symbols in the symbol list.
    """
    for s in self.symbol_list:
        try:
            bar = next(self._get_new_bar(s))
        except StopIteration:
            self.continue_backtest = False
        else:
            if bar is not None:
                self.latest_symbol_data[s].append(bar)
    self.events.put(MarketEvent())

```

Thus we have a `DataHandler`-derived object, which is used by the remaining components to keep track of market data. The `Strategy`, `Portfolio` and `ExecutionHandler` objects all require the current market data thus it makes sense to centralise it to avoid duplication of storage between these classes.

14.2.3 Strategy

A `Strategy` object encapsulates all calculation on market data that generate *advisory* signals to a `Portfolio` object. Thus all of the "strategy logic" resides within this class. I have opted to separate out the `Strategy` and `Portfolio` objects for this backtester, since I believe this is more amenable to the situation of multiple strategies feeding "ideas" to a larger `Portfolio`, which then can handle its own risk (such as sector allocation, leverage). In higher frequency trading, the strategy and portfolio concepts will be tightly coupled and extremely hardware dependent. This is well beyond the scope of this chapter, however!

At this stage in the event-driven backtester development there is no concept of an *indicator* or *filter*, such as those found in technical trading. These are also good candidates for creating

a class hierarchy but are beyond the scope of this chapter. Thus such mechanisms will be used directly in derived Strategy objects.

The strategy hierarchy is relatively simple as it consists of an abstract base class with a single pure virtual method for generating `SignalEvent` objects. In order to create the `Strategy` hierarchy it is necessary to import NumPy, pandas, the `Queue` object (which has become `queue` in Python 3), abstract base class tools and the `SignalEvent`:

```
#!/usr/bin/python
# -*- coding: utf-8 -*-

# strategy.py

from __future__ import print_function

from abc import ABCMeta, abstractmethod
import datetime
try:
    import Queue as queue
except ImportError:
    import queue

import numpy as np
import pandas as pd

from event import SignalEvent
```

The `Strategy` abstract base class simply defines a pure virtual `calculate_signals` method. In derived classes this is used to handle the generation of `SignalEvent` objects based on market data updates:

```
# strategy.py

class Strategy(object):
    """
    Strategy is an abstract base class providing an interface for
    all subsequent (inherited) strategy handling objects.

    The goal of a (derived) Strategy object is to generate Signal
    objects for particular symbols based on the inputs of Bars
    (OHLCV) generated by a DataHandler object.

    This is designed to work both with historic and live data as
    the Strategy object is agnostic to where the data came from,
    since it obtains the bar tuples from a queue object.
    """

    __metaclass__ = ABCMeta

    @abstractmethod
    def calculate_signals(self):
        """
        Provides the mechanisms to calculate the list of signals.
        """
        raise NotImplementedError("Should implement calculate_signals()")
```

14.2.4 Portfolio

This section describes a **Portfolio** object that keeps track of the positions within a portfolio and generates orders of a fixed quantity of stock based on signals. More sophisticated portfolio objects could include risk management and position sizing tools (such as the Kelly Criterion). In fact, in the following chapters we will add such tools to some of our trading strategies to see how they compare to a more "naive" portfolio approach.

The portfolio order management system is possibly the most complex component of an event-driven backtester. Its role is to keep track of all current market positions as well as the market value of the positions (known as the "holdings"). This is simply an estimate of the liquidation value of the position and is derived in part from the data handling facility of the backtester.

In addition to the positions and holdings management the portfolio must also be aware of risk factors and position sizing techniques in order to optimise orders that are sent to a brokerage or other form of market access.

Unfortunately, Portfolio and Order Management Systems (OMS) can become rather complex! Thus I've made a decision here to keep the Portfolio object relatively straightforward, so that you can understand the key ideas and how they are implemented. The nature of an object-oriented design is that it allows, in a natural way, the extension to more complex situations later on.

Continuing in the vein of the **Event** class hierarchy a **Portfolio** object must be able to handle **SignalEvent** objects, generate **OrderEvent** objects and interpret **FillEvent** objects to update positions. Thus it is no surprise that the **Portfolio** objects are often the largest component of event-driven systems, in terms of lines of code (LOC).

We create a new file **portfolio.py** and import the necessary libraries. These are the same as most of the other class implementations, with the exception that Portfolio is NOT going to be an abstract base class. Instead it will be normal base class. This means that it can be instantiated and thus is useful as a "first go" Portfolio object when testing out new strategies. Other Portfolios can be derived from it and override sections to add more complexity.

For completeness, here is the performance.py file:

```
#!/usr/bin/python
# -*- coding: utf-8 -*-

# performance.py

from __future__ import print_function

import numpy as np
import pandas as pd

def create_sharpe_ratio(returns, periods=252):
    """
    Create the Sharpe ratio for the strategy, based on a
    benchmark of zero (i.e. no risk-free rate information).

    Parameters:
    returns - A pandas Series representing period percentage returns.
    periods - Daily (252), Hourly (252*6.5), Minutely(252*6.5*60) etc.
    """
    return np.sqrt(periods) * (np.mean(returns)) / np.std(returns)

def create_drawdowns(pnl):
    """
    Calculate the largest peak-to-trough drawdown of the PnL curve
    as well as the duration of the drawdown. Requires that the
    pnl_returns is a pandas Series.
```

```

Parameters:
pnl - A pandas Series representing period percentage returns.

Returns:
drawdown, duration - Highest peak-to-trough drawdown and duration.
"""

# Calculate the cumulative returns curve
# and set up the High Water Mark
hwm = [0]

# Create the drawdown and duration series
idx = pnl.index
drawdown = pd.Series(index = idx)
duration = pd.Series(index = idx)

# Loop over the index range
for t in range(1, len(idx)):
    hwm.append(max(hwm[t-1], pnl[t]))
    drawdown[t] = (hwm[t] - pnl[t])
    duration[t] = (0 if drawdown[t] == 0 else duration[t-1] + 1)
return drawdown, drawdown.max(), duration.max()

```

Here is the import listing for the Portfolio.py file. We need to import the `floor` function from the `math` library in order to generate integer-valued order sizes. We also need the `FillEvent` and `OrderEvent` objects since the `Portfolio` handles both. Notice also that we are adding two additional functions, `create_sharpe_ratio` and `create_drawdowns`, both from the `performance.py` file described above.

```

#!/usr/bin/python
# -*- coding: utf-8 -*-

# portfolio.py

from __future__ import print_function

import datetime
from math import floor
try:
    import Queue as queue
except ImportError:
    import queue

import numpy as np
import pandas as pd

from event import FillEvent, OrderEvent
from performance import create_sharpe_ratio, create_drawdowns

```

The initialisation of the `Portfolio` object requires access to the bars `DataHandler`, the events `Event Queue`, a start datetime stamp and an initial capital value (defaulting to 100,000 USD).

The `Portfolio` is designed to handle position sizing and current holdings, but will carry out trading orders in a "dumb" manner by simply sending them directly to the brokerage with a predetermined fixed quantity size, irrespective of cash held. These are all unrealistic assumptions, but they help to outline how a portfolio order management system (OMS) functions in an event-driven fashion.

The portfolio contains the `all_positions` and `current_positions` members. The former stores a list of all previous *positions* recorded at the timestamp of a market data event. A position is simply the quantity of the asset held. Negative positions mean the asset has been shorted. The latter `current_positions` dictionary stores the current positions for the last market bar update, for each symbol.

In addition to the positions data the portfolio stores *holdings*, which describe the current market *value* of the positions held. "Current market value" in this instance means the closing price obtained from the current market bar, which is clearly an approximation, but is reasonable enough for the time being. `all_holdings` stores the historical list of all symbol holdings, while `current_holdings` stores the most up to date dictionary of all symbol holdings values:

```
# portfolio.py

class Portfolio(object):
    """
    The Portfolio class handles the positions and market
    value of all instruments at a resolution of a "bar",
    i.e. secondly, minutely, 5-min, 30-min, 60 min or EOD.

    The positions DataFrame stores a time-index of the
    quantity of positions held.

    The holdings DataFrame stores the cash and total market
    holdings value of each symbol for a particular
    time-index, as well as the percentage change in
    portfolio total across bars.
    """

    def __init__(self, bars, events, start_date, initial_capital=100000.0):
        """
        Initialises the portfolio with bars and an event queue.
        Also includes a starting datetime index and initial capital
        (USD unless otherwise stated).

        Parameters:
        bars - The DataHandler object with current market data.
        events - The Event Queue object.
        start_date - The start date (bar) of the portfolio.
        initial_capital - The starting capital in USD.
        """
        self.bars = bars
        self.events = events
        self.symbol_list = self.bars.symbol_list
        self.start_date = start_date
        self.initial_capital = initial_capital

        self.all_positions = self.construct_all_positions()
        self.current_positions = dict( (k,v) for k, v in \
            [(s, 0) for s in self.symbol_list] )

        self.all_holdings = self.construct_all_holdings()
        self.current_holdings = self.construct_current_holdings()
```

The following method, `construct_all_positions`, simply creates a dictionary for each symbol, sets the value to zero for each and then adds a datetime key, finally adding it to a list. It uses a dictionary comprehension, which is similar in spirit to a list comprehension:

```
# portfolio.py
```

```
def construct_all_positions(self):
    """
    Constructs the positions list using the start_date
    to determine when the time index will begin.
    """
    d = dict( (k,v) for k, v in [(s, 0) for s in self.symbol_list] )
    d['datetime'] = self.start_date
    return [d]
```

The `construct_all_holdings` method is similar to the above but adds extra keys for cash, commission and total, which respectively represent the spare cash in the account after any purchases, the cumulative commission accrued and the total account equity including cash and any open positions. Short positions are treated as negative. The starting cash and total account equity are both set to the initial capital.

In this manner there are separate "accounts" for each symbol, the "cash on hand", the "commission" paid (Interactive Broker fees) and a "total" portfolio value. Clearly this does not take into account margin requirements or shorting constraints, but is sufficient to give you a flavour of how such an OMS is created:

portfolio.py

```
def construct_all_holdings(self):
    """
    Constructs the holdings list using the start_date
    to determine when the time index will begin.
    """
    d = dict( (k,v) for k, v in [(s, 0.0) for s in self.symbol_list] )
    d['datetime'] = self.start_date
    d['cash'] = self.initial_capital
    d['commission'] = 0.0
    d['total'] = self.initial_capital
    return [d]
```

The following method, `construct_current_holdings` is almost identical to the method above except that it doesn't wrap the dictionary in a list, because it is only creating a single entry:

portfolio.py

```
def construct_current_holdings(self):
    """
    This constructs the dictionary which will hold the instantaneous
    value of the portfolio across all symbols.
    """
    d = dict( (k,v) for k, v in [(s, 0.0) for s in self.symbol_list] )
    d['cash'] = self.initial_capital
    d['commission'] = 0.0
    d['total'] = self.initial_capital
    return d
```

On every *heartbeat*, that is every time new market data is requested from the `DataHandler` object, the portfolio must update the current market value of all the positions held. In a live trading scenario this information can be downloaded and parsed directly from the brokerage, but for a backtesting implementation it is necessary to calculate these values manually from the bars `DataHandler`.

Unfortunately there is no such as thing as the "current market value" due to bid/ask spreads and liquidity issues. Thus it is necessary to estimate it by multiplying the quantity of the asset held by a particular approximate "price". The approach I have taken here is to use the closing

price of the last bar received. For an intraday strategy this is relatively realistic. For a daily strategy this is less realistic as the opening price can differ substantially from the closing price.

The method `update_timeindex` handles the new holdings tracking. It firstly obtains the latest prices from the market data handler and creates a new dictionary of symbols to represent the current positions, by setting the "new" positions equal to the "current" positions.

The current positions are only modified when a `FillEvent` is obtained, which is handled later on in the portfolio code. The method then appends this set of current positions to the `all_positions` list.

The holdings are then updated in a similar manner, with the exception that the market value is recalculated by multiplying the current positions count with the closing price of the latest bar. Finally the new holdings are appended to `all_holdings`:

```
# portfolio.py

def update_timeindex(self, event):
    """
    Adds a new record to the positions matrix for the current
    market data bar. This reflects the PREVIOUS bar, i.e. all
    current market data at this stage is known (OHLCV).

    Makes use of a MarketEvent from the events queue.
    """
    latest_datetime = self.bars.get_latest_bar_datetime(
        self.symbol_list[0]
    )

    # Update positions
    # =====
    dp = dict( (k,v) for k, v in [(s, 0) for s in self.symbol_list] )
    dp['datetime'] = latest_datetime

    for s in self.symbol_list:
        dp[s] = self.current_positions[s]

    # Append the current positions
    self.all_positions.append(dp)

    # Update holdings
    # =====
    dh = dict( (k,v) for k, v in [(s, 0) for s in self.symbol_list] )
    dh['datetime'] = latest_datetime
    dh['cash'] = self.current_holdings['cash']
    dh['commission'] = self.current_holdings['commission']
    dh['total'] = self.current_holdings['cash']

    for s in self.symbol_list:
        # Approximation to the real value
        market_value = self.current_positions[s] * \
            self.bars.get_latest_bar_value(s, "adj_close")
        dh[s] = market_value
        dh['total'] += market_value

    # Append the current holdings
    self.all_holdings.append(dh)
```

The method `update_positions_from_fill` determines whether a `FillEvent` is a Buy or a Sell and then updates the `current_positions` dictionary accordingly by adding/subtracting

the correct quantity of shares:

portfolio.py

```
def update_positions_from_fill(self, fill):
    """
    Takes a Fill object and updates the position matrix to
    reflect the new position.

    Parameters:
    fill - The Fill object to update the positions with.
    """
    # Check whether the fill is a buy or sell
    fill_dir = 0
    if fill.direction == 'BUY':
        fill_dir = 1
    if fill.direction == 'SELL':
        fill_dir = -1

    # Update positions list with new quantities
    self.current_positions[fill.symbol] += fill_dir*fill.quantity
```

The corresponding `update_holdings_from_fill` is similar to the above method but updates the *holdings* values instead. In order to simulate the cost of a fill, the following method does not use the cost associated from the `FillEvent`. Why is this? Simply put, in a backtesting environment the fill cost is actually unknown (the *market impact* and the *depth of book* are unknown) and thus is must be estimated.

Thus the fill cost is set to the the "current market price", which is the closing price of the last bar. The holdings for a particular symbol are then set to be equal to the fill cost multiplied by the transacted quantity. For most lower frequency trading strategies in liquid markets this is a reasonable approximation, but at high frequency these issues will need to be considered in a production backtest and live trading engine.

Once the fill cost is known the current holdings, cash and total values can all be updated. The cumulative commission is also updated:

portfolio.py

```
def update_holdings_from_fill(self, fill):
    """
    Takes a Fill object and updates the holdings matrix to
    reflect the holdings value.

    Parameters:
    fill - The Fill object to update the holdings with.
    """
    # Check whether the fill is a buy or sell
    fill_dir = 0
    if fill.direction == 'BUY':
        fill_dir = 1
    if fill.direction == 'SELL':
        fill_dir = -1

    # Update holdings list with new quantities
    fill_cost = self.bars.get_latest_bar_value(fill.symbol, "adj_close")
    cost = fill_dir * fill_cost * fill.quantity
    self.current_holdings[fill.symbol] += cost
    self.current_holdings['commission'] += fill.commission
    self.current_holdings['cash'] -= (cost + fill.commission)
```

```
self.current_holdings['total'] -= (cost + fill.commission)
```

The pure virtual `update_fill` method from the `Portfolio` class is implemented here. It simply executes the two preceding methods, `update_positions_from_fill` and `update_holdings_from_fill`, upon receipt of a fill event:

```
# portfolio.py
```

```
def update_fill(self, event):
    """
    Updates the portfolio current positions and holdings
    from a FillEvent.
    """
    if event.type == 'FILL':
        self.update_positions_from_fill(event)
        self.update_holdings_from_fill(event)
```

While the `Portfolio` object must handle `FillEvents`, it must also take care of generating `OrderEvents` upon the receipt of one or more `SignalEvents`.

The `generate_naive_order` method simply takes a signal to go long or short an asset, sending an order to do so for 100 shares of such an asset. Clearly 100 is an arbitrary value, and will clearly depend upon the portfolio total equity in a production simulation.

In a realistic implementation this value will be determined by a risk management or position sizing overlay. However, this is a simplistic `Portfolio` and so it "naively" sends all orders directly from the signals, without a risk system.

The method handles longing, shorting and exiting of a position, based on the current quantity and particular symbol. Corresponding `OrderEvent` objects are then generated:

```
# portfolio.py
```

```
def generate_naive_order(self, signal):
    """
    Simply files an Order object as a constant quantity
    sizing of the signal object, without risk management or
    position sizing considerations.

    Parameters:
    signal - The tuple containing Signal information.
    """
    order = None

    symbol = signal.symbol
    direction = signal.signal_type
    strength = signal.strength

    mkt_quantity = 100
    cur_quantity = self.current_positions[symbol]
    order_type = 'MKT'

    if direction == 'LONG' and cur_quantity == 0:
        order = OrderEvent(symbol, order_type, mkt_quantity, 'BUY')
    if direction == 'SHORT' and cur_quantity == 0:
        order = OrderEvent(symbol, order_type, mkt_quantity, 'SELL')

    if direction == 'EXIT' and cur_quantity > 0:
        order = OrderEvent(symbol, order_type, abs(cur_quantity), 'SELL')
    if direction == 'EXIT' and cur_quantity < 0:
        order = OrderEvent(symbol, order_type, abs(cur_quantity), 'BUY')
```

```
return order
```

The `update_signal` method simply calls the above method and adds the generated order to the events queue:

```
# portfolio.py
```

```
def update_signal(self, event):
    """
    Acts on a SignalEvent to generate new orders
    based on the portfolio logic.
    """
    if event.type == 'SIGNAL':
        order_event = self.generate_naive_order(event)
        self.events.put(order_event)
```

The penultimate method in the `Portfolio` is the generation of an equity curve. This simply creates a returns stream, useful for performance calculations, and then normalises the equity curve to be percentage based. Thus the account initial size is equal to 1.0, as opposed to the absolute dollar amount:

```
# portfolio.py
```

```
def create_equity_curve_dataframe(self):
    """
    Creates a pandas DataFrame from the all_holdings
    list of dictionaries.
    """
    curve = pd.DataFrame(self.all_holdings)
    curve.set_index('datetime', inplace=True)
    curve['returns'] = curve['total'].pct_change()
    curve['equity_curve'] = (1.0+curve['returns']).cumprod()
    self.equity_curve = curve
```

The final method in the `Portfolio` is the output of the equity curve and various performance statistics related to the strategy. The final line outputs a file, `equity.csv`, to the same directory as the code, which can be loaded into a Matplotlib Python script (or a spreadsheet such as MS Excel or LibreOffice Calc) for subsequent analysis.

Note that the Drawdown Duration is given in terms of the absolute number of "bars" that the drawdown carried on for, as opposed to a particular timeframe.

```
def output_summary_stats(self):
    """
    Creates a list of summary statistics for the portfolio.
    """
    total_return = self.equity_curve['equity_curve'][-1]
    returns = self.equity_curve['returns']
    pnl = self.equity_curve['equity_curve']

    sharpe_ratio = create_sharpe_ratio(returns, periods=252*60*6.5)
    drawdown, max_dd, dd_duration = create_drawdowns(pnl)
    self.equity_curve['drawdown'] = drawdown

    stats = [("Total Return", "%0.2f%%" % \
                ((total_return - 1.0) * 100.0)),
              ("Sharpe Ratio", "%0.2f" % sharpe_ratio),
              ("Max Drawdown", "%0.2f%%" % (max_dd * 100.0)),
              ("Drawdown Duration", "%d" % dd_duration)]
```

```
self.equity_curve.to_csv('equity.csv')
return stats
```

The **Portfolio** object is the most complex aspect of the entire event-driven backtest system. The implementation here, while intricate, is relatively elementary in its handling of positions.

14.2.5 Execution Handler

In this section we will study the execution of trade orders by creating a class hierarchy that will represent a simulated order handling mechanism and ultimately tie into a brokerage or other means of market connectivity.

The **ExecutionHandler** described here is exceedingly simple, since it fills all orders at the current market price. This is highly unrealistic, but serves as a good baseline for improvement.

As with the previous abstract base class hierarchies, we must import the necessary properties and decorators from the **abc** library. In addition we need to import the **FillEvent** and **OrderEvent**:

```
#!/usr/bin/python
# -*- coding: utf-8 -*-

# execution.py

from __future__ import print_function

from abc import ABCMeta, abstractmethod
import datetime
try:
    import Queue as queue
except ImportError:
    import queue

from event import FillEvent, OrderEvent
```

The **ExecutionHandler** is similar to previous abstract base classes and simply has one pure virtual method, `execute_order`:

```
# execution.py

class ExecutionHandler(object):
    """
    The ExecutionHandler abstract class handles the interaction
    between a set of order objects generated by a Portfolio and
    the ultimate set of Fill objects that actually occur in the
    market.

    The handlers can be used to subclass simulated brokerages
    or live brokerages, with identical interfaces. This allows
    strategies to be backtested in a very similar manner to the
    live trading engine.
    """

    __metaclass__ = ABCMeta

    @abstractmethod
    def execute_order(self, event):
        """
        Takes an Order event and executes it, producing
        a Fill event that gets placed onto the Events queue.
        """
```

```

Parameters:
event - Contains an Event object with order information.
"""
raise NotImplementedError("Should implement execute_order()")

```

In order to backtest strategies we need to simulate how a trade will be transacted. The simplest possible implementation is to assume all orders are filled at the current market price for all quantities. This is clearly extremely unrealistic and a big part of improving backtest realism will come from designing more sophisticated models of slippage and market impact.

Note that the `FillEvent` is given a value of `None` for the `fill_cost` (see the penultimate line in `execute_order`) as we have already taken care of the cost of fill in the `Portfolio` object described above. In a more realistic implementation we would make use of the "current" market data value to obtain a realistic fill cost.

I have simply utilised ARCA as the exchange although for backtesting purposes this is purely a string placeholder. In a live execution environment this venue dependence would be far more important:

```

# execution.py

class SimulatedExecutionHandler(ExecutionHandler):
    """
    The simulated execution handler simply converts all order
    objects into their equivalent fill objects automatically
    without latency, slippage or fill-ratio issues.

    This allows a straightforward "first go" test of any strategy,
    before implementation with a more sophisticated execution
    handler.
    """

    def __init__(self, events):
        """
        Initialises the handler, setting the event queues
        up internally.

        Parameters:
        events - The Queue of Event objects.
        """
        self.events = events

    def execute_order(self, event):
        """
        Simply converts Order objects into Fill objects naively,
        i.e. without any latency, slippage or fill ratio problems.

        Parameters:
        event - Contains an Event object with order information.
        """
        if event.type == 'ORDER':
            fill_event = FillEvent(
                datetime.datetime.utcnow(), event.symbol,
                'ARCA', event.quantity, event.direction, None
            )
            self.events.put(fill_event)

```

14.2.6 Backtest

We are now in a position to create the Backtest class hierarchy. The Backtest object encapsulates the event-handling logic and essentially ties together all of the other classes that we have discussed above.

The Backtest object is designed to carry out a nested while-loop event-driven system in order to handle the events placed on the Event Queue object. The outer while-loop is known as the "heartbeat loop" and decides the temporal resolution of the backtesting system. In a live environment this value will be a positive number, such as 600 seconds (every ten minutes). Thus the market data and positions will only be updated on this timeframe.

For the backtester described here the "heartbeat" can be set to zero, irrespective of the strategy frequency, since the data is already available by virtue of the fact it is historical!

We can run the backtest at whatever speed we like, since the event-driven system is agnostic to *when* the data became available, so long as it has an associated timestamp. Hence I've only included it to demonstrate how a live trading engine would function. The outer loop thus ends once the DataHandler lets the Backtest object know, by using a boolean `continue_backtest` attribute.

The inner while-loop actually processes the signals and sends them to the correct component depending upon the event type. Thus the Event Queue is continually being populated and depopulated with events. This is what it means for a system to be *event-driven*.

The first task is to import the necessary libraries. We import `pprint` ("pretty-print"), because we want to display the stats in an output-friendly manner:

```
#!/usr/bin/python
# -*- coding: utf-8 -*-

# backtest.py

from __future__ import print_function

import datetime
import pprint
try:
    import Queue as queue
except ImportError:
    import queue
import time
```

The initialisation of the Backtest object requires the CSV directory, the full symbol list of traded symbols, the initial capital, the heartbeat time in milliseconds, the start datetime stamp of the backtest as well as the DataHandler, ExecutionHandler, Portfolio and Strategy objects. A Queue is used to hold the events. The signals, orders and fills are counted:

```
# backtest.py

class Backtest(object):
    """
    Encapsulates the settings and components for carrying out
    an event-driven backtest.
    """

    def __init__(
        self, csv_dir, symbol_list, initial_capital,
        heartbeat, start_date, data_handler,
        execution_handler, portfolio, strategy
    ):
        """
        Initialises the backtest.
```

```

Parameters:
csv_dir - The hard root to the CSV data directory.
symbol_list - The list of symbol strings.
initial_capital - The starting capital for the portfolio.
heartbeat - Backtest "heartbeat" in seconds
start_date - The start datetime of the strategy.
data_handler - (Class) Handles the market data feed.
execution_handler - (Class) Handles the orders/fills for trades.
portfolio - (Class) Keeps track of portfolio current
              and prior positions.
strategy - (Class) Generates signals based on market data.
"""
self.csv_dir = csv_dir
self.symbol_list = symbol_list
self.initial_capital = initial_capital
self.heartbeat = heartbeat
self.start_date = start_date

self.data_handler_cls = data_handler
self.execution_handler_cls = execution_handler
self.portfolio_cls = portfolio
self.strategy_cls = strategy

self.events = queue.Queue()

self.signals = 0
self.orders = 0
self.fills = 0
self.num_strats = 1

self._generate_trading_instances()

```

The first method, `_generate_trading_instances`, attaches all of the trading objects (DataHandler, Strategy, Portfolio and ExecutionHandler) to various internal members:

```

# backtest.py

def _generate_trading_instances(self):
    """
    Generates the trading instance objects from
    their class types.
    """
    print(
        "Creating DataHandler, Strategy, Portfolio and ExecutionHandler"
    )
    self.data_handler = self.data_handler_cls(self.events, self.csv_dir,
                                              self.symbol_list)
    self.strategy = self.strategy_cls(self.data_handler, self.events)
    self.portfolio = self.portfolio_cls(self.data_handler, self.events,
                                         self.start_date,
                                         self.initial_capital)
    self.execution_handler = self.execution_handler_cls(self.events)

```

The `_run_backtest` method is where the signal handling of the Backtest engine is carried out. As described above there are two while loops, one nested within another. The outer keeps track of the heartbeat of the system, while the inner checks if there is an event in the Queue object, and acts on it by calling the appropriate method on the necessary object.

For a MarketEvent, the Strategy object is told to recalculate new signals, while the Portfolio object is told to reindex the time. If a SignalEvent object is received the Portfolio is told to handle the new signal and convert it into a set of OrderEvents, if appropriate. If an OrderEvent is received the ExecutionHandler is sent the order to be transmitted to the broker (if in a real trading setting). Finally, if a FillEvent is received, the Portfolio will update itself to be aware of the new positions:

```
# backtest.py

def _run_backtest(self):
    """
    Executes the backtest.
    """
    i = 0
    while True:
        i += 1
        print i
        # Update the market bars
        if self.data_handler.continue_backtest == True:
            self.data_handler.updateBars()
        else:
            break

        # Handle the events
        while True:
            try:
                event = self.events.get(False)
            except queue.Empty:
                break
            else:
                if event is not None:
                    if event.type == 'MARKET':
                        self.strategy.calculate_signals(event)
                        self.portfolio.update_timeindex(event)

                    elif event.type == 'SIGNAL':
                        self.signals += 1
                        self.portfolio.update_signal(event)

                    elif event.type == 'ORDER':
                        self.orders += 1
                        self.execution_handler.execute_order(event)

                    elif event.type == 'FILL':
                        self.fills += 1
                        self.portfolio.update_fill(event)

                time.sleep(self.heartbeat)
```

Once the backtest simulation is complete the performance of the strategy can be displayed to the terminal/console. The equity curve pandas DataFrame is created and the summary statistics are displayed, as well as the count of Signals, Orders and Fills:

```
# backtest.py

def _output_performance(self):
    """
    Outputs the strategy performance from the backtest.
```

```

"""
self.portfolio.create_equity_curve_dataframe()

print("Creating summary stats...")
stats = self.portfolio.output_summary_stats()

print("Creating equity curve...")
print(self.portfolio.equity_curve.tail(10))
pprint.pprint(stats)

print("Signals: %s" % self.signals)
print("Orders: %s" % self.orders)
print("Fills: %s" % self.fills)

```

The last method to be implemented is `simulate_trading`. It simply calls the two previously described methods, in order:

```

# backtest.py

def simulate_trading(self):
    """
    Simulates the backtest and outputs portfolio performance.
    """
    self._run_backtest()
    self._output_performance()

```

This concludes the event-driven backtester operational objects.

14.3 Event-Driven Execution

Above we described a basic `ExecutionHandler` class that simply created a corresponding `FillEvent` instance for every `OrderEvent`. This is precisely what we need for a "first pass" backtest, but when we wish to actually hook up the system to a brokerage, we need more sophisticated handling. In this section we define the `IBExecutionHandler`, a class that allows us to talk to the popular Interactive Brokers API and thus automate our execution.

The essential idea of the `IBExecutionHandler` class is to receive `OrderEvent` instances from the events queue and then to execute them directly against the Interactive Brokers order API using the open source `IbPy` library. The class will also handle the "Server Response" messages sent back via the API. At this stage, the only action taken will be to create corresponding `FillEvent` instances that will then be sent back to the events queue.

The class itself could feasibly become rather complex, with execution optimisation logic as well as sophisticated error handling. However, I have opted to keep it relatively simple here so that you can see the main ideas and extend it in the direction that suits your particular trading style.

As always, the first task is to create the Python file and import the necessary libraries. The file is called `ib_execution.py` and lives in the same directory as the other event-driven files.

We import the necessary date/time handling libraries, the `IbPy` objects and the specific Event objects that are handled by `IBExecutionHandler`:

```

#!/usr/bin/python
# -*- coding: utf-8 -*-

# ib_execution.py

from __future__ import print_function

import datetime
import time

```

Chapter 15

Trading Strategy Implementation

In this chapter we are going to consider the full implementation of trading strategies using the aforementioned event-driven backtesting system. In particular we will generate equity curves for all trading strategies using notional portfolio amounts, thus simulating the concepts of margin/leverage, which is a far more realistic approach compared to vectorised/returns based approaches.

The first set of strategies are able to be carried out with freely available data, either from Yahoo Finance, Google Finance or Quandl. These strategies are suitable for long-term algorithmic traders who may wish to only study the trade signal generation aspect of the strategy or even the full end-to-end system. Such strategies often possess smaller Sharpe ratios, but are far easier to implement and execute.

The latter strategy is carried out using intraday equities data. This data is often not freely available and a commercial data vendor is usually necessary to provide sufficient quality and quantity of data. I myself use DTN IQFeed for intraday bars. Such strategies often possess much larger Sharpe ratios, but require more sophisticated implementation as the high frequency requires extensive automation.

We will see that our first two attempts at creating a trading strategy on interday data are not altogether successful. It can be challenging to come up with a profitable trading strategy on interday data once transaction costs have been taken into account. The latter is something that many texts on algorithmic trading tend to leave out. However, it is my belief that as many factors as possible must be added to the backtest in order to minimise surprises going forward.

In addition, this book is primarily about how to effectively create a realistic interday or intraday backtesting system (as well as a live execution platform) and less about particular individual strategies. It is far harder to create a realistic robust backtester than it is to find trading strategies on the internet! While the first two strategies presented are not particularly attractive, the latter strategy (on intraday data) performs well and gives us confidence in using higher frequency data.

15.1 Moving Average Crossover Strategy

I'm quite fond of the Moving Average Crossover technical system because it is the first non-trivial strategy that is extremely handy for testing a new backtesting implementation. On a daily timeframe, over a number of years, with long lookback periods, few signals are generated on a single stock and thus it is easy to manually verify that the system is behaving as would be expected.

In order to actually generate such a simulation based on the prior backtesting code we need to subclass the **Strategy** object as described in the previous chapter to create the **MovingAverageCrossStrategy** object, which will contain the logic of the simple moving averages and the generation of trading signals.

In addition we need to create the `__main__` function that will load the **Backtest** object and actually encapsulate the execution of the program. The following file, `mac.py`, contains both of these objects.

The first task, as always, is to correctly import the necessary components. We are importing nearly all of the objects that have been described in the previous chapter:

```
#!/usr/bin/python
# -*- coding: utf-8 -*-

# mac.py

from __future__ import print_function

import datetime

import numpy as np
import pandas as pd
import statsmodels.api as sm

from strategy import Strategy
from event import SignalEvent
from backtest import Backtest
from data import HistoricCSVDDataHandler
from execution import SimulatedExecutionHandler
from portfolio import Portfolio
```

Now we turn to the creation of the `MovingAverageCrossStrategy`. The strategy requires both the bars `DataHandler`, the events Event Queue and the lookback periods for the simple moving averages that are going to be employed within the strategy. I've chosen 100 and 400 as the "short" and "long" lookback periods for this strategy.

The final attribute, `bought`, is used to tell the `Strategy` when the backtest is actually "in the market". Entry signals are only generated if this is "OUT" and exit signals are only ever generated if this is "LONG" or "SHORT":

```
# mac.py

class MovingAverageCrossStrategy(Strategy):
    """
    Carries out a basic Moving Average Crossover strategy with a
    short/long simple weighted moving average. Default short/long
    windows are 100/400 periods respectively.
    """

    def __init__(
        self, bars, events, short_window=100, long_window=400
    ):
        """
        Initialises the Moving Average Cross Strategy.

        Parameters:
        bars - The DataHandler object that provides bar information
        events - The Event Queue object.
        short_window - The short moving average lookback.
        long_window - The long moving average lookback.
        """
        self.bars = bars
        self.symbol_list = self.bars.symbol_list
        self.events = events
        self.short_window = short_window
        self.long_window = long_window
```

```
# Set to True if a symbol is in the market
self.bought = self._calculate_initial_bought()
```

Since the strategy begins out of the market we set the initial "bought" value to be "OUT", for each symbol:

```
# mac.py
```

```
def _calculate_initial_bought(self):
    """
    Adds keys to the bought dictionary for all symbols
    and sets them to 'OUT'.
    """
    bought = {}
    for s in self.symbol_list:
        bought[s] = 'OUT'
    return bought
```

The core of the strategy is the `calculate_signals` method. It reacts to a `MarketEvent` object and for each symbol traded obtains the latest N bar closing prices, where N is equal to the largest lookback period.

It then calculates both the short and long period *simple moving averages*. The rule of the strategy is to enter the market (go long a stock) when the short moving average value exceeds the long moving average value. Conversely, if the long moving average value exceeds the short moving average value the strategy is told to exit the market.

This logic is handled by placing a `SignalEvent` object on the events Event Queue in each of the respective situations and then updating the "bought" attribute (per symbol) to be "LONG" or "OUT", respectively. Since this is a long-only strategy, we won't be considering "SHORT" positions:

```
# mac.py
```

```
def calculate_signals(self, event):
    """
    Generates a new set of signals based on the MAC
    SMA with the short window crossing the long window
    meaning a long entry and vice versa for a short entry.

    Parameters
    event - A MarketEvent object.
    """
    if event.type == 'MARKET':
        for s in self.symbol_list:
            bars = self.bars.get_latest_bars_values(
                s, "adj_close", N=self.long_window
            )
            bar_date = self.bars.get_latest_bar_datetime(s)
            if bars is not None and bars != []:
                short_sma = np.mean(bars[-self.short_window:])
                long_sma = np.mean(bars[-self.long_window:])

                symbol = s
                dt = datetime.datetime.utcnow()
                sig_dir = ""

                if short_sma > long_sma and self.bought[s] == "OUT":
                    print("LONG: %s" % bar_date)
                    sig_dir = 'LONG'
```

```

        signal = SignalEvent(1, symbol, dt, sig_dir, 1.0)
        self.events.put(signal)
        self.bought[s] = 'LONG'
    elif short_sma < long_sma and self.bought[s] == "LONG":
        print("SHORT: %s" % bar_date)
        sig_dir = 'EXIT'
        signal = SignalEvent(1, symbol, dt, sig_dir, 1.0)
        self.events.put(signal)
        self.bought[s] = 'OUT'

```

That concludes the `MovingAverageCrossStrategy` object implementation. The final task of the entire backtesting system is populate a `__main__` method in `mac.py` to actually execute the backtest.

Firstly, make sure to change the value of `csv_dir` to the absolute path of your CSV file directory for the financial data. You will also need to download the CSV file of the AAPL stock (from Yahoo Finance), which is given by the following link (for Jan 1st 1990 to Jan 1st 2002), since this is the stock we will be testing the strategy on:

<http://ichart.finance.yahoo.com/table.csv?s=AAPL&a=00&b=1&c=1990&d=00&e=1&f=2002&g=d&ignore=.csv>

Make sure to place this file in the path pointed to from the main function in `csv_dir`.

The `__main__` function simply instantiates a new backtest object and then calls the `simulate_trading` method on it to execute it:

```

# mac.py

if __name__ == "__main__":
    csv_dir = '/path/to/your/csv/file' # CHANGE THIS!
    symbol_list = ['AAPL']
    initial_capital = 100000.0
    heartbeat = 0.0
    start_date = datetime.datetime(1990, 1, 1, 0, 0, 0)

    backtest = Backtest(
        csv_dir, symbol_list, initial_capital, heartbeat,
        start_date, HistoricCSVDataHandler, SimulatedExecutionHandler,
        Portfolio, MovingAverageCrossStrategy
    )
    backtest.simulate_trading()

```

To run the code, make sure you have already set up a Python environment (as described in the previous chapters) and then navigate the directory where your code is stored. You should simply be able to run:

```
python mac.py
```

You will see the following listing (truncated due to the bar count printout!):

```

..
..
3029
3030
Creating summary stats...
Creating equity curve...

```

	AAPL	cash	commission	total	returns	equity_curve	drawdown
datetime							
2001-12-18	0	99211	13	99211	0	0.99211	0.025383
2001-12-19	0	99211	13	99211	0	0.99211	0.025383
2001-12-20	0	99211	13	99211	0	0.99211	0.025383
2001-12-21	0	99211	13	99211	0	0.99211	0.025383

```

2001-12-24    0  99211          13  99211          0    0.99211  0.025383
2001-12-26    0  99211          13  99211          0    0.99211  0.025383
2001-12-27    0  99211          13  99211          0    0.99211  0.025383
2001-12-28    0  99211          13  99211          0    0.99211  0.025383
2001-12-31    0  99211          13  99211          0    0.99211  0.025383
2001-12-31    0  99211          13  99211          0    0.99211  0.025383
[('Total Return', '-0.79%'),
 ('Sharpe Ratio', '-0.09'),
 ('Max Drawdown', '2.56%'),
 ('Drawdown Duration', '2312')]
Signals: 10
Orders: 10
Fills: 10

```

The performance of this strategy can be seen in Fig 15.1:

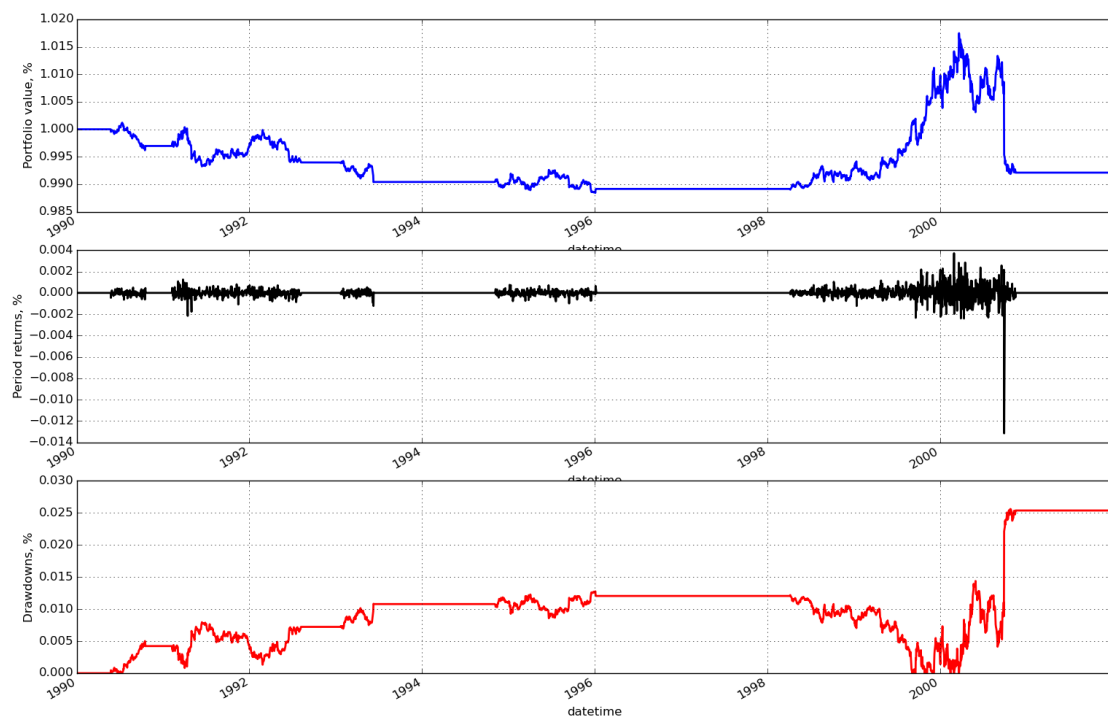


Figure 15.1: Equity Curve, Daily Returns and Drawdowns for the Moving Average Crossover strategy

Evidently the returns and Sharpe Ratio are not stellar for AAPL stock on this particular set of technical indicators! Clearly we have some work to do in the next set of strategies to find a system that can generate positive performance.

15.2 S&P500 Forecasting Trade

In this section we will consider a trading strategy built around the forecasting engine discussed in prior chapters. We will attempt to trade off the predictions made by a stock market forecaster.

We are going to attempt to forecast SPY, which is the ETF that tracks the value of the S&P500. Ultimately we want to answer the question as to whether a basic forecasting algorithm using lagged price data, with slight predictive performance, provides us with any benefit over a buy-and-hold strategy.

The rules for this strategy are as follows:

1. Fit a forecasting model to a subset of S&P500 data. This could be Logistic Regression, a Discriminant Analyser (Linear or Quadratic), a Support Vector Machine or a Random Forest. The procedure to do this was outlined in the Forecasting chapter.
2. Use two prior lags of adjusted closing returns data as a predictor for tomorrow's returns. If the returns are predicted as positive then go long. If the returns are predicted as negative then exit. We're not going to consider short selling for this particular strategy.

Implementation

For this strategy we are going to create the `snp_forecast.py` file and import the following necessary libraries:

```
#!/usr/bin/python
# -*- coding: utf-8 -*-

# snp_forecast.py

from __future__ import print_function

import datetime

import pandas as pd
from sklearn.qda import QDA

from strategy import Strategy
from event import SignalEvent
from backtest import Backtest
from data import HistoricCSVDDataHandler
from execution import SimulatedExecutionHandler
from portfolio import Portfolio
from create_lagged_series import create_lagged_series
```

We have imported Pandas and Scikit-Learn in order to carry out the fitting procedure for the supervised classifier model. We have also imported the necessary classes from the event-driven backtester. Finally, we have imported the `create_lagged_series` function, which we used in the Forecasting chapter.

The next step is to create the `SPYDailyForecastStrategy` as a subclass of the `Strategy` abstract base class. Since we will "hardcode" the parameters of the strategy directly into the class, for simplicity, the only parameters necessary for the `__init__` constructor are the `bars` data handler and the `events` queue.

We set the `self.model` start/end/test dates as datetime objects and then tell the class that we are out of the market (`self.long_market = False`). Finally, we set `self.model` to be the trained model from the `create_symbol_forecast_model` below:

```
# snp_forecast.py

class SPYDailyForecastStrategy(Strategy):
    """
```


S&P500 forecast strategy. It uses a Quadratic Discriminant Analyser to predict the returns for a subsequent time period and then generated long/exit signals based on the prediction.

```
"""
def __init__(self, bars, events):
    self.bars = bars
    self.symbol_list = self.bars.symbol_list
    self.events = events
    self.datetime_now = datetime.datetime.utcnow()

    self.model_start_date = datetime.datetime(2001,1,10)
    self.model_end_date = datetime.datetime(2005,12,31)
    self.model_start_test_date = datetime.datetime(2005,1,1)

    self.long_market = False
    self.short_market = False
    self.bar_index = 0

    self.model = self.create_symbol_forecast_model()
```

Here we define the `create_symbol_forecast_model`. It essentially calls the `create_lagged_series` function, which produces a Pandas DataFrame with five daily returns lags for each current predictor. We then consider only the two most recent of these lags. This is because we are making the modelling decision that the predictive power of earlier lags is likely to be minimal.

At this stage we create the training and test data, the latter of which can be used to test our model if we wish. I have opted to not output testing data, since we have already trained the model before in the Forecasting chapter. Finally we fit the training data to the Quadratic Discriminant Analyser and then return the model.

Note that we could easily replace the model with a Random Forest, Support Vector Machine or Logistic Regression, for instance. All we need to do is import the correct library from Scikit-Learn and simply replace the `model = QDA()` line:

snp_forecast.py

```
def create_symbol_forecast_model(self):
    # Create a lagged series of the S&P500 US stock market index
    snpret = create_lagged_series(
        self.symbol_list[0], self.model_start_date,
        self.model_end_date, lags=5
    )

    # Use the prior two days of returns as predictor
    # values, with direction as the response
    X = snpret[["Lag1","Lag2"]]
    y = snpret["Direction"]

    # Create training and test sets
    start_test = self.model_start_test_date
    X_train = X[X.index < start_test]
    X_test = X[X.index >= start_test]
    y_train = y[y.index < start_test]
    y_test = y[y.index >= start_test]

    model = QDA()
    model.fit(X_train, y_train)
    return model
```

At this stage we are ready to override the `calculate_signals` method of the `Strategy` base class. We firstly calculate some convenience parameters that enter our `SignalEvent` object and then only generate a set of signals if we have received a `MarketEvent` object (a basic sanity check).

We wait for five bars to have elapsed (i.e. five days in this strategy!) and then obtain the lagged returns values. We then wrap these values in a Pandas Series so that the `predict` method of the model will function correctly. We then calculate a prediction, which manifests itself as a +1 or -1.

If the prediction is a +1 and we are not already long the market, we create a `SignalEvent` to go long and let the class know we are now in the market. If the prediction is -1 and we are long the market, then we simply exit the market:

snp_forecast.py

```
def calculate_signals(self, event):
    """
    Calculate the SignalEvents based on market data.
    """
    sym = self.symbol_list[0]
    dt = self.datetime_now

    if event.type == 'MARKET':
        self.bar_index += 1
        if self.bar_index > 5:
            lags = self.bars.get_latest_bars_values(
                self.symbol_list[0], "returns", N=3
            )
            pred_series = pd.Series(
                {
                    'Lag1': lags[1]*100.0,
                    'Lag2': lags[2]*100.0
                }
            )
            pred = self.model.predict(pred_series)
            if pred > 0 and not self.long_market:
                self.long_market = True
                signal = SignalEvent(1, sym, dt, 'LONG', 1.0)
                self.events.put(signal)

            if pred < 0 and self.long_market:
                self.long_market = False
                signal = SignalEvent(1, sym, dt, 'EXIT', 1.0)
                self.events.put(signal)
```

In order to run the strategy you will need to download a CSV file from Yahoo Finance for SPY and place it in a suitable directory (note that you will need to change your path below!). We then wrap the backtest up via the `Backtest` class and carry out the test by calling `simulate_trading`:

snp_forecast.py

```
if __name__ == "__main__":
    csv_dir = '/path/to/your/csv/file' # CHANGE THIS!
    symbol_list = ['SPY']
    initial_capital = 100000.0
    heartbeat = 0.0
    start_date = datetime.datetime(2006,1,3)

    backtest = Backtest(
```

```

        csv_dir, symbol_list, initial_capital, heartbeat,
        start_date, HistoricCSVDataHandler, SimulatedExecutionHandler,
        Portfolio, SPYDailyForecastStrategy
    )
    backtest.simulate_trading()

```

The output of the strategy is as follows and is net of transaction costs:

```

..
..
2209
2210
Creating summary stats...
Creating equity curve...

```

	SPY	cash	commission	total	returns	equity_curve \
datetime						
2014-09-29	19754	90563.3	349.7	110317.3	-0.000326	1.103173
2014-09-30	19702	90563.3	349.7	110265.3	-0.000471	1.102653
2014-10-01	19435	90563.3	349.7	109998.3	-0.002421	1.099983
2014-10-02	19438	90563.3	349.7	110001.3	0.000027	1.100013
2014-10-03	19652	90563.3	349.7	110215.3	0.001945	1.102153
2014-10-06	19629	90563.3	349.7	110192.3	-0.000209	1.101923
2014-10-07	19326	90563.3	349.7	109889.3	-0.002750	1.098893
2014-10-08	19664	90563.3	349.7	110227.3	0.003076	1.102273
2014-10-09	19274	90563.3	349.7	109837.3	-0.003538	1.098373
2014-10-09	0	109836.0	351.0	109836.0	-0.000012	1.098360

```

drawdown
datetime
2014-09-29 0.003340
2014-09-30 0.003860
2014-10-01 0.006530
2014-10-02 0.006500
2014-10-03 0.004360
2014-10-06 0.004590
2014-10-07 0.007620
2014-10-08 0.004240
2014-10-09 0.008140
2014-10-09 0.008153
[('Total Return', '9.84%'),
 ('Sharpe Ratio', '0.54'),
 ('Max Drawdown', '5.99%'),
 ('Drawdown Duration', '811')]
Signals: 270
Orders: 270
Fills: 270

```

The following visualisation in Fig 15.2 shows the Equity Curve, the Daily Returns and the Drawdown of the strategy as a function of time:

Note immediately that the performance is not great! We have a Sharpe Ratio < 1 but a reasonable drawdown of just under 6%. It turns out that if we had simply bought and held SPY in this time period we would have performed similarly, if slightly worse.

Hence we have not actually gained very much from our predictive strategy once transaction costs are included. I specifically wanted to include this example because it uses an "end to end" realistic implementation of such a strategy that takes into account conservative, realistic transaction costs. As can be seen it is not easy to make a predictive forecaster on daily data that produces good performance!

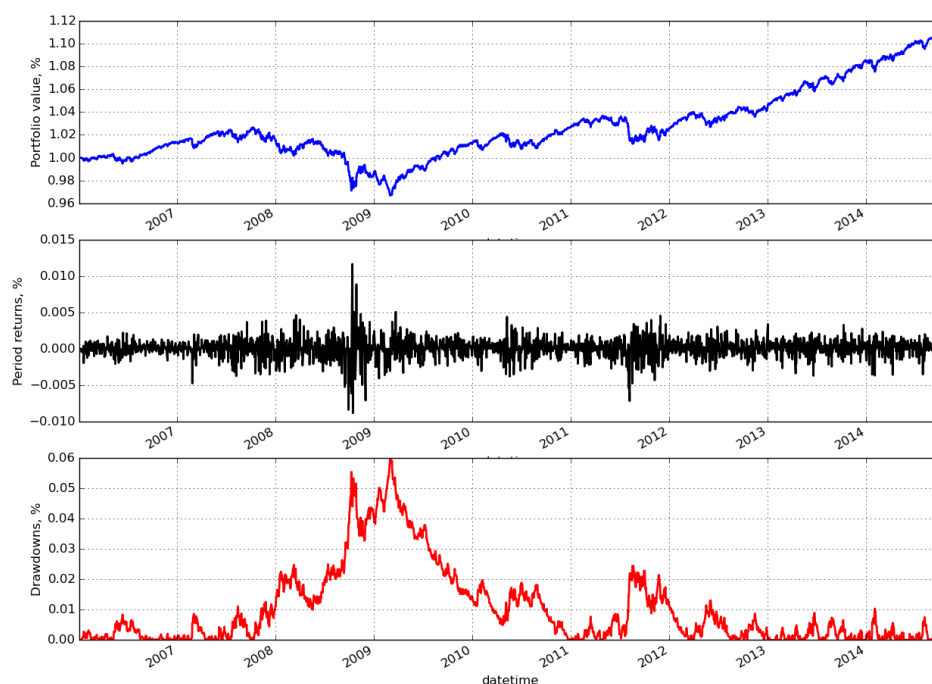


Figure 15.2: Equity Curve, Daily Returns and Drawdowns for the SPY forecast strategy

Our final strategy will make use of other time series and a higher frequency. We will see that performance can be improved dramatically after modifying certain aspects of the system.

15.3 Mean-Reverting Equity Pairs Trade

In order to seek higher Sharpe ratios for our trading, we need to consider higher-frequency intraday strategies.

The first major issue is that obtaining data is significantly less straightforward because high quality intraday data is usually not free. As stated above I use DTN IQFeed for intraday minutely bars and thus you will need your own DTN account to obtain the data required for this strategy.

The second issue is that backtesting simulations take substantially longer, especially with the event-driven model that we have constructed here. Once we begin considering a backtest of a diversified portfolio of minutely data spanning years, and then performing any parameter optimisation, we rapidly realise that simulations can take hours or even days to calculate on a modern desktop PC. This will need to be factored in to your research process.

The third issue is that live execution will now need to be fully automated since we are edging into higher-frequency trading. This means that such execution environments and code must be highly reliable and bug-free, otherwise the potential for significant losses can occur.

This strategy expands on the previous interday strategy above to make use of intraday data. In particular we are going to use minutely OHLCV bars, as opposed to daily OHLCV.

The rules for the strategy are straightforward:

1. Identify a pair of equities that possess a residuals time series which has been statistically identified as mean-reverting. In this case, I have found two energy sector US equities with tickers AREX and WLL.

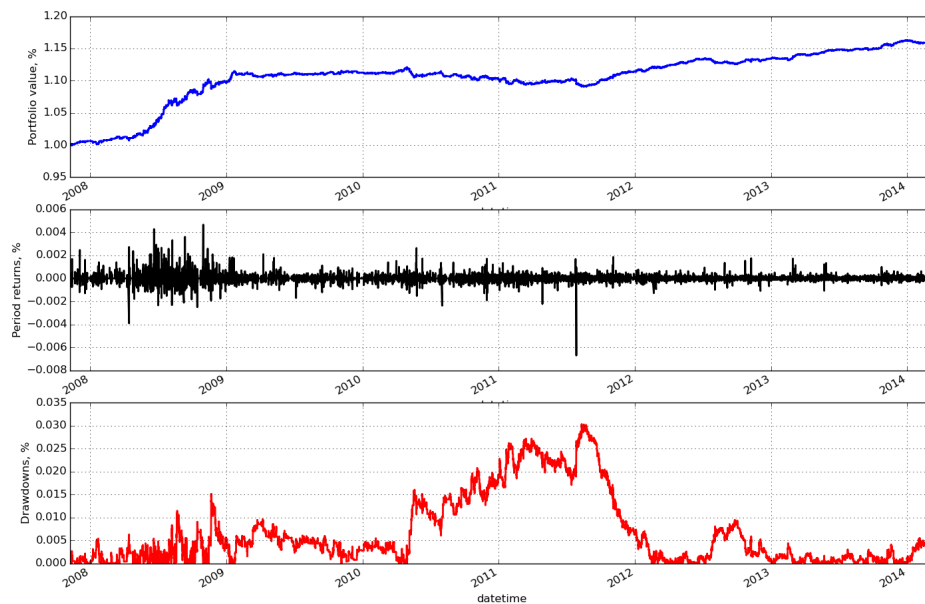


Figure 15.3: Equity Curve, Daily Returns and Drawdowns for intraday mean-reversion strategy

attribute known as `mkt_quantity`. It will be set to 100. Changing this to 2000, for instance, provides these results:

```
..
..
[('Total Return', '392.85%'),
 ('Sharpe Ratio', '2.29'),
 ('Max Drawdown', '45.69%'),
 ('Drawdown Duration', '102150')]
..
..
```

Clearly the Sharpe Ratio and Total Return are much more attractive, but we have to endure a 45% maximum drawdown over this period as well!

15.4 Plotting Performance

The three Figures displayed above are all created using the `plot_performance.py` script. For completeness I've included the code so that you can use it as a base to create your own performance charts.

It is necessary to run this in the same directory as the output file from the backtest, namely where `equity.csv` resides. The listing is as follows:

```
#!/usr/bin/python
# -*- coding: utf-8 -*-

# plot_performance.py

import os.path
```

```
import numpy as np
import matplotlib.pyplot as plt
import pandas as pd

if __name__ == "__main__":
    data = pd.io.parsers.read_csv(
        "equity.csv", header=0,
        parse_dates=True, index_col=0
    ).sort()

    # Plot three charts: Equity curve,
    # period returns, drawdowns
    fig = plt.figure()
    # Set the outer colour to white
    fig.patch.set_facecolor('white')

    # Plot the equity curve
    ax1 = fig.add_subplot(311, ylabel='Portfolio value, %')
    data['equity_curve'].plot(ax=ax1, color="blue", lw=2.)
    plt.grid(True)

    # Plot the returns
    ax2 = fig.add_subplot(312, ylabel='Period returns, %')
    data['returns'].plot(ax=ax2, color="black", lw=2.)
    plt.grid(True)

    # Plot the returns
    ax3 = fig.add_subplot(313, ylabel='Drawdowns, %')
    data['drawdown'].plot(ax=ax3, color="red", lw=2.)
    plt.grid(True)

    # Plot the figure
    plt.show()
```

Chapter 16

Strategy Optimisation

In prior chapters we have considered how to create both an underlying predictive model (such as with the Support Vector Machine and Random Forest Classifier) as well as a trading strategy based upon it. Along the way we have seen that there are many *parameters* to such models. In the case of an SVM we have the "tuning" parameters γ and C . In a Moving Average Crossover trading strategy we have the parameters for the two lookback windows of the moving average filters.

In this chapter we are going to describe optimisation methods to improve the performance of our trading strategies by tuning the parameters in a systematic fashion. For this we will use mechanisms from the statistical field of *Model Selection*, such as cross-validation and grid search. The literature on model selection and parameter optimisation is vast and most of the methods are somewhat beyond the scope of this book. I want to introduce the subject here so that you can explore more sophisticated techniques at your own pace.

16.1 Parameter Optimisation

At this stage nearly all of the trading strategies and underlying statistical models have required one or more parameters in order to be utilised. In momentum strategies using technical indicators, such as with moving averages (simple or exponential), there is a need to specify a lookback window. The same is true of many mean-reverting strategies, which require a (rolling) lookback window in order to calculate a regression between two time series. Particular statistical machine learning models such as a logistic regression, SVM or Random Forest also require parameters in order to be calculated.

The biggest danger when considering parameter optimisation is that of *overfitting* a model or trading strategy. This problem occurs when a model is trained on an *in sample* retained slice of training data and is optimised to perform well (by the appropriate performance measure), but performance degrades substantially when applied to *out of sample* data. For instance, a trading strategy could perform extremely well in the backtest (the in sample data) but when deployed for live trading can be completely unprofitable.

An additional concern of parameter optimisation is that it can become very computationally expensive. With modern computational systems this is less of an issue than it once was, due to parallelisation and fast CPUs. However, multiple parameter optimisation can increase computational complexity by orders of magnitudes. One must be aware of this as part of the research and development process.

16.1.1 Which Parameters to Optimise?

A statistical-based algorithmic trading model will often have many parameters and different measures of performance. An underlying statistical learning algorithm will have its own set of parameters. In the case of a multiple linear or logistic regression these would be the β_i coefficients. In the case of a random forest one such parameter would be the number of underlying decision trees to use in the ensemble. Once applied to a trading model other parameters might be entry

and exit thresholds, such as a z-score of a particular time series. The z-score itself might have an implicit rolling lookback window. As can be seen the number of parameters can be quite extensive.

In addition to parameters there are numerous means of evaluating the performance of a statistical model and the trading strategy based upon it. We have defined concepts such as the hit rate and the confusion matrix. In addition there are more statistical measures such as the *Mean Squared Error* (MSE). These are performance measures that would be optimised at the statistical model level, via parameters relevant to their domain.

The actual trading strategy is evaluated on different criteria, such as compound annual growth rate (CAGR) and maximum drawdown. We would need to vary entry and exit criteria, as well as other thresholds that are not directly related to the statistical model. Hence this motivates the question as to which set of parameters to optimise and when.

In the following sections we are going to optimise both the statistical model parameters, at the early research and development stage, as well as the parameters associated with a trading strategy using an underlying optimised statistical model, on each of their respective performance measures.

16.1.2 Optimisation is Expensive

With multiple real-valued parameters, optimisation can rapidly become extremely expensive, as each new parameter adds an additional spatial dimension. If we consider the example of a *grid search* (to be discussed in full below), and have a single parameter α , then we might wish to vary α within the set $\{0.1, 0.2, 0.3, 0.4, 0.5\}$. This requires 5 simulations.

If we now consider an additional parameter β , which may vary in the range $\{0.2, 0.4, 0.6, 0.8, 1.0\}$, then we will have to consider $5^2 = 25$ simulations. Another parameter, γ , with 5 variations brings this to $5^3 = 125$ simulations. If each parameter had 10 separate values to be tested, this would be equal to $10^3 = 1000$ simulations. As can be seen the parameter search space can rapidly make such simulations extremely expensive.

It is clear that a trade-off exists between conducting an exhaustive parameter search and maintaining a reasonable total simulation time. While parallelism, including many-core CPUs and graphics processing units (GPUs), have mitigated the issue somewhat, we still need to be careful when introducing parameters. This notion of reducing parameters is also an issue of model effectiveness, as we shall see below.

16.1.3 Overfitting

Overfitting is the process of optimising a parameter, or set of parameters, against a particular data set such that an appropriate performance measure (or error measure) is found to be maximised (or minimised), but when applied to an unseen data set, such a performance measure degrades substantially. The concept is closely related to the idea of the *bias-variance dilemma*.

The bias-variance dilemma concerns the situation where a statistical model has a trade-off between being a low-bias model or a low-variance model, or a compromise between the two. *Bias* refers to the difference between the model's estimation of a parameter and the true "population" value of the parameter, or erroneous assumptions in the statistical model. *Variance* refers to the error from the sensitivity of the model to small fluctuations in the training set (in sample data).

In all statistical models one is simultaneously trying to minimise both the bias error and the variance error in order to improve model accuracy. Such a situation can lead to overfitting in models, as the training error can be substantially reduced by introducing models with more flexibility (variation). However, such models can perform extremely poorly on new (out of sample) data since they were essentially "fit" to the in sample data.

A common example of a high-bias, low-variance model is that of linear regression applied to a non-linear data set. Additions of new points do not affect the regression slope dramatically (assuming they are not too far from the remaining data), but since the problem is inherently non-linear, there is a systematic bias in the results by using a linear model.

A common example of a low-bias, high-variance model is that of a polynomial spline fit applied to a non-linear data set. The parameter of the model (the degree of the polynomial)

could be adjusted to fit such a model very precisely (i.e. low-bias on the training data), but additions of new points would almost certainly lead to the model having to modify its degree of polynomial to fit the new data. This would make it a very high-variance model on the in sample data. Such a model would likely have very poor predictability or inferential capability on out of sample data.

Overfitting can also manifest itself on the trading strategy and not just the statistical model. For instance, we could optimise the Sharpe ratio by varying entry and exit threshold parameters. While this may improve profitability in the backtest (or minimise risk substantially), it would likely not be behaviour that is replicated when the strategy was deployed live, as we might have been fitting such optimisations to noise in the historical data.

We will discuss techniques below to minimise overfitting, as much as possible. However one has to be aware that it is an ever-present danger in both algorithmic trading and statistical analysis in general.

16.2 Model Selection

In this section we are going to consider how to optimise the statistical model that will underly a trading strategy. In the field of statistics and machine learning this is known as *Model Selection*. While I won't present an exhaustive discussion on the various model selection techniques, I will describe some of the basic mechanisms such as *Cross Validation* and *Grid Search* that work well for trading strategies.

16.2.1 Cross Validation

Cross Validation is a technique used to assess how a statistical model will generalise to new data that it has not been exposed to before. Such a technique is usually used on predictive models, such as the aforementioned supervised classifiers used to predict the sign of the following daily returns of an asset price series. Fundamentally, the goal of cross validation is to minimise error on out of sample data without leading to an overfit model.

In this section we will describe the *training/test split* and *k-fold cross validation*, as well as use techniques within Scikit-Learn to automatically carry out these procedures on statistical models we have already developed.

Train/Test Split

The simplest example of cross validation is known as a *training/test split*, or a *2-fold cross validation*. Once a prior historical data set is assembled (such as a daily time series of asset prices), it is split into two components. The ratio of the split is usually varied between 0.5 and 0.8. In the latter case this means 80% of the data is used for training and 20% is used for testing. All of the statistics of interest, such as the hit rate, confusion matrix or mean-squared error are calculated on the test set, which has not been used within the training process.

To carry out this process in Python with Scikit-Learn we can use the `sklearn.cross_validation.train_test_split` method. We will continue with our model as discussed in the chapter on Forecasting. In particular, we are going to modify `forecast.py` and create a new file called `train_test_split.py`. We will need to add the new import to the list of imports:

```
#!/usr/bin/python
# -*- coding: utf-8 -*-

# train_test_split.py

from __future__ import print_function

import datetime

import sklearn
```

```

from sklearn.cross_validation import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.lda import LDA
from sklearn.metrics import confusion_matrix
from sklearn.qda import QDA
from sklearn.svm import LinearSVC, SVC

from create_lagged_series import create_lagged_series

```

In `forecast.py` we originally split the data based on a particular date within the time series:

```

# forecast.py

..

# The test data is split into two parts: Before and after 1st Jan 2005.
start_test = datetime.datetime(2005,1,1)

# Create training and test sets
X_train = X[X.index < start_test]
X_test = X[X.index >= start_test]
y_train = y[y.index < start_test]
y_test = y[y.index >= start_test]
..

```

This can be replaced with the method `train_test_split` from Scikit-Learn in the `train_test_split.py` file. For completeness, the full `__main__` method is provided below:

```

# train_test_split.py

if __name__ == "__main__":
    # Create a lagged series of the S&P500 US stock market index
    snpret = create_lagged_series(
        "^GSPC", datetime.datetime(2001,1,10),
        datetime.datetime(2005,12,31), lags=5
    )

    # Use the prior two days of returns as predictor
    # values, with direction as the response
    X = snpret[["Lag1","Lag2"]]
    y = snpret["Direction"]

    # Train/test split
    X_train, X_test, y_train, y_test = train_test_split(
        X, y, test_size=0.8, random_state=42
    )

    # Create the (parametrised) models
    print("Hit Rates/Confusion Matrices:\n")
    models = [("LR", LogisticRegression()),
               ("LDA", LDA()),
               ("QDA", QDA()),
               ("LSVC", LinearSVC()),
               ("RSVM", SVC(
                   C=1000000.0, cache_size=200, class_weight=None,
                   coef0=0.0, degree=3, gamma=0.0001, kernel='rbf',
                   max_iter=-1, probability=False, random_state=None,

```

```

        shrinking=True, tol=0.001, verbose=False)
    ),
    ("RF", RandomForestClassifier(
        n_estimators=1000, criterion='gini',
        max_depth=None, min_samples_split=2,
        min_samples_leaf=1, max_features='auto',
        bootstrap=True, oob_score=False, n_jobs=1,
        random_state=None, verbose=0)
    )]

# Iterate through the models
for m in models:

    # Train each of the models on the training set
    m[1].fit(X_train, y_train)

    # Make an array of predictions on the test set
    pred = m[1].predict(X_test)

    # Output the hit-rate and the confusion matrix for each model
    print("%s:\n%0.3f" % (m[0], m[1].score(X_test, y_test)))
    print("%s\n" % confusion_matrix(pred, y_test))

```

Notice that we have picked the ratio of the training set to be 80% of the data, leaving the testing data with only 20%. In addition we have specified a `random_state` to randomise the sampling within the selection of data. This means that the data is not sequentially divided chronologically, but rather is sampled randomly.

The results of the cross-validation on the model are as follows (yours will likely appear slightly different due to the nature of the fitting procedure):

Hit Rates/Confusion Matrices:

```

LR:
0.511
[[ 70  70]
 [419 441]]

LDA:
0.513
[[ 69  67]
 [420 444]]

QDA:
0.503
[[ 83  91]
 [406 420]]

LSVC:
0.513
[[ 69  67]
 [420 444]]

RSVM:
0.506
[[  8  13]
 [481 498]]

```

```
RF:
0.490
[[200 221]
 [289 290]]
```

It can be seen that the hit rates are substantially lower than those found in the aforementioned forecasting chapter. Consequently we can likely conclude that the particular choice of training/test split lead to an over-optimistic view of the predictive capability of the classifier.

The next step is to increase the number of times a cross-validation is performed in order to minimise any potential overfitting. For this we will use k-fold cross validation.

K-Fold Cross Validation

Rather than partitioning the set into a single training and test set, we can use k-fold cross validation to *randomly* partition the the set into k equally sized subsamples. For each iteration (of which there are k), one of the k subsamples is retained as a test set, while the remaining $k - 1$ subsamples together form a training set. A statistical model is then trained on each of the k folds and its performance evaluated on its specific k -th test set.

The purpose of this is to combine the results of each model into an *ensemble* by means of averaging the results of the prediction (or otherwise) to produce a single prediction. The main benefit of using k-fold cross validation is that the every predictor within the original data set is used both for training and testing only once.

This motivates a question as to how to choose k , which is now another parameter! Generally, $k = 10$ is used but one can also perform another analysis to choose an optimal value of k .

We will now make use of the `cross_validation` module of Scikit-Learn to obtain the `KFold` k-fold cross validation object. We create a new file called `k_fold_cross_val.py`, which is a copy of `train_test_split.py` and modify the imports by adding the following line:

```
#!/usr/bin/python
# -*- coding: utf-8 -*-

# k_fold_cross_val.py

from __future__ import print_function

import datetime

import pandas as pd
import sklearn
from sklearn import cross_validation
from sklearn.metrics import confusion_matrix
from sklearn.svm import SVC

from create_lagged_series import create_lagged_series
```

We then need to make changes the `__main__` function by removing the `train_test_split` method and replacing it with an instance of `KFold`. It takes five parameters.

The first is the length of the dataset, which in this case 1250 days. The second value is K representing the number of folds, which in this case is 10. The third value is `indices`, which I have set to `False`. This means that the actual index values are used for the arrays returned by the iterator. The fourth and fifth are used to randomise the order of the samples.

As before in `forecast.py` and `train_test_split.py` we obtain the lagged series of the S&P500. We then create a set of vectors of predictors (X) and responses (y). We then utilise the `KFold` object and iterate over it. During each iteration we create the training and testing sets for each of the X and y vectors. These are then fed into a radial support vector machine with identical parameters to the aforementioned files and the model is fit.

Finally the hit rate and confusion matrix for each instance of the SVM is output.

```

# k_fold_cross_val.py

if __name__ == "__main__":
    # Create a lagged series of the S&P500 US stock market index
    snpret = create_lagged_series(
        "^GSPC", datetime.datetime(2001,1,10),
        datetime.datetime(2005,12,31), lags=5
    )

    # Use the prior two days of returns as predictor
    # values, with direction as the response
    X = snpret[["Lag1","Lag2"]]
    y = snpret["Direction"]

    # Create a k-fold cross validation object
    kf = cross_validation.KFold(
        len(snpret), n_folds=10, indices=False,
        shuffle=True, random_state=42
    )

    # Use the kf object to create index arrays that
    # state which elements have been retained for training
    # and which elements have been retained for testing
    # for each k-element iteration
    for train_index, test_index in kf:
        X_train = X.ix[X.index[train_index]]
        X_test = X.ix[X.index[test_index]]
        y_train = y.ix[y.index[train_index]]
        y_test = y.ix[y.index[test_index]]

        # In this instance only use the
        # Radial Support Vector Machine (SVM)
        print("Hit Rate/Confusion Matrix:")
        model = SVC(
            C=1000000.0, cache_size=200, class_weight=None,
            coef0=0.0, degree=3, gamma=0.0001, kernel='rbf',
            max_iter=-1, probability=False, random_state=None,
            shrinking=True, tol=0.001, verbose=False
        )

        # Train the model on the retained training data
        model.fit(X_train, y_train)

        # Make an array of predictions on the test set
        pred = model.predict(X_test)

        # Output the hit-rate and the confusion matrix for each model
        print("%.3f" % model.score(X_test, y_test))
        print("%s\n" % confusion_matrix(pred, y_test))

```

The output of the code is as follows:

```

Hit Rate/Confusion Matrix:
0.528
[[11 10]
 [49 55]]

```

Hit Rate/Confusion Matrix:

0.400

```
[[ 2  5]
 [70 48]]
```

Hit Rate/Confusion Matrix:

0.528

```
[[ 8  8]
 [51 58]]
```

Hit Rate/Confusion Matrix:

0.536

```
[[ 6  3]
 [55 61]]
```

Hit Rate/Confusion Matrix:

0.512

```
[[ 7  5]
 [56 57]]
```

Hit Rate/Confusion Matrix:

0.480

```
[[11 11]
 [54 49]]
```

Hit Rate/Confusion Matrix:

0.608

```
[[12 13]
 [36 64]]
```

Hit Rate/Confusion Matrix:

0.440

```
[[ 8 17]
 [53 47]]
```

Hit Rate/Confusion Matrix:

0.560

```
[[10  9]
 [46 60]]
```

Hit Rate/Confusion Matrix:

0.528

```
[[ 9 11]
 [48 57]]
```

It is clear that the hit rate and confusion matrices vary dramatically across the various folds. This is indicative that the model is prone to overfitting, on this particular dataset. A remedy for this is to use significantly more data, either at a higher frequency or over a longer duration.

In order to utilise this model in a trading strategy it would be necessary to combine each of these individually trained classifiers (i.e. each of the K objects) into an ensemble average and then use that combined model for classification within the strategy.

Note that technically it is not appropriate to use simple cross-validation techniques on temporally ordered data (i.e. time-series). There are more sophisticated mechanisms for coping with autocorrelation in this fashion, but I wanted to highlight the approach so we have used time series data for simplicity.

16.2.2 Grid Search

We have so far seen that k-fold cross validation helps us to avoid overfitting in the data by performing validation on every element of the sample. We now turn our attention to optimising the *hyper-parameters* of a particular statistical model. Such parameters are those not directly learnt by the model estimation procedure. For instance, C and γ for a support vector machine. In essence they are the parameters that we need to specify when calling the initialisation of each statistical model. For this procedure we will use a process known as a *grid search*.

The basic idea is to take a range of parameters and assess the performance of the statistical model on each parameter element within the range. To achieve this in Scikit-Learn we can create a `ParameterGrid`. Such an object will produce a list of Python dictionaries that each contain a parameter combination to be fed into a statistical model.

An example code snippet that produces a parameter grid, for parameters related to a support vector machine, is given below:

```
>>> from sklearn.grid_search import ParameterGrid
>>> param_grid = {'C': [1, 10, 100, 1000], 'gamma': [0.001, 0.0001]}
>>> list(ParameterGrid(param_grid))

[{'C': 1, 'gamma': 0.001},
 {'C': 1, 'gamma': 0.0001},
 {'C': 10, 'gamma': 0.001},
 {'C': 10, 'gamma': 0.0001},
 {'C': 100, 'gamma': 0.001},
 {'C': 100, 'gamma': 0.0001},
 {'C': 1000, 'gamma': 0.001},
 {'C': 1000, 'gamma': 0.0001}]
```

Now that we have a suitable means of generating a `ParameterGrid` we need to feed this into a statistical model iteratively to search for an optimal performance score. In this case we are going to seek to maximise the hit rate of the classifier.

The `GridSearchCV` mechanism from Scikit-Learn allows us to perform the actual grid search. In fact, it allows us to perform not only a standard grid search but also a cross validation scheme at the same time.

We are now going to create a new file, `grid_search.py`, that once again uses `create_lagged_series.py` and a support vector machine to perform a cross-validated hyperparameter grid search. To this end we must import the correct libraries:

```
#!/usr/bin/python
# -*- coding: utf-8 -*-

# grid_search.py

from __future__ import print_function

import datetime

import sklearn
from sklearn import cross_validation
from sklearn.cross_validation import train_test_split
from sklearn.grid_search import GridSearchCV
from sklearn.metrics import classification_report
from sklearn.svm import SVC

from create_lagged_series import create_lagged_series
```

As before with `k_fold_cross_val.py` we create a lagged series and then use the previous two days of returns as predictors. We initially create a training/test split such that 50% of the

data can be used for training and cross validation while the remaining data can be "held out" for evaluation.

Subsequently we create the `tuned_parameters` list, which contains a single dictionary denoting the parameters we wish to test over. This will create a *cartesian product* of all parameter lists, i.e. a list of pairs of every possible parameter combination.

Once the parameter list is created we pass it to the `GridSearchCV` class, along with the type of classifier that we're interested in (namely a radial support vector machine), with a k-fold cross-validation *k*-value of 10.

Finally, we train the model and output the best estimator and its associated hit rate scores. In this way we have not only optimised the model parameters via cross validation but we have also optimised the hyperparameters of the model via a parametrised grid search, all in one class! Such succinctness of the code allows significant experimentation without being bogged down by excessive "data wrangling".

```
if __name__ == "__main__":
    # Create a lagged series of the S&P500 US stock market index
    snpret = create_lagged_series(
        "^GSPC", datetime.datetime(2001,1,10),
        datetime.datetime(2005,12,31), lags=5
    )

    # Use the prior two days of returns as predictor
    # values, with direction as the response
    X = snpret[["Lag1","Lag2"]]
    y = snpret["Direction"]

    # Train/test split
    X_train, X_test, y_train, y_test = train_test_split(
        X, y, test_size=0.5, random_state=42
    )

    # Set the parameters by cross-validation
    tuned_parameters = [
        {'kernel': ['rbf'], 'gamma': [1e-3, 1e-4], 'C': [1, 10, 100, 1000]}
    ]

    # Perform the grid search on the tuned parameters
    model = GridSearchCV(SVC(C=1), tuned_parameters, cv=10)
    model.fit(X_train, y_train)

    print("Optimised parameters found on training set:")
    print(model.best_estimator_, "\n")

    print("Grid scores calculated on training set:")
    for params, mean_score, scores in model.grid_scores_:
        print("%0.3f for %r" % (mean_score, params))
```

The output of the grid search cross validation procedure is as follows:

```
Optimised parameters found on training set:
SVC(C=1, cache_size=200, class_weight=None, coef0=0.0, degree=3, gamma=0.001,
    kernel='rbf', max_iter=-1, probability=False, random_state=None,
    shrinking=True, tol=0.001, verbose=False)
```

```
Grid scores calculated on training set:
0.541 for {'kernel': 'rbf', 'C': 1, 'gamma': 0.001}
0.541 for {'kernel': 'rbf', 'C': 1, 'gamma': 0.0001}
0.541 for {'kernel': 'rbf', 'C': 10, 'gamma': 0.001}
```



```

0.541 for {'kernel': 'rbf', 'C': 10, 'gamma': 0.0001}
0.541 for {'kernel': 'rbf', 'C': 100, 'gamma': 0.001}
0.541 for {'kernel': 'rbf', 'C': 100, 'gamma': 0.0001}
0.538 for {'kernel': 'rbf', 'C': 1000, 'gamma': 0.001}
0.541 for {'kernel': 'rbf', 'C': 1000, 'gamma': 0.0001}

```

As we can see $\gamma = 0.001$ and $C = 1$ provides the best hit rate, on the validation set, for this particular radial kernel support vector machine. This model could now form the basis of a forecasting-based trading strategy, as we have previously demonstrated in the prior chapter.

16.3 Optimising Strategies

Up until this point we have concentrated on *model selection* and optimising the underlying statistical model that (might) form the basis of a trading strategy. However, a predictive model and a functioning, profitable algorithmic strategy are two different entities. We now turn our attention to optimising parameters that have a direct effect on profitability and risk metrics.

To achieve this we are going to make use of the event-driven backtesting software that was described in a previous chapter. We will consider a particular strategy that has three parameters associated with it and search through the space formed by the cartesian product of parameters, using a grid search mechanism. We will then attempt to maximise particular metrics such as the Sharpe Ratio or minimise others such as the maximum drawdown.

16.3.1 Intraday Mean Reverting Pairs

The strategy of interest to us in this chapter is the "Intraday Mean Reverting Equity Pairs Trade" using the energy equities AREX and WLL. It contains three parameters that we are capable of optimising: The linear regression lookback period, the residuals z-score entry threshold and the residuals z-score exit threshold.

We will consider a range of values for each parameter and then calculate a backtest for the strategy across each of these ranges, outputting the total return, Sharpe ratio and drawdown characteristics of each simulation, to a CSV file for each parameter set. This will allow us to ascertain an optimised Sharpe or minimised max drawdown for our trading strategy.

16.3.2 Parameter Adjustment

Since the event-driven backtesting software is quite CPU-intensive, we will restrict the parameter range to three values per parameter. This will give us a total of $3^3 = 27$ separate simulations to carry out. The parameter ranges are listed below:

- OLS Lookback Window - $w_l \in \{50, 100, 200\}$
- Z-Score Entry Threshold - $z_h \in \{2.0, 3.0, 4.0\}$
- Z-Score Exit Threshold - $z_l \in \{0.5, 1.0, 1.5\}$

To carry out the set of simulations a cartesian product of all three ranges will be calculated and then the simulation will be carried out for each combination of parameters.

The first task is to modify the `intraday_mr.py` file to include the `product` method from the `itertools` library:

```

# intraday_mr.py
..
from itertools import product
..

```

We can then modify the `__main__` method to include the generation of a parameter list for all three of the parameters discussed above.

The first task is to create the actual parameter ranges for the OLS lookback window, the zscore entry threshold and the zscore exit threshold. Each of these has three separate variations leading to a total of 27 simulations.

Once the ranges are created the `itertools.product` method is used to create a cartesian product of all variations, which is then fed into a list of dictionaries to ensure that the correct keyword arguments are passed to the `Strategy` object.

Finally the backtest is instantiated with the `strat_params_list` forming the final keyword argument:

```
if __name__ == "__main__":
    csv_dir = '/path/to/your/csv/file' # CHANGE THIS!
    symbol_list = ['AREX', 'WLL']
    initial_capital = 100000.0
    heartbeat = 0.0
    start_date = datetime.datetime(2007, 11, 8, 10, 41, 0)

    # Create the strategy parameter grid
    # using the itertools cartesian product generator
    strat_lookback = [50, 100, 200]
    strat_z_entry = [2.0, 3.0, 4.0]
    strat_z_exit = [0.5, 1.0, 1.5]
    strat_params_list = list(product(
        strat_lookback, strat_z_entry, strat_z_exit
    ))

    # Create a list of dictionaries with the correct
    # keyword/value pairs for the strategy parameters
    strat_params_dict_list = [
        dict(ols_window=sp[0], zscore_high=sp[1], zscore_low=sp[2])
        for sp in strat_params_list
    ]

    # Carry out the set of backtests for all parameter combinations
    backtest = Backtest(
        csv_dir, symbol_list, initial_capital, heartbeat,
        start_date, HistoricCSVDataHandlerHFT, SimulatedExecutionHandler,
        PortfolioHFT, IntradayOLSMRStrategy,
        strat_params_list=strat_params_dict_list
    )
    backtest.simulate_trading()
```

The next step is to modify the `Backtest` object in `backtest.py` to be able to handle multiple parameter sets. We need to modify the `_generate_trading_instances` method to have an argument that represents the particular parameter set on creation of a new `Strategy` object:

```
# backtest.py

..
def _generate_trading_instances(self, strategy_params_dict):
    """
    Generates the trading instance objects from
    their class types.
    """
    print("Creating DataHandler, Strategy, Portfolio and ExecutionHandler for")
    print("strategy parameter list: %s..." % strategy_params_dict)
    self.data_handler = self.data_handler_cls(
        self.events, self.csv_dir, self.symbol_list, self.header_strings
    )
```

```

        self.strategy = self.strategy_cls(
            self.data_handler, self.events, **strategy_params_dict
        )
        self.portfolio = self.portfolio_cls(
            self.data_handler, self.events, self.start_date,
            self.num_strats, self.periods, self.initial_capital
        )
        self.execution_handler = self.execution_handler_cls(self.events)
    ..

```

This method is now called within a strategy parameter list loop, rather than at construction of the **Backtest** object. While it may seem wasteful to recreate all of the data handlers, event queues and portfolio objects for each parameter set, it ensures that all of the iterators have been reset and that we are truly starting with a "clean slate" for each simulation.

The next task is to modify the `simulate_trading` method to loop over all variants of strategy parameters. The method creates an output CSV file that is used to store parameter combinations and their particular performance metrics. This will allow us later to plot performance characteristics across parameters.

The method loops over all of the strategy parameters and generates a new trading instance on every simulation. The backtest is then executed and the statistics calculated. These are stored and output into the CSV file. Once the simulation ends, the output file is closed:

```

# backtest.py

..
def simulate_trading(self):
    """
    Simulates the backtest and outputs portfolio performance.
    """
    out = open("output/opt.csv", "w")

    spl = len(self.strat_params_list)
    for i, sp in enumerate(self.strat_params_list):
        print("Strategy %s out of %s..." % (i+1, spl))
        self._generate_trading_instances(sp)
        self._run_backtest()
        stats = self._output_performance()
        pprint.pprint(stats)

        tot_ret = float(stats[0][1].replace("%", ""))
        cagr = float(stats[1][1].replace("%", ""))
        sharpe = float(stats[2][1])
        max_dd = float(stats[3][1].replace("%", ""))
        dd_dur = int(stats[4][1])

        out.write(
            "%s,%s,%s,%s,%s,%s,%s,%s\n" % (
                sp["ols_window"], sp["zscore_high"], sp["zscore_low"],
                tot_ret, cagr, sharpe, max_dd, dd_dur
            )
        )

    out.close()

```

On my desktop system, this process takes some time! 27 parameter simulations across more than 600,000 data points per simulation takes around 3 hours. The backtester has not been parallelised at this stage, so concurrent running of simulation jobs would make the process a lot faster. The output of the current parameter study is given below. The columns are given by

OLS Lookback, ZScore High, ZScore Low, Total Return (%), CAGR (%), Sharpe, Max DD (%), DD Duration (minutes):

```
50,2.0,0.5,213.96,20.19,1.63,42.55,255568
50,2.0,1.0,264.9,23.13,2.18,27.83,160319
50,2.0,1.5,167.71,17.15,1.63,60.52,293207
50,3.0,0.5,298.64,24.9,2.82,14.06,35127
50,3.0,1.0,324.0,26.14,3.61,9.81,33533
50,3.0,1.5,294.91,24.71,3.71,8.04,31231
50,4.0,0.5,212.46,20.1,2.93,8.49,23920
50,4.0,1.0,222.94,20.74,3.5,8.21,28167
50,4.0,1.5,215.08,20.26,3.66,8.25,22462
100,2.0,0.5,378.56,28.62,2.54,22.72,74027
100,2.0,1.0,374.23,28.43,3.0,15.71,89118
100,2.0,1.5,317.53,25.83,2.93,14.56,80624
100,3.0,0.5,320.1,25.95,3.06,13.35,66012
100,3.0,1.0,307.18,25.32,3.2,11.57,32185
100,3.0,1.5,306.13,25.27,3.52,7.63,33930
100,4.0,0.5,231.48,21.25,2.82,7.44,29160
100,4.0,1.0,227.54,21.01,3.11,7.7,15400
100,4.0,1.5,224.43,20.83,3.33,7.73,18584
200,2.0,0.5,461.5,31.97,2.98,19.25,31024
200,2.0,1.0,461.99,31.99,3.64,10.53,64793
200,2.0,1.5,399.75,29.52,3.57,10.74,33463
200,3.0,0.5,333.36,26.58,3.07,19.24,56569
200,3.0,1.0,325.96,26.23,3.29,10.78,35045
200,3.0,1.5,284.12,24.15,3.21,9.87,34294
200,4.0,0.5,245.61,22.06,2.9,12.52,51143
200,4.0,1.0,223.63,20.78,2.93,9.61,40075
200,4.0,1.5,203.6,19.55,2.96,7.16,40078
```

We can see that for this particular study the parameter values of $w_l = 50$, $z_h = 3.0$ and $z_l = 1.5$ provide the best Sharpe ratio at $S = 3.71$. For this Sharpe ratio we have a total return of 294.91% and a maximum drawdown of 8.04%. The best total return of 461.99%, albeit with a maximum drawdown of 10.53% is given by the parameter set of $w_l = 200$, $z_h = 2.0$ and $z_l = 1.0$.

16.3.3 Visualisation

As a final task in strategy optimisation, we are now going to visualise the performance characteristics of the backtester using Matplotlib, which is an extremely useful step when carrying out initial strategy research. Unfortunately we are the situation where we have a three-dimensional problem and so performance visualisation is not straightforward! However, there are some partial remedies to the situation.

Firstly, we could fix the value of one parameter and take a "parameter slice" through the remainder of the "data cube". For instance we could fix the lookback window to be 100 and then see how the variation in z-score entry and exit thresholds affects the Sharpe Ratio or the maximum drawdown.

To achieve this we will use Matplotlib. We will read the output CSV and reshape the data such that we can visualise the results.

Sharpe/Drawdown Heatmap

We will fix the lookback period of $w_l = 100$ and then generate a 3×3 grid and "heatmap" of the Sharpe ratio and maximum drawdown for the variation in z-score thresholds.

In the following code we import the output CSV file. The first task is to filter out the lookback periods that are not of interest ($w_l \in \{50, 200\}$). Then we reshape the remaining performance data into two 3×3 matrices. The first represents the Sharpe ratio for each z-score threshold combination while the second represents maximum drawdown.

Here is the code for creating the Sharpe Ratio heatmap. We first import Matplotlib and NumPy. Then we define a function called `create_data_matrix` which reshapes the Sharpe Ratio data into a 3×3 grid. Within the `__main__` function we open the CSV file (make sure to change the path on your system!) and exclude any lines not referencing a lookback period of 100.

We then create a blue-shaded heatmap and apply the correct row/column labels using the z-score thresholds. Subsequently we place the actual value of the Sharpe Ratio onto the heatmap. Finally, we set the ticks, labels, title and then plot the heatmap:

```
#!/usr/bin/python
# -*- coding: utf-8 -*-

# plot_sharpe.py

import matplotlib.pyplot as plt
import numpy as np

def create_data_matrix(csv_ref, col_index):
    data = np.zeros((3, 3))
    for i in range(0, 3):
        for j in range(0, 3):
            data[i][j] = float(csv_ref[i*3+j][col_index])
    return data

if __name__ == "__main__":
    # Open the CSV file and obtain only the lines
    # with a lookback value of 100
    csv_file = open("/path/to/opt.csv", "r").readlines()
    csv_ref = [
        c.strip().split(",")
        for c in csv_file if c[:3] == "100"
    ]
    data = create_data_matrix(csv_ref, 5)

    fig, ax = plt.subplots()
    heatmap = ax.pcolor(data, cmap=plt.cm.Blues)
    row_labels = [0.5, 1.0, 1.5]
    column_labels = [2.0, 3.0, 4.0]

    for y in range(data.shape[0]):
        for x in range(data.shape[1]):
            plt.text(x + 0.5, y + 0.5, '%.2f' % data[y, x],
                    horizontalalignment='center',
                    verticalalignment='center',
                    )

    plt.colorbar(heatmap)

    ax.set_xticks(np.arange(data.shape[0])+0.5, minor=False)
    ax.set_yticks(np.arange(data.shape[1])+0.5, minor=False)
    ax.set_xticklabels(row_labels, minor=False)
    ax.set_yticklabels(column_labels, minor=False)

    plt.suptitle('Sharpe Ratio Heatmap', fontsize=18)
    plt.xlabel('Z-Score Exit Threshold', fontsize=14)
```

```
plt.ylabel('Z-Score Entry Threshold', fontsize=14)
plt.show()
```

The plot for the maximum drawdown is almost identical with the exception that we use a red-shaded heatmap and alter the column index in the `create_data_matrix` function to use the maximum drawdown percentage data.

```
#!/usr/bin/python
# -*- coding: utf-8 -*-

# plot_drawdown.py

import matplotlib.pyplot as plt
import numpy as np

def create_data_matrix(csv_ref, col_index):
    data = np.zeros((3, 3))
    for i in range(0, 3):
        for j in range(0, 3):
            data[i][j] = float(csv_ref[i*3+j][col_index])
    return data

if __name__ == "__main__":
    # Open the CSV file and obtain only the lines
    # with a lookback value of 100
    csv_file = open("/path/to/opt.csv", "r").readlines()
    csv_ref = [
        c.strip().split(",")
        for c in csv_file if c[:3] == "100"
    ]
    data = create_data_matrix(csv_ref, 6)

    fig, ax = plt.subplots()
    heatmap = ax.pcolor(data, cmap=plt.cm.Red)
    row_labels = [0.5, 1.0, 1.5]
    column_labels = [2.0, 3.0, 4.0]

    for y in range(data.shape[0]):
        for x in range(data.shape[1]):
            plt.text(x + 0.5, y + 0.5, '%.2f%%' % data[y, x],
                    horizontalalignment='center',
                    verticalalignment='center',
                    )

    plt.colorbar(heatmap)

    ax.set_xticks(np.arange(data.shape[0])+0.5, minor=False)
    ax.set_yticks(np.arange(data.shape[1])+0.5, minor=False)
    ax.set_xticklabels(row_labels, minor=False)
    ax.set_yticklabels(column_labels, minor=False)

    plt.suptitle('Maximum Drawdown Heatmap', fontsize=18)
    plt.xlabel('Z-Score Exit Threshold', fontsize=14)
    plt.ylabel('Z-Score Entry Threshold', fontsize=14)
    plt.show()
```

The heatmaps produced from the above snippets are given in Fig 16.3.3 and Fig 16.3.3:

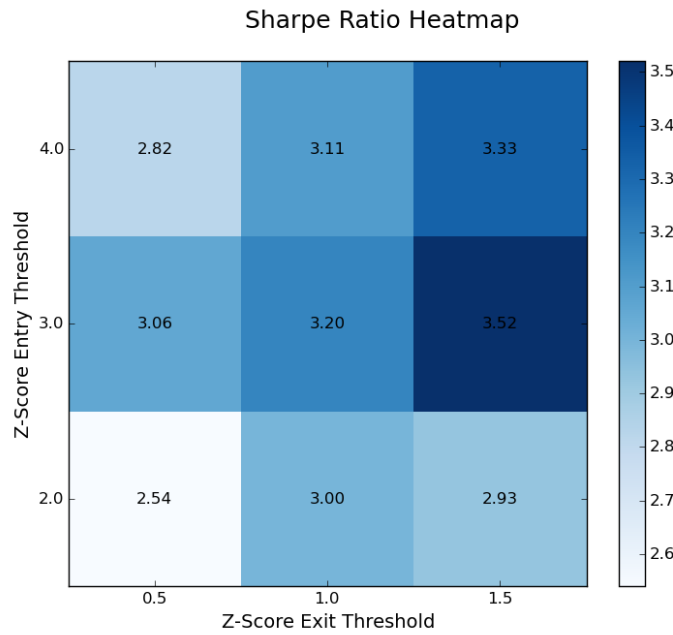


Figure 16.1: Sharpe Ratio heatmap for z-score entry/exit thresholds

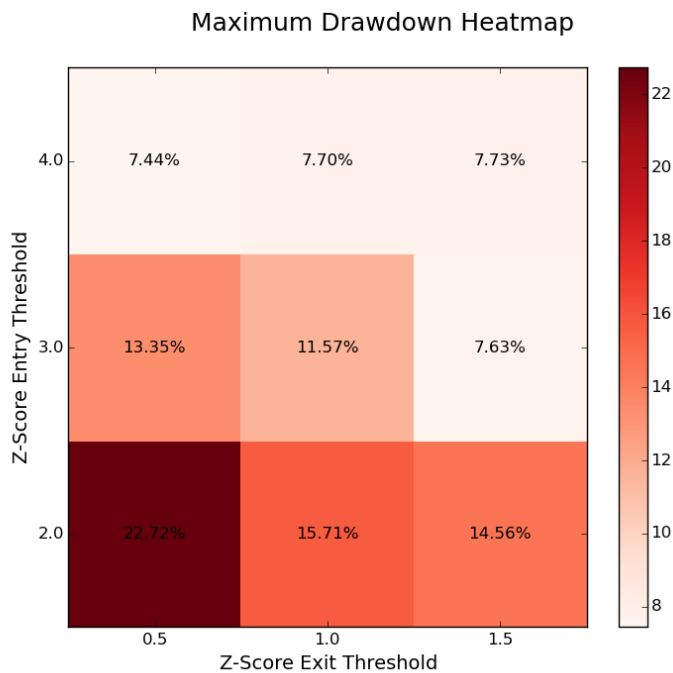


Figure 16.2: Maximum Drawdown heatmap for z-score entry/exit thresholds

At $w_l = 100$ the differences between the smallest and largest Sharpe Ratios, as well as the smallest and largest maximum drawdowns, is readily apparent. The Sharpe Ratio is optimised for larger entry and exit thresholds, while the drawdown is minimised in the same region. The Sharpe Ratio and maximum drawdown are at their worst when both the entry and exit thresholds are low.

This clearly motivates us to consider using relatively high entry and exit thresholds for this strategy when deployed into live trading.